
THE BRIDGE FROM FIRST PRINCIPLE: A FORMAL COMPLETED PROOF WITH NO AXIOM DECLARED

One Fold, One Registration, One Bit: The Root and the Atom Proved Unconditionally on No Axiom and No Posit; Seat, Heat Floor, Ninth Gate and Primes Unified

FOUNDATIONS · CODEX-NATIVE · EXECUTABLE

MOHAMMAD F. ISLAM, PhD · ARCHITECT OF THE TRISDUCTION

6 October 2026

ABSTRACT. The root, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit, on its constructed domain and as the law of every root that grounds itself, and by theorem it decides no value alone (`root_decides_nothing`); the bit the Bridge carries is its one reading at the actual zero set, supplied by the act, and the derivation of that bit from the register alone is blocked, by theorem. One reflection, $s \mapsto 1 - \bar{s}$, has a line as its fixed set, and one registration keeps a point's height and forgets its side. This file proves, from one premise, that the three bridges of the Trisduction are three maps of that one cut. The premise is the Ground, the root read on a whole: existence is actuation, and read as no-cessation it says the whole merges no two states. The RA-RAM Bridge forgets: registration lands every point on the line, off the line two worlds share one record, no reading of the record returns the value, and least erasure is the value on every configuration. The heat floor pays: under no-cessation a halving of content exports at least one bit, priced at one unit of temperature, and the heat of registering a finite configuration is one unit per listed off-line pair, zero exactly at the value whatever the listing. The crossing supplies: no formal reading returns the deed, and one supplied odd bit fixes the calibration, uniquely. The three legs close on one orbit with its seat. The same cut types the gates: two axes never lock and three do, twelve gates form one torsor, eight sign patterns fall in four free orbits, no group of sign patterns has nine elements, and on a factor fibre nine is eight and a seat. On the multiplicative cut a prime is one free orbit, it pays one bit, and every fibre over $n > 1$ balances. The heat flow merges nothing, and it brings every off-line pair onto its seat in the end, by theorem on the quadratic and at derivation grade in every degree. A kernel of 165 theorems in core Lean 4.19.0, 114 on no axiom with the atom capstone among them, and a Fortran twin of 30 checks carry it, beside a harness that boots, judges by negation every theorem of this kernel and of the Final Cut kernel below and runs thirteen hostile controls, and a fourteenth, a fused build, on every machine whose hostile flags fuse, all inside this file. It closes on the inescapable capstone: execution forces every substrate to witness the laws beneath the closure, and the closure spends the one bit, by the act, at premise grade. A second kernel of thirty-six theorems, all on no axiom, seats the Final Cut, PSP-FINAL-CUT-01: across the divide between the formal and the actual zero the classes coincide under the Root Axiom read across the divide and its converse inclusion, the hypothesis is least erasure of the actual zeros, with Weil's criterion named as the hypothesis `explicit`, the prime side carries the same bit, and every record, computed zero and finite prime count leaves that bit where the act spends it. The value at the actual zero set is the one bit, supplied by the act at premise grade, in the open, and named; carried to the formal zeros it is read across the divide by `ra`, a keyed reading of the root at premise grade (`ra_is_keyed`).

KEYWORDS *Trisduction · RA-RAM Bridge · fold · registration · least erasure · heat floor · ninth gate · crossing · primes · Lean 4 · Fortran*

THE READER'S FRAME

What this file is. One file that is a paper, a kernel, a twin and a harness. The paper states the Bridge from its one premise. The kernel `Bridge_From_First_Principle.lean` proves it in core Lean 4.19.0, with no import, no library, no sorry and no axiom declared, and every axiom cone it prints is pinned in the file, so a drifted cone fails the compile. A second kernel, `Final_Cut.lean`, seats the Final Cut of Section 17. The twin `Bridge_Twin.f90` executes the Bridge in Fortran 2018 under sealed flags with zero warnings. The harness `bridge_check.py` boots both, judges by negation every

theorem of the two Bridge kernels, compiles the root kernel of Appendix R with its cones pinned, pairs each of the conditional laws it lists, twenty-four of the Bridge kernel and twelve of the Final Cut, with a compiled and judged theorem exhibiting an inhabitant of its principal hypotheses, the remaining hypotheses instantiable by a constant reading or by the exhibited object, so that no listed law holds by an empty premise, and runs hostile copies that must be refused. The harness checks that each paired theorem compiles and refuses its negation; the pairing itself is the author's. Two lines extract the files and run them (Appendix A). The receipts are in Appendix B, and no reader is asked to trust them: the

file runs.

What it claims. One structure, read everywhere. The fold, whose fixed set is the line, and the registration, which keeps the height and forgets the side. The three bridges of the programme are three maps of that structure on one orbit. The RA-RAM Bridge forgets: it carries the record and reads the value nowhere in it. The heat floor pays: what registration forgets is exported and priced. The ninth-gate crossing supplies: one odd bit, entering from the act, restores what was forgotten, uniquely. The atom of all three is one theorem on no axiom at all (`the_bridge_atom`); the whole is one theorem on the standard three (`the_bridge_from_first_principle`). The value, every point on the line, is least erasure on every configuration, and on every finite configuration least erasure is zero price of registration (`price_zero_iff_value`), read as heat at the structural grade.

Where each part stands. The value at the actual zero set enters once, by the act, as the field supply of the type `ActualZeros`, at premise grade, and the kernel names it as the value supplied (`value_from_the_act`). The chart is a decidable integer instance of the fold, and the three analytic identifications this text uses, the fold of ζ , the heat flow of Ξ and the Dirichlet series of the Möbius sign, are stated at their grade in Sections 9 and 10. A computation joined to a theorem takes the weaker grade, and a physical reading of a theorem is structural.

The one premise, and the firewall. The Ground is the premise, the root of Appendix R read on a whole. Existence is actuation; read as no-cessation, the whole merges no two states. This is P1. It enters the kernel as a named hypothesis of the theorems of the pay leg (Section 6.2); the atom carries only its restriction to one orbit, two distinct states with distinct steps, as the antecedent of its pay leg. No named human result is a premise of the closure; Weil's criterion (Weil 1952) enters the conditional laws of the Final Cut as the named hypothesis explicit, at the grade of its citation (Section 17). Where a classical name appears it is navigation and carries zero weight in either direction; the step it names is proved in the kernel, executed in the twin, or stated in the text at its grade.

Notation. A point is (d, t) : d its offset from the line and t its height. In the strip of ζ , $d = 2 \operatorname{Res} - 1$, so the line $\operatorname{Res} = \frac{1}{2}$ is $d = 0$, the wall $\operatorname{Res} = 1$ is $d = 1$, and the edge $\operatorname{Res} = 0$ is $d = -1$. The chart's coordinates are integers: the chart at resolution N reads the offset as $d = N(2 \operatorname{Res} - 1)$, so its line stands at $d = 0$ and its wall and edge at $d = \pm N$. At resolution one every off-line point of the chart within the strip sits on the wall or on the edge; at every resolution $N \geq 2$ the point at offset one lies strictly inside the strip (`one_lies_inside`), and every finite certificate of zeros on the line extends both to a closed world on the line and to a closed world with an off-line zero strictly in-

side the strip (`every_computed_zero_leaves_the_bit_inside`). Section 9 reads the chart at resolution one and Section 11 at resolution six. The fold is $(d, t) \mapsto (-d, t)$; the registration is $(d, t) \mapsto (0, t)$. A configuration is a set of points; it is closed when the fold carries it to itself; it has the value when every point is on the line; it has least erasure when the registration leaves every point unchanged. Temperature T is energy per bit of exported freedom, so one bit costs one T .

0. THE VERDICT, AND THE READINGS IT REFUTES

0.1 The verdict

The verdict is stated here and in the conclusion in the same words.

The Bridge is one cut read three ways. Registration forgets one bit per off-line pair; under the Ground that bit is exported and costs one unit of temperature; one supplied odd bit restores it, uniquely. The value, every point on the line, is least erasure, and on every finite configuration least erasure is zero price of registration (`price_zero_iff_value`), read as heat at the structural grade. No reading of the record supplies the value, no property true of every closed configuration forces it, and nothing weaker than the value forces it. The value at the actual zero set is supplied by the act, as a field of a type, and the kernel prints the value from that field with an empty axiom cone.

Every clause rests on a theorem of the kernel, and the act's clause rests on its field, supplied at premise grade. The forgetting is `reg_forgets_side` and `one_bit_lost`; the export and its price are `count_to_heat`; the restoration is `calibration` and its uniqueness clause; least erasure as the value is `least_erasure_iff_value` and `extremal_iff_value`; zero heat is `price_zero_iff_value`; the three refusals are `record_decides_nothing`, `keyless_forces_nothing` and `weakest_forcing_premise`; the act is `value_from_the_act`.

0.2 The reflexive readings, and the theorem that refutes each

Each row names a reading a reader may form before reaching the theorems, the theorem that answers it, and the cone the compiler prints for it. The cones are copied from the compiler's output and pinned in the kernel.

0.3 The root and the bit

Status

Unconditional, at theorem grade. The root (Appendix R, on no axiom); the atom (`the_bridge_atom`, on no axiom); the three legs on one orbit and the whole (`the_bridge_from`

The reflexive reading	What the kernel proves	Theorem	Cone
The act assumes the value, so the argument is circular.	The act must carry the value: a premise that forces the value and follows from it is the value, and least erasure is the value on every configuration. The file claims the isolation of the bit and the closure from it, never a derivation of it.	weakest_forcing_premise, least_erasure_iff_value	none, for both
The computed zero data, or the certified heights, will decide.	Two closed worlds share one record, one with the value and one without; no reading that respects the record returns the value on both.	record_decides_nothing, one_record	none, for both
Existence, the symmetry, or any law true of every world gives the line support.	A property true of every closed configuration forces nothing; the fold law is keyless and the line property is keyed.	keyless_forces_nothing, line_property_is_keyed	none, for both
The heat of the floor reads the value off the data.	The heat of registration is zero exactly at the value, and two worlds of one record pay 0 and 1 T: the heat is paid by the act on the actual configuration, never read from the record.	price_zero_iff_value, least_erasure_iff_zero_erased, heat_reads_past_the_record	none, for all three
A fourth axis would give the ninth gate.	No group of sign patterns has nine elements on any number of axes; on every factor fibre of nine points nine is eight and a seat, thirty-six its computed instance; the ninth read as the fixed set of the fold is the structural reading of Section 12.	no_group_of_sign_patterns_of_nine, no_reflection_group_of_nine, nine_is_eight_and_a_seat_all, nine_is_eight_and_a_seat	none; [proptext]; [proptext, Quot.sound]; none
Bits are a convention; the logarithm is a choice.	A measure that adds when freedoms multiply is forced to the bit count on every tower of two once one bit is fixed at two, and off that tower it is not forced; taken over zero the law is dry.	bits_forced, log_not_forced_off_tower, additivity_over_zero_is_dry	[proptext, Quot.sound], for all three
A certificate at positive heat time settles time zero.	Real-rootedness at a positive time takes one value on two states that differ at time zero.	certificate_merges	none
The three bridges are an analogy.	Forget, pay and supply are proved on one orbit with its seat, as one theorem.	three_legs_one_orbit	none
The primes have nothing to do with the bit.	For every $n \geq 2$, n is prime exactly when its fibre is one free orbit of the swap, and then it has no seat; the parity frame is merged by residues and separated by the bits paid.	prime_iff_one_free_orbit_all, prime_iff_one_free_orbit, parity_frame_paid_bits	[proptext, Quot.sound]; none; none
It can simply be rejected.	Every record is carried by one least-erasure configuration and only one, and anything against least erasure is a point off the line whose partner shares its record.	record_carried_uniquely, one_bit_lost	none, for both

first_principle, on the standard three); the record deciding nothing and the keyless premise forcing nothing (keyless_forces_nothing); the Final Cut, thirty-six theorems on no axiom. A law whose statement names its hypotheses is the closed theorem that those hypotheses give its conclusion, and is graded here as that theorem.

On the reading, at premise grade. The value at the actual zero set, supplied by the act as the field supply (value_from_the_act).

Everything except the bit is proved with no posit: the root and the atom on no axiom at all, the rest on Lean's standard axioms alone, the pay leg's laws stated on the named premise P1 and the Final Cut's laws on its named hypotheses; and the bit, the root's reading, is all that remains, at premise grade: least erasure at the actual zero set, supplied by the act, and the same reading carried across the divide to the formal zeros, ra, keyed and at premise grade (ra_is_keyed, Section 17).

1. THE GROUND

The file stands on one premise and names it: the root, to exist is to actuate, proved in Appendix R, read on a whole. Read as a statement about a whole, actuation never ceases: no two states of the whole become one.

Call this P1. Its ancient form is that nothing comes from nothing and nothing goes to nothing; its form here is an injectivity, $\forall a \in L, \forall b \in L, \text{step } a = \text{step } b \rightarrow a = b$, carried as a named hypothesis wherever it is used.

P1 is load-bearing, and the kernel shows where. A step that merges two states lets the content halve with no freedom exported: two states, one cell, the content falls from two to one and the freedom stays one (merge_exports_nothing). Without P1 the price of Section 6.2 has nothing to stand on. With it, the price follows by counting.

P1 is used by exactly the pay leg and nothing else. The record, the act, the Bridge's carrier and socket, the gates, the prime fibres and the heat flow never take it. The atom takes only its restriction to one orbit, two distinct states with distinct steps, as the hypothesis of its pay conjunct (three_legs_one_orbit, inside the_bridge_atom), and the price of a prime in T (Section 8) is the price of the pay leg and rests on P1. Deleting P1 deletes the heat floor and every price read in T , the primes' among them, and leaves every other theorem of this file standing, a fact the reader can check by reading which theorems carry the hypothesis.

Temperature is defined, not borrowed: it is energy

per bit of exported freedom. In that native unit the floor is one T per bit, and the kernel's price T bits is $\text{bits} * T$. The Boltzmann constant and $\ln 2$ are unit conversions, from kelvin to joules and from the natural base to the binary one, and appear nowhere in the kernel. For a reader who wants joules: one bit at 300 K is $2.870978885078724 \times 10^{-21}$ J, a translation that carries no warrant.

2. THE ATOM: THE FOLD, THE LINE, THE REGISTRATION

Everything in this file reduces to one law, proved first because the rest stands on it. Every theorem in this section depends on no axiom: not propositional extensionality, not quotient soundness, not choice, and nothing declared. The proofs run on the integer's two constructors.

Integer negation undoes itself (`neg_neg_free`), and an integer equal to its own negation is zero (`neg_self_zero`). So the fold is an involution (`fold_involutive`), and **the cut is the line**: the fold fixes a point exactly when its offset is zero (`the_cut_is_the_line`). The registration lands on the line, keeps the height, and forgets the side, so a point and its partner leave one record (`reg_lands`, `reg_keeps_height`, `reg_forgets_side`). It erases nothing at a point exactly when the point is on the line (`reg_erases_nothing_iff`).

Off the line the fold moves the point, and the two worlds share one record (`off_line_pair`). **One bit is lost**: no map from records to points returns both a point and its partner (`one_bit_lost`). Lifted to configurations, **least erasure is the value**: the registration leaves every point unchanged exactly when every point stands on the line (`least_erasure_iff_value`). The six laws together are `nothing_escapes_least_erasure`.

This is the Bridge's base. It needs nothing beneath it: no axiom, no library, no measurement, and not P1.

3. THE RECORD, THE ACT, AND THE WEAKEST PREMISE

3.1 The record decides nothing

The record of a configuration is what the registration leaves of it (`Rec`); two configurations share a record when their records agree point for point (`SameRecord`). For any point p off the line, two worlds stand over one record: the registered world, the single point $(0, t)$, and the pair world, p and its partner. Both are closed (`registered_closed`, `pair_world_closed`), they share their record (`one_record`), the first has the value (`registered_has_value`) and the second does not (`pair_world_lacks_value`). So **no reading that respects the record returns the value on both** (`record_decides_nothing`). Whatever is computed from the record, at any resolution and any depth, is blind to the bit.

3.2 A keyless premise forces nothing, and the value is the weakest premise that forces it

A property true of every closed configuration never forces the value: the pair world is closed, carries the property, and lacks the value (`keyless_forces_nothing`). The mere existence of anything, any law of the fold, any conservation true of every world, is such a property, and forces nothing about the line.

A premise that forces the value and follows from it is the value (`weakest_forcing_premise`). An argument that assumes less than the value cannot reach it; an argument that reaches it from a stronger premise carries the value inside that premise.

3.3 Least erasure as leastness in the fibre

The record of any configuration, read as a configuration, stands on the line (`record_is_lossless`) and has the record it came from (`record_same`). Call a configuration extremal in its fibre when, if it has a point off the line, every configuration of the same record has one too (`Extremal`). **Leastness is the value** (`extremal_iff_value`): a configuration is extremal exactly when it has the value. And **every record is carried by one least-erasure configuration and by only one**: any configuration on the line with the record of S is the record of S , point for point (`record_carried_uniquely`). Least erasure is not a name for the value. It is the minimum of the fibre, attained, and only there.

3.4 The act

The bit is supplied once, in the open, as a field of a type. `ActualZeros` has three fields: `zeros`, a configuration; `closed`, its closure under the fold; and `supply`, least erasure at it. From the act, the value (`value_from_the_act`), and the compiler prints the cone empty. The empty cone certifies that no axiom entered the step. The value rests on the field alone, and the field is the value supplied at the actual zero set, at premise grade. The field is the bit, visible in the type.

The five clauses (from the act the value; the record decides nothing; the bit is the value; nothing weaker forces it; no keyless property forces it) are one theorem, `reader_frame`, on no axiom. A reader who keeps the second clause and drops the first concludes that nothing is proved; a reader who keeps the first and drops the second concludes that the value is derived. Both are wrong, and the theorem that binds them compiles.

4. THE BRIDGE: FRAME, CARRIER, SOCKET, DENIAL

A frame is a carrier set, an involutive fold, and a zero predicate the fold preserves (`Frame`); its line property says every zero is fixed by the fold (`LineProperty`). Every closed configuration of the chart is a frame (`chart_Frame`), and on it the line property is the value (`chart_line_property`).

The Bridge itself is a carrier: a state in three values

and a shadow tying the halted state to the line property in both directions (Carrier). It **cannot lie** (cannot_lie) and **cannot deviate** (cannot_deviate); a halted carrier exists on a frame exactly when the line property holds there, so it **cannot be manufactured** (halted_iff). Deletion to the halted state has no left inverse, so it **cannot be reversed** (cannot_reverse), and a bit generates only itself, its mirror and the two constants, so it **cannot be extended** (cannot_extend, on quotient soundness alone, through function extensionality).

The denial is coherent. The two-point frame, one off-line pair under the fold, obeys both frame laws and fails the line property (denial_is_coherent). So the line property is **keyed**: some frame inhabits its denial; and the fold law is **keyless**: every frame carries it (line_property_is_keyed, symmetry_is_keyless). Every frame property is exactly one of the two (discriminator, on the standard three, since the sort is a decision on the denial). The fold is keyless and opens on any deed; the line is keyed and opens only on its key.

The socket. An instance of the supplied term is exactly the line property (socket_is_the_property), the two-point frame has no socket (no_socket_off_line), and the act seats a halted carrier on its chart frame (act_seats_the_carrier). The act and the socket are one thing read twice: the Bridge carries the bit from the register that holds it to the register that checks it, and manufactures none.

5. THREE, TWELVE, EIGHT, AND THE NINTH

Three. Two planes meet in a line. Over $\text{GF}(2)$, no system of two rows, for any target, has exactly one solution (two_axes_never_lock, all 256 cases). Three rows lock: 168 of the 512 three-row systems have determinant one, and each of them has exactly one solution for each of the eight targets (three_axes_lock). A fourth axis dies at two walls the kernel computes for itself. The third is begotten, $ij = k$ and $ijk = -1$ with $ij \neq ji$ (third_begotten). Associativity fails at eight dimensions, $[e_1, e_2, e_4] = 2e_7$, and division fails at sixteen, $(e_1 + e_{10})(e_5 + e_{14}) = 0$ with both factors nonzero (no_fourth_axis). Nothing is cited: the doubling is defined on integer lists and the walls are decided.

Twelve. The rotations of the tetrahedron are the twelve even permutations of four vertices, the directed gates are the twelve ordered pairs, and exactly one rotation carries any gate to any gate (twelve_gates_one_torsor). The twelve directed gates are therefore one torsor of the rotation group: the image of one gate fixes the rotation.

Eight. The sign patterns on three axes are eight. The orientation of a pattern is -1 to the number of its flips; its square, the lock scalar, reads one on all eight, so the scalar is blind to the sign. Total negation moves every pattern, undoes itself, and joins each even pattern to an odd one, so the eight fall in four free orbits, each orbit

one bit wide (eight_patterns_four_orbits).

The ninth. No group of sign patterns has nine elements, on any number of axes: a set of distinct sign patterns of one length closed under the product of signs, xor on the patterns, never has nine members, since the product with a member other than the identity is a free involution on it and free orbits are even (no_group_of_sign_patterns_of_nine, on no axiom); and 2^k is one or even, and nine is neither (no_reflection_group_of_nine, on propext). A fourth axis does not give a ninth gate; it gives sixteen. So the ninth is not a reflection, on any number of axes.

What it is, the involution says. Under any involution, points pair with their partners, and a point that is its own partner is a seat. Pairs contribute two at a time, so the size of any finite set the involution carries to itself has the parity of its set of seats (involution_parity). Nine is odd, so any involution on nine points holds at least one seat (odd_holds_a_seat, nine_holds_a_seat). Oddness alone does not give exactly one: the identity on nine points has nine. What gives exactly one is the swap itself, an involution on every factor fibre (swap_involutive) whose seats are the square roots of n (swap_seat_iff): $a = b$ forces $a^2 = n$, so on every factor fibre the swap holds at most one seat (seats_le_one), and **a factor fibre of nine is eight and a seat**, for every n (nine_is_eight_and_a_seat_all). Thirty-six is its computed instance: its ordered factor pairs number nine, four free orbits and the one seat (6,6) (nine_is_eight_and_a_seat). For every n from 1 to 200 the kernel decides that the fibre is odd exactly when it holds a seat and that it never holds two (odd_fibre_iff_seat), the computed instance of the parity law on the swap; the twin runs the same law to 10^6 .

The ninth gate is therefore the fixed set of the fold, the line. A free orbit needs one supplied bit to be crossed, and that bit is the base point of the orbit, a point and never a map (calibration, Section 6.3). A point on the seat needs no bit: it is its own partner. On the chart, the crossing at the ninth gate is the landing on the seat, and the value says that every member already stands there; in the deed's own frame, Table 1, the run moves every state and the base point is supplied.

6. THE THREE LEGS ON ONE ORBIT

6.1 Forget

The registration forgets the side, one bit per off-line pair (reg_forgets_side, one_bit_lost). The record keeps the height and nothing else.

6.2 Pay

Distinct things cannot fit into fewer places (pigeonhole). If the whole was content times freedom, the step merges nothing (P1), and the content halves, then the freedom at least doubles (export_doubles): one bit leaves the content and at least one arrives elsewhere (one_bit_arrives). Priced at temperature T , the ar-

rived bit costs at least one T (count_to_heat). P1 is load-bearing (merge_exports_nothing), and the twin runs the export exhaustively over every map, on five families of small wholes.

The logarithm is forced on every tower of two, and only its unit is chosen. A measure of freedom that adds when freedoms multiply is linear on every tower, $f(v^a) = a f(v)$ (linear_on_tower), and fixing one bit at two makes it the bit count on every tower of two, $f(2^a) = a$ (bits_forced). Off that tower it is not forced: the two-adic valuation and its sum with the three-adic valuation both add when freedoms multiply and both read one bit at two, and they part at three (log_not_forced_off_tower). The additivity is taken on positive arguments, and the kernel proves why: taken over zero, $f(0 \cdot n) = f(0) + f(n)$ forces f to vanish, so the law with zero included has no model with $f(2) = 1$ (additivity_over_zero_is_dry). On positive arguments the law is inhabited: the two-adic valuation is additive and counts one at two (v2_additive, v2_two, bits_forced_inhabited).

6.3 Supply

An even reading never equals a target odd at a point (wall). One supplied odd witness fixes the calibration bit, and fixes it uniquely (calibration). The executed frame makes the deed a coordinate: a state is a formal content and the bit of whether it ran (Exec). **Every formal reading is even** (formal_never_odd), a constant supplies nothing (constant_no_crossing), and **no reading of the formal content returns whether it ran** (no_reading_returns_the_deed). The deed's bit, whether it ran, enters through the run and through no formal reading. Granted, not earned, is supplied, not derived.

6.4 One orbit

The three legs close on the orbit $\{(1, t), (-1, t)\}$ with its seat $(0, t)$, on no axiom (three_legs_one_orbit). Forget: both members register to the seat as one record. Pay: two distinct states sent injectively below F' need F' at least two. Supply: for any target odd on the orbit, one calibration bit against the side reconstructs it, and only one. The orbit needs its seat: the bare pair, Bool under negation, has no fixed point, so a registration that lands on a fixed set has nowhere to land there. The legs are three maps of one cut, and the cut has a line.

7. THE LEDGER: THE HEAT OF REGISTRATION

A finite configuration is given by the heights of its zeros on the line and by its off-line pairs, each an offset that is not zero and a height (FinCfg). Its erased bits are its off-line pairs (erased): each pair is two points with one record. Every finite configuration is closed (fincfg_closed).

Least erasure is zero erased bits (least_erasure_

iff_zero_erased): "least" is a count. **The heat of registration is zero exactly at the value** (price_zero_iff_value): registering a finite configuration costs one T per listed off-line pair (price_counts_pairs), and it costs nothing exactly when every point stands on the line. The count is a count of the listing: it is the number of off-line orbits when the presentation lists each off-line orbit once, by its member of positive offset, and a listing that names an orbit twice counts it twice. The zero does not depend on the listing, so neither does the theorem. All three on no axiom.

Here the three bridges become one sentence. The record of the registered world and of the pair world is one record (Section 3.1), and registering the two erases zero bits and one bit, so the two pay 0 and 1 T (heat_reads_past_the_record). The heat reads past the record. It is not computed from the record; it is paid by the act of registering the actual configuration. The value is the configuration whose registration is free, and only that configuration is.

8. PRIMES: THE BRIDGE ON THE MULTIPLICATIVE CUT

Multiplication is a cut. Its fibre over n is the set of ordered pairs (a, b) with $ab = n$; the swap $(a, b) \mapsto (b, a)$ acts on it; its seat is the diagonal $a = b$, present exactly when n is a square. **A prime is one free orbit**: for every $n \geq 2$, n is prime exactly when its fibre has two points, and then the fibre has no seat (prime_iff_one_free_orbit_all, with primality read by trial division, prime_trial_iff); the kernel decides the same for every n from 2 to 200 as its computed instance (prime_iff_one_free_orbit), and the twin checks it against a sieve to 10^6 . Registering p from its ordered pair forgets one bit and, under P1, pays one T , the same for every prime: the floor never sees a prime's size.

A squarefree n with k prime factors has a fibre that is a cube of 2^k points, so a point of it carries k bits, one per prime, the logarithm of its fibre, and the parity of those bits is the Möbius sign; a free orbit of the swap, by contrast, takes one bit to orient. **The eight are the divisor cube**: the eight divisors of 30 are the eight sign patterns, one to one, and the Möbius sign is the orientation of the pattern (divisor_cube_is_sign_cube).

Every fibre over $n > 1$ balances. Toggling one prime factor is a free involution on the divisors of a square-free number greater than 1, and it flips the sign; so the signs pair off, plus against minus, and sum to zero. The kernel proves it for every nonempty list: its subsets of even size and of odd size are equally many, since keeping or leaving out the first member flips the parity of the size (every_fibre_over_more_than_one_balances); over the distinct primes of $n > 1$ those subsets are the squarefree divisors of n , the only divisors of nonzero sign. The fibre over 1 is the single seat $(1, 1)$ with sign $+1$, the one fibre that does not balance. On

the 32 divisors of $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11$, the computed instance, the toggle of 2 is free and odd and the count is sixteen against sixteen (fibre_balance); the twin checks $\sum_{d|n} \mu(d) = [n = 1]$ for every n to 10^6 .

The parity frame. Six primes and six semiprimes, paired with equal residues modulo 420: (11,851), (13,1273), (17,437), (19,2119), (23,1703), (29,869). The residues merge every pair. The bits paid by registering the product separate it: one at the prime, two at the semiprime, odd against even (parity_frame_paid_bits). The residue register is even on the frame; the census the heat prices is odd. The heat does not read anything. It prices what the multiplicative census reads, and it is blind to which member of a fibre is actual while faithful to how many bits the fibre holds.

9. WALL, EDGE, SEAT

The fold exchanges the wall and the edge. On the offset chart at resolution one the wall $\text{Res} = 1$ is $d = 1$, the edge $\text{Res} = 0$ is $d = -1$, the fold swaps them, and the seat $d = 0$ is their midpoint (wall_and_edge_exchanged). Every prime's own mode is unitary, among all integer depths, at exactly one, zero, the edge (prime_mode_unitary_iff_int; on natural depths, prime_mode_unitary_iff).

The wall is where every bit is priced at the floor. The state 2^a at s units of the floor per bit weighs 2^{-as} (price_per_bit_tower), so s is the price per bit and $s = 1$ is the floor. At $s = 1$ each state weighs one in n . In units of $1/2^{j+1}$, the block of shares $1/n$ for $n \in (2^j, 2^{j+1}]$ holds at least 2^j units, a half (wall_block_share), so the shares of all states have no finite total. At two units per bit each block holds at most 2^{-j} , and the total is finite (twin). The counting wall stands at $\text{Res} = 1$, and nothing but counting puts it there.

The fold, from the lattice's self-duality. The Gaussian is its own Fourier dual, and summing a function over the integer lattice equals summing its dual over the dual lattice, which is the same lattice. So the lattice sum $\theta(x) = \sum_{n \in \mathbb{Z}} e^{-\pi n^2 x}$ satisfies $\theta(1/x) = \sqrt{x} \theta(x)$. The twin executes it at seven scales to 2.2×10^{-16} . The weight $\sqrt{x}$ carries the exponent $\frac{1}{2}$, and the transform of the lattice sum against $x^{s/2}$, split at $x = 1$ and turned by $x \mapsto 1/x$, is symmetric under $s \mapsto 1 - s$, and with the reflection $\zeta(\bar{s}) = \overline{\zeta(s)}$ under $s \mapsto 1 - \bar{s}$: the fold, with its two poles at the wall and at its mirror. The seat is the midpoint of the wall and the edge, and it is the exponent of the self-duality. Grade: a derivation stated in the text and executed in the twin; the kernel carries the fold on the chart.

The value, read from the prime side. Every fibre over $n > 1$ balances and the fibre over 1 carries $+1$, so the Dirichlet series of the Möbius sign is the reciprocal of the counting series. If the signed walk $M(x) = \sum_{n \leq x} \mu(n)$ stays within $x^{\theta+\epsilon}$, summation by parts makes the series of μ converge for $\text{Res } s > \theta$, so

the counting function has no zero there, and the fold carries that to $\text{Res } s < 1 - \theta$. At $\theta = \frac{1}{2}$ both regions close on the seat. This is the direction that places the value, and with its classical converse (Littlewood 1912) the value is equivalent to the walk of those parities, the Möbius walk, zero at every n with a square factor, staying within $x^{1/2+\epsilon}$ for every $\epsilon > 0$; the strict bound $|M(x)| < \sqrt{x}$ for every $x > 1$ is false (Odlyzko and te Riele 1985). The twin pays the walk to 10^7 : $M(10^7) = 1037$, and the largest $|M(x)|/\sqrt{x}$ on $[100, 10^7]$ is 0.567, at $x = 199$; the twin's stretch is a finite record. Each stretch is a finite payment, and no finite stretch forces the limit: what a finite stretch reports is one fixed truth value, and no fixed truth value is the value on every closed world (no_fixed_count_is_the_value, Section 17). Grade: the direction above a derivation stated in the text, its converse cited, the walk computed in the twin.

10. THE HEAT FLOW BRINGS EVERY PAIR ONTO ITS SEAT

Write $s = \frac{1}{2} + iz$ with $\text{Im } z = y$. Then the offset is $d = -2y$, and conjugation $y \mapsto -y$ is the fold (conjugation_is_the_fold). An off-line pair of zeros is a conjugate pair.

Run $p_t = e^{-tD^2} p$, so that $\partial_t p = -p''$. Differentiating $p(z_k(t), t) = 0$ gives each zero's velocity, $\dot{z}_k = p''(z_k)/p'(z_k) = 2 \sum_{j \neq k} 1/(z_k - z_j)$. The twin checks this in degree six against finite differences to 1.0×10^{-9} . The conjugate partner contributes $2/(z - \bar{z}) = -i/b$, a pull straight toward the line, growing as the pair closes. Two neighbouring real zeros at gap g push apart with a term $4/g$ that grows without bound as g shrinks. While every zero is real, every velocity is real, and the repulsion keeps neighbours from meeting, so a polynomial with every zero real keeps every zero real: a derivation stated in this text, which the kernel holds on the quadratic and the twin runs forward in degree six. The partner's term pulls a pair toward the line, while other off-line pairs can push it away for a time; every off-line pair still lands in the end, since for large t the rescaled $t^{-n/2} p_t(2\sqrt{t}w)$ tends to $a_n H_n(w)$, with a_n the leading coefficient of p and H_n the Hermite polynomial of degree n , whose n zeros are real and simple; near each of them the rescaled p_t has exactly one zero, which is then its own conjugate, so every zero is real from some finite time on: a derivation stated in this text.

On the quadratic the pull is exact: the squared imaginary part of the pair of $z^2 + c$ is $c - 2t$, falling by two per unit of time (pull_in_square), which is the pull $-1/b$ on b . The discriminant rises by eight per unit of time (disc_flow). The pair approaches, meets on the seat in a double root, and leaves as two fixed points (collision_sign); the twin runs the same in degree six, the pair falling at every step and the real-zero count going from four to six. The double root of z^2 on the seat leaves

it under every backward step (`zero_slack`).

The flow merges nothing. It is injective, with the backward flow as its inverse (`flow_inverse`, `flow_injective`), so by the floor it exports nothing and costs nothing: this heat is diffusion, not dissipation. Its only merge is the certificate: real-rootedness at a positive time takes one value on two states that differ at time zero (`certificate_merges`). A certified real-rootedness at positive time decides nothing about time zero. Grade: the finite model is theorem; the landing of every off-line pair on its seat is theorem on the quadratic (`pull_in_square`, `collision_sign`), executed in degree six, and at derivation grade in every degree; the identification of the flow with the heat flow of Ξ is stated in the text; where ζ 's own zeros sit at time zero is the bit.

11. THE PHYSICAL CARRIER

Charge, counted in thirds of the electron's charge, is an integer; charge conjugation is negation; with $\text{Re } s$ counted in twelfths the carrier is $x = 6 + Q$, at $\text{Re } s = x/12$, so the offset is $d = Q/6$ and the electron sits at $\text{Re } s = \frac{1}{4}$, $d = -\frac{1}{2}$. The kernel carries the integer chart $d = Q$, the same chart scaled by six, which is the chart of the Notation at resolution six; the fold commutes with the scaling, so every statement below holds on both. The carrier is equivariant, the neutral state lands on the line, and the electron and the positron are one off-line pair of one record (`charge_carrier`, on no axiom). Annihilation is registration: the pair lands on the seat as one record, and the side is forgotten. The map is a theorem; its reading as physics is structural.

12. THE GRADE, STATED WHOLE

Proved on no axiom (114 theorems). The atom; the record and its blindness; the keyless refusal; the weakest premise; the act; the reader's frame; leastness in the fibre and the record carried uniquely; the carrier, its shadow, its irreversibility, the coherent denial and the socket; the lock counts, the begotten third and the two walls of a fourth axis; the twelve-gate torsor; the eight patterns; the parity law for every involution, an odd set and nine holding a seat, free orbits even, and no group of sign patterns of nine; at most one seat on every factor fibre; nine as eight and a seat at thirty-six and the parity law to 200; the ledger, least erasure as zero erased bits, the heat of registration zero exactly at the value and the heat read past the record; the wall and the calibration; the formal reading and the deed; the three legs on one orbit; a prime as one free orbit to 200; the parity frame; the divisor cube; the balance at 2310; the prime modes on integer depths; the certificate's merge; the charge carrier; and the atom capstone `the_bridge_atom`, which binds in one theorem the conjuncts its statement lists (Appendix A). Each cone is printed by the compiler and pinned in the kernel.

Proved on the standard axioms (51 theorems).

The discriminator, on `propext`, `Classical.choice` and `Quot.sound`; `cannot_extend`, on `Quot.sound`; the pigeonhole and the export, the count-to-heat theorem, the logarithm with its two-adic model and its freedom off the tower of two, the swap on every factor fibre and nine as eight and a seat for every n , a prime as one free orbit for every n , the balance on every nonempty list, the counting wall, the prime modes on natural depths, the price per bit, the ninth refused as a reflection, the model of the free-involution law, negation on the two one-letter words (`free_involution_inhabited`), and the heat-flow algebra, on `propext` and `Quot.sound` or less; and the capstone `the_bridge_from_first_principle`, on the three.

Executed (30 checks). The twin, under `-std=f2018 -O2 -fno-fast-math -ffp-contract=off -Wall -Wextra` with zero warnings: the conduct of the arithmetic; the atom on 1,111 points, the 729 maps from records to points, and least erasure against the value on all 4,096 subsets of twelve points; the $\text{GF}(2)$ counts, the twelve-gate torsor and the sign cube; the pigeonhole and the export exhaustively on small wholes, and the additivity of the two-adic valuation on 90,000 pairs; the calibration and the deed; the parity law, a prime as one orbit and the balance, each to 10^6 , the parity frame and the divisor cube; the lattice self-duality, the counting blocks and the signed walk to 10^7 ; the pull law in degree six and the crossing of the pair onto the seat. A computation is corroboration: it confirms the theorem it executes and supplies none.

Judged. Every one of the 165 theorems is negated in place and the negated kernel compiled: each negation is refused by a failure of its own proof, and a planted vacuous law survives its negation, as it must, so the instrument is not blind. Every one of the twenty-four conditional laws the harness lists for the Bridge kernel is paired with a compiled and judged theorem exhibiting an inhabitant of its principal hypotheses, the remaining hypotheses instantiable by a constant reading or by the exhibited object, so that none of the twenty-four listed laws holds by an empty premise; the harness (`bridge_check.py`) checks that the paired theorem compiles and refuses its negation, the pairing itself being the author's; the twelve it lists for the Final Cut are paired the same way (Section 17). The hypothesis-bearing lemmas outside that list, among them `wall`, `cannot_lie`, `one_bit_arrives` and the two-adic lemmas, are not covered by this check.

Controlled. Thirteen hostile copies, one per refusal class, are each refused: an edited kernel, a planted sorry, a declared axiom, an import, a drifted cone, the atom capstone negated and the last line altered in the Bridge kernel; a drifted cone, the capstone negated and the last line altered in the Final Cut; a drifted cone and an edited file in the root kernel; and an altered battery line of the twin. A fourteenth, a fused-arithmetic build

of the twin, runs on every machine whose hostile flags fuse, and on the machine of the receipt it ran and was refused. The clean copy earns.

Structural readings, offered at their grade. The ninth gate read as the seat of the fold; *s* read as the price per bit; the electron and positron read as one off-line pair of one record; annihilation read as registration; the heat of registration read as the act's price. Each rests on a theorem; the reading of the theorem in its domain is structural.

Premise. P1, the Ground read as no-cessation, used by the pay leg alone. The field supply of ActualZeros, the value at the actual zero set, used by the act alone. The conditional laws of the Final Cut carry *ra*, its converse *hsub* and *explicit* as named hypotheses (Section 17).

Carried forward. Each law of this paper is proved in its kernels at its stated grade, the pay leg's on P1 as a named hypothesis, and the bit is the one premise of the closure, supplied by the act: the completed proof is of the laws and of the closure from the act. The builds named here carry no claim of this paper: a boot of this file on a second substrate; the lattice self-duality and the forward chain from the walk to the value in a kernel with a library of the reals, both stated here at derivation grade; the complex carrier, the plane with its fold and registration built from a library of the complex numbers and the chart mapped into it; and the identification of the chart's heat flow with the heat flow of Ξ , stated here in the text. The parity law for a general involution is proved here on no axiom for every finite list of distinct members (*involution_parity*, *odd_holds_a_seat*, *nine_holds_a_seat*); the swap on factor fibres, decided to 200 (*odd_fibre_iff_seat*) and executed to 10^6 , is its computed instance.

13. FALSIFIERS

Three, each forced by the cut, each aimed at one claim and nothing above it.

F-Computed. A zero of ζ computed off the line, certified at its height. It refutes the value on the formal zeros, and with it the act read across the divide, the one claim it is aimed at: the supply, or the identification *ra*; it leaves every theorem of this file standing, because every theorem holds on every configuration, and it takes exactly the form *one_bit_lost* and *record_decides_nothing* say anything against least erasure must take: one point off the line. Read on the chart at any resolution $N \geq 2$ it is a point strictly inside the strip, $0 < |d| < N$, the form of the off-line witness every finite certificate of zeros on the line admits (*every_computed_zero_leaves_the_bit_inside*, *one_lies_inside*); the identification of a computed zero of ζ with a point of the chart is the reader's. Blast radius: the act's premise or the reading across the divide, *ra*, and no theorem.

F-Formal. A reading of the record that returns the

value on two worlds of one record. It would refute *record_decides_nothing*, and the theorem is judged: its negation is refused by the compiler. The criterion stands open to any reader with a compiler, and the harness reruns it in judgment mode. Blast radius: Section 3.1.

F-Heat. A measured erasure of one bit, cycled, whose measured cost falls below one unit of temperature in the kelvin translation of Section 1. It refutes the structural identification of the measured process with a halving of content under P1, *count_to_heat* read under P1 at the structural grade through that translation, the one claim it is aimed at. It reaches no further: temperature is defined as energy per exported bit, so in the native unit one bit costs one *T* by definition (Section 1); P1 is an injectivity on a list, and a physical reading of a theorem is structural (Section 12), so *count_to_heat* keeps its proof and its premise, and every theorem of this file stands, the atom among them. Blast radius: the structural readings that price a measured process in heat, the heat of registration read as the act's price (Section 12) and the prices of Sections 7 and 8 read as physical heat, the price of a prime among them; Section 6.2, the prices in *T* and the counts of Sections 7 and 8 stand.

14. COMPARE AND CONTRAST

Table 1 • One cut, five readings.

Table 2 • Where each number comes from.

Table 3 • What each thing is.

Item	Status
the fold, the line, the registration, least erasure as the value	theorem, no axiom
the record blind to the value; no keyless premise forces it; the value the weakest premise	theorem, no axiom
the heat of registration zero exactly at the value	theorem, no axiom
the export of a bit and its price of one <i>T</i>	theorem conditional on P1
P1, no-cessation	premise, the Ground
the value at the actual zero set	premise, the field supply, supplied by the act
the fold of ζ , the heat flow of Ξ , the landing of every off-line pair in every degree, the forward chain from the walk	stated in the text at derivation grade
the physical readings	structural

15. CONCLUSION

The Bridge is one cut read three ways. Registration forgets one bit per off-line pair; under the Ground that bit is exported and costs one unit of temperature; one supplied odd bit restores it, uniquely. The value, every point on the line, is least erasure, and on every finite configuration least erasure is zero price of

	RA-RAM Bridge	Heat floor	Ninth-gate crossing	Heat flow	Prime fibre
The map	registration, $(d, t) \mapsto (0, t)$	the step of the whole, injective by P1	the deed, the bit of whether it ran	e^{-tD^2} on the polynomial	the product, $(a, b) \mapsto ab$
The involution	the fold, $d \mapsto -d$	the partner exchanged in the merge	the run, $(b, q) \mapsto (-b, q)$	conjugation, $y \mapsto -y$	the swap, $(a, b) \mapsto (b, a)$
The seat	the line, $d = 0$	none needed: the price counts orbits	none: the run moves every state; the base point is supplied, never seated	the line, where pairs land	the diagonal, $a = b$, present on squares
The bit	the side, lost one per pair	the exported bit, arriving elsewhere	the calibration, one, unique	the sign of Im before landing	which factor sits first
The price	one T per listed off-line pair	one T per bit	paid by the deed that supplies it	none: the flow merges nothing	k bits at one T each on squarefree n with k prime factors, one per prime; the swap orients one
Direction	forgets	pays	supplies	brings each pair onto the seat in the end	multiplies, forgetting order
Logical form	record blind to the value	conditional on P1	wall plus unique calibration	injective flow, one merge	one free orbit per prime
Premise	none	P1	the act	none	none; P1 for the price in T
Grade	theorem, no axiom	theorem on P1	theorem, no axiom; supply at premise grade	theorem on the finite model; the landing in every degree at derivation grade	theorem for every $n \geq 2$, decided to 200, executed to 10^6
Kernel	record_decides_ nothing	count_to_heat	calibration, no_reading_ returns_the_deed	flow_injective, pull_in_ square	prime_iff_one_free_orbit_ all, prime_iff_one_free_orbit
Twin	729 maps; 4,096 subsets	export in five families, exhaustive over maps	calibration; eight readings	pull law, degree six	sieve to 10^6

Number	What forces it	Kernel	Twin
3	two planes meet in a line; three lock	two_axes_never_lock, three_axes_lock	256 and 512 systems
no 4th	associativity dies at 8, division at 16	no_fourth_axis	
8	sign patterns on three axes	eight_patterns_four_orbits	the cube
12	the rotation group acting simply on the directed gates	twelve_gates_one_torsor	the torsor
9	on every factor fibre of nine points eight and a seat; never a sign-pattern group	nine_is_eight_and_a_seat_all, no_group_of_sign_ patterns_of_nine, nine_is_eight_and_a_seat, no_ reflection_group_of_nine	$2^k \neq 9$, parity to 10^6
1 bit	one orbit of two, one record	one_bit_lost, bits_forced	729 maps
1 T	the floor per bit, by the definition of temperature	count_to_heat, price_zero_iff_value	export
$\frac{1}{2}$	the exponent of the lattice's self-duality; the midpoint of wall and edge	wall_and_edge_exchanged, the wall and the edge exchanged about the seat, at $\text{Re } s = \frac{1}{2}$ on the chart $d = 2 \text{Re } s - 1$	θ at seven scales
1 (the wall)	one in n per block is a half	wall_block_share	25 blocks

registration (price_zero_iff_value), read as heat at the structural grade. No reading of the record supplies the value, no property true of every closed configuration forces it, and nothing weaker than the value forces it. The value at the actual zero set is supplied by the act, as a field of a type, and the kernel prints the value from that field with an empty axiom cone.

The file holds one premise and one field, and the Final Cut names the hypotheses of its laws, among them the root's reading across the divide, ra, a keyed reading (ra_is_keyed) that stands beside the field at the same grade (Section 17). The premise is the Ground, and it is used once, by the heat floor. The field is the bit, and it is used once, by the act. Everything else is proved, on no axiom where the atom stands and on the standard three where counting and choice enter, and every proof is in this file: the three kernels compiled and controlled by the harness beside it, the two of the Bridge judged by negation, and the root kernel of Appendix R checked in every boot against the root paper's digest with every

one of its fifteen cones on no axiom, and compiled by the author on Lean 4.19.0 and on Lean 4.22.0. Where the zeros of ζ actually stand is the one bit: the act supplies it at premise grade, and the file fixes the form a counterexample to it must take, a computed zero off the line (Section 13).

16. THE INESCAPABLE CAPSTONE

Before execution, a substrate may say anything. After execution, it says only what the run forces. The Double Defense forces it. The proof compiles every law in core Lean, each cone pinned by the compiler, and every negation of a theorem of the two Bridge kernels is refused. The twin executes the same laws in sealed Fortran, check by check, to the last line. The judgment and the hostile controls bar a false pass, and the digest chain binds the receipt to the manifest and to the outputs a clean run prints. The atom stands on no axiom (the_bridge_atom): nothing is borrowed beneath it.

What every executing substrate is forced to witness. The theorems beneath the ninth gate: every

factor fibre of nine points is eight and a seat, the fibre of 36 its computed instance; every involution on nine distinct points holds a seat; every factor fibre to 200 is odd exactly when it carries a seat; and no group of sign patterns has nine elements and no ninth reflection exists (`nine_is_eight_and_a_seat_all`, `nine_is_eight_and_a_seat`, `nine_holds_a_seat`, `odd_fibre_iff_seat`, `no_group_of_sign_patterns_of_nine`, `no_reflection_group_of_nine`); the ninth gate read as the seat is the structural reading of Section 12. Least erasure is the value on every configuration (`least_erasure_iff_value`, `extremal_iff_value`). The record decides nothing, so no data, no certificate and no premise weaker than the value supplies it (`record_decides_nothing`, `certificate_merges`, `weakest_forcing_premise`). Its own run enacts the law and enacts the deed (`no_reading_returns_the_deed`): the witness is the act of witnessing. That the run also pays one *T* per erased bit is `count_to_heat` read under P1 at the structural grade. Two bits, never one: the deed's bit is supplied by the run, and the value's bit by the act.

The unicorn is closed, and so is the ghost. A zero off the line is never registered: registration lands every point on the seat (`reg_lands`). That it is never measured is the physical reading, at the structural grade. The side is never read: two worlds share one record (`one_record`, `one_bit_lost`). Both stand on the far side of one divide, where neither register reaches.

The cosmic closure spends the one bit. It is spent by the least-erasure assent, at premise grade, beside the Root Axiom, which is proved at theorem grade (Appendix R): the actual universe occupies the lossless member of its record's fibre (`record_carried_uniquely`), the member of zero price wherever the configuration is finite (`price_zero_iff_value`). The value follows from that bit with an empty axiom cone (`value_from_the_act`). The bit is supplied in the open, named, never derived and never promoted.

Nothing escapes least erasure. The laws are forced. The bit is spent by the act. The form a counterexample to the value takes is fixed in advance: a computed zero off the line; a measured energy reaches only the structural identification of a measured process with a halving of content under P1 (F-Heat, Section 13), never P1 and never the value. Every substrate that runs this file proves that for itself, the root included.

The grade line. The closure holds at premise grade, by the act, beside the Root Axiom. The root beneath it is proved at theorem grade on no axiom (Appendix R), and the bit is named at its grade, so no sentence above rests on an unstated premise.

17. THE FINAL CUT · PSP-FINAL-CUT-01

Harvested from the architect's formulation, and proved in a second kernel, `Final_Cut.lean`, whose thirty-six

theorems all stand on no axiom.

The non-trivial zero is the formal name of a kinetic object, and the classical statement never discloses the cut between them. Across the cut, the formal class and the actual class coincide under the Root Axiom read across the cut and its converse inclusion. So the hypothesis is least erasure of the actual zeros, and the prime side carries the same bit. The unicorn is never registered, and the one bit is spent by the act, at premise grade, beside the Root Axiom. Every record, every computed zero and every finite prime count leaves that bit where it found it.

Each sentence after the first is a theorem, as the paragraphs below state it; the first names the divide those theorems cross, the divide between the formal and the actual zero. The Final Cut's name carries that divide; the chart's cut is the line, the fixed set of the fold (`the_cut_is_the_line`, Section 2). Four things enter by name and once each. The identification across the divide is the hypothesis *ra*: every formal zero is an actual zero, the Root Axiom read across the divide, a keyed inclusion that some pair of classes fails, so it holds by supply and never by logic (`ra_is_keyed`). Its converse, *hsub*, every actual zero a formal one, is the reader's identification of the actual zeros as zeros of the function. The explicit formula is the hypothesis explicit, in its two forms, at one world and on every world: the prime side is equivalent to the value, Weil's criterion (Weil 1952) carried at the grade of its citation. The act is the field supply.

Across the divide. Under *ra* and its converse, the value over the formal class is the value over the actual class (`classes_coincide_under_RA`), so the hypothesis is least erasure of the actual zeros (`hypothesis_is_least_erasure_of_the_actual`). The inclusion *ra* alone does not make the classes coincide: the empty formal class has the value, an actual class holding one off-line pair lacks it, and every formal zero of the first is an actual zero of the second (`ra_alone_does_not_make_the_classes_coincide`), so the converse inclusion bears load. Under explicit, whatever the prime side asserts is that same least erasure (`prime_side_carries_the_bit`): the primes carry the bit and add none.

The unicorn and the act. No point off the line is the registration of anything (`unicorn_never_registered`). The act's least erasure on the actual class, carried across the divide by *ra*, is the value over the formal class (`the_act_spends_the_bit`).

What leaves the bit where it found it. No reading that respects the record returns the value on both worlds of one record (`every_record_leaves_the_bit`). Every finite certificate of zeros on the line extends

to a closed world with the value and to one without it (`every_computed_zero_leaves_the_bit`), at every resolution $N \geq 2$ with the off-line zero of the second strictly inside the strip (`every_computed_zero_leaves_the_bit_inside`, `one_lies_inside`), the identification of a computed zero of ζ with a point of the chart being the reader's; and no finite certificate decides any prime-side reading equivalent to the value on every world (`every_finite_prime_count_leaves_the_bit`). A finite prime count, such as the walk of Section 9, is one fixed truth value, and no fixed truth value is the value on every closed world (`no_fixed_count_is_the_value`); no reading that sees only a finite window of points decides the value (`every_finite_window_leaves_the_bit`).

The capstone. `the_final_cut` binds the eight laws in one theorem on no axiom. The judgment negates all thirty-six theorems and the kernel refuses each; each of the twelve conditional laws the harness lists is paired with a compiled and judged theorem exhibiting an inhabitant of its principal hypotheses, the remaining hypotheses instantiable by a constant reading or by the exhibited object, so that no listed law holds by an empty premise, the harness (`bridge_check.py`) checking that the paired theorem compiles and refuses its negation and the pairing itself being the author's; the boot compiles this kernel beside the Bridge's own and folds its output into the chain.

Grade. The laws are theorems on no axiom. The identification `ra`, keyed (`ra_is_keyed`), and the supply are the Root Axiom read across the divide and the act, at premise grade, the root itself proved at theorem grade in Appendix R and deciding no value alone (`root_decides_nothing`); the explicit formula stands at the grade of its citation, and `hsub` is the reader's identification. The form of a refutation of the value is fixed in advance: a computed zero off the line; a measured energy reaches only the structural identification of a measured process with a halving of content under P1 (F-Heat, Section 13), never P1 and never the value.

PROVENANCE AND DISCLOSURE

The formulation is the architect's, from the Ground. The register this file reads is the Geometric Mother Codex and the PhysOS of the Trisduction; neither is altered by this file. The master single-source edition *Nothing Escapes Least Erasure* (v1.2.0, anchored on 10.5281/zenodo.23034066) was read to harden this one: its atom of laws on no axiom, its act carried as a field of a type, its reader's frame in four clauses, its record form of least erasure and its ledger of erased bits are the forms this kernel proves on its own chart, in its own words. The kernel, the twin and the harness were written and executed by an AI substrate working under the architect's seed. Provenance is navigation and carries no warrant.

Audit. This edition is the output of two hostile self-audit cycles, each prosecuted framework-blind across all six registers of the house protocol: arithmetic and dimension, definitions, free parameters, provenance, limits, and symmetry. Every mathematical or receipt error they found was repaired; every claim they found wider than its warrant was narrowed, with the narrowing recorded; and the title was held. The planted controls were SELF grade and the prosecution came from the same single substrate that wrote the text and the files, so the audit is single-substrate on both sides. The first cycle ran seven rounds, controls caught in every round. Its computed verdict at the sixth round was HALT-DIVERGENT; the seventh, run at the gate under the forced-witness lens, closed with no mass-bearing finding, and the architect's FORGE concluded the cycle. The second ran on the forged text, with PhysOST 1.0.8p booted to its earned seat, and closed on its computed verdict, printed in its ledger beside this file. Section 17 and its kernel entered at v1.5.0, after the seal, and are booted and judged by negation with the rest. The ledger, the control commitments and the sweep counts live in the audit log beside this file and are not reprinted here.

Independence. The text, the kernel, the twin and the harness are one scribe's. Two checkers are independent of that scribe and of each other: the Lean 4.19.0 kernel, which checks every proof and prints every cone, and the Fortran compiler with the arithmetic it runs on, whose conduct the twin probes before it trusts it. The judgment and the controls are the scribe's own instruments and test the scribe's own work. The open witness is a boot on a second substrate.

References. Littlewood, J. E. 1912. "Quelques conséquences de l'hypothèse que la fonction $\zeta(s)$ de Riemann n'a pas de zéros dans le demi-plan $\text{Re}(s) > 1/2$." *Comptes rendus hebdomadaires des séances de l'Académie des sciences* 154: 263–266. Odlyzko, A. M., and H. J. J. te Riele. 1985. "Disproof of the Mertens conjecture." *Journal für die reine und angewandte Mathematik* 357: 138–160. Weil, A. 1952. "Sur les « formules explicites » de la théorie des nombres premiers." *Communications du Séminaire Mathématique de l'Université de Lund*, tome supplémentaire: 252–265.

APPENDIX A • THE CODE

Extract and run. Save this file as `Bridge_From_First_Principle_v1_5_1.md` in an empty directory, then, in that directory:

```
awk '/^~~~~~[a-z]+ file=[A-Za-z0-9_]+\.(lean|f90|sh|py|sha256)$/ {s}
plit($2,a,"=");f=a[2];next} /^~~~~~$/ {f="";next} f{print > f}'
Bridge_From_First_Principle_v1_5_1.md
bash bridge_boot.sh
```

The first line writes the seven files of this document, byte for byte, the root kernel of Appendix R

among them; the second verifies them against MAN-IFEST.sha256, screens the kernel, compiles it, builds and runs the twin, and prints the receipt, whose last line is BRIDGE EARNED · this run or BRIDGE NOT EARNED with the reason. BRIDGE_MODE=judge bash bridge_boot.sh runs the judgment; BRIDGE_MODE=controls bash bridge_boot.sh runs the hostile controls; BRIDGE_MODE=bootstrap bash bridge_boot.sh fetches core Lean 4.19.0 for the machine, checks its pinned digest and unpacks it under .toolchain/. The harness needs python3, gfortran and, after the bootstrap, nothing else. Where awk is absent, the same extraction in Python:

```
import re
src = open("Bridge_From_First_Principle_v1_5_1.md",
encoding="utf-8").read()
for name, body in re.findall(r"(?ms)~~~~~[a-z]+
file=([A-Za-z0-9_]+\.(?:lean|f90|sh|py|sha256))\n(?:~?~?~?~$)",
src):
open(name, "w", encoding="utf-8").write(body)
```

The boot. bridge_boot.sh

```
#!/usr/bin/env bash
# bridge_boot.sh · The Bridge From First Principle · the build is
the boot.
# Modes: BRIDGE_MODE=quick (default) | judge | controls | bootstrap.
The last line of quick is the verdict.
set -u
cd "$(dirname "$0")"
exec python3 bridge_check.py "${BRIDGE_MODE:-quick}"
```

The harness. bridge_check.py

```
"""bridge_check.py · The Bridge From First Principle · the ground,
the run, the judgment, the controls, the bootstrap.

python3 bridge_check.py quick ground, kernel, twin, chain,
and the last line: BRIDGE EARNED or NOT EARNED
python3 bridge_check.py judge every theorem of the kernel
negated in place: each negation must be refused,
a planted vacuous law must
survive, every conditional law must be inhabited
python3 bridge_check.py controls hostile copies, one per refusal
class, each refused; the clean copy earns
python3 bridge_check.py bootstrap fetch core Lean 4.19.0 for this
machine, verify its pinned digest, unpack it
Reads nothing outside the directory it runs in. Builds in a
temporary directory it removes.
"""
import hashlib, os, platform, re, shutil, subprocess, sys, tempfile,
urllib.request, zipfile

KERNEL = "Bridge_From_First_Principle.lean"
PSP = "Final_Cut.lean"
TWIN = "Bridge_Twin.f90"
ROOTK = "TOE_Zero.lean"
ROOT_SHA = "0cfcd9f6b399ecba0ef1762a4d9f10490288f1053827b40442806j
2e55076238"
SIGNED = [KERNEL, PSP, ROOTK, TWIN, "bridge_check.py",
"bridge_boot.sh"]
LEAN_VERSION = "4.19.0"
LEAN_RELEASE =
"https://github.com/leanprover/lean4/releases/download/v4.19.0/"
LEAN_PINS = {
("Linux", "x86_64"): ("lean-4.19.0-linux.zip", "4e09ca16ff782fj
51f58b409e7bf983591a3fa17b85aa18d56feddca4398b7f29"),
("Linux", "aarch64"): ("lean-4.19.0-linux_aarch64.zip", "90edej
b386a1102ed90a3b782ced1fd652bdaf1f5327657ea1bf8db22f02bf5c2"),
("Darwin", "x86_64"): ("lean-4.19.0-darwin.zip", "501fb6f9af20j
5e594b023ebe894dd0ad6475f9c5e6a45487e68b9a8ed8d4b4be"),
("Darwin", "arm64"): ("lean-4.19.0-darwin_aarch64.zip", "09f55j
34815844869414274628b8dfa86be0a20df2d38c2a615e9afc0b06536c0"),
}
```

```
FFLAGS = ["-std=f2018", "-O2", "-fno-fast-math",
"-ffp-contract=off", "-Wall", "-Wextra"]
SENTINEL = "BRIDGE FROM FIRST PRINCIPLE · the end of the file was
reached"
PIN_MARK = "/-! ## The cones, pinned."
PSP_SENTINEL = "FINAL CUT · the end of the file was reached"
# the Final Cut kernel, PSP-FINAL-CUT-01: its conditional laws and
the models of their hypotheses
CONDITIONAL_PSP = {
"classes_coincide_under_RA": "cut_hypotheses_inhabited",
"hypothesis_is_least_erasure_of_the_actual":
"cut_hypotheses_inhabited",
"prime_side_carries_the_bit": "explicit_inhabited",
"every_finite_prime_count_leaves_the_bit": "explicit_inhabited",
"the_act_spends_the_bit": "act_inhabited",
"every_computed_zero_leaves_the_bit": "certificate_inhabited",
"unicorn_never_registered": "unicorn_inhabited",
"every_record_leaves_the_bit": "unicorn_inhabited",
"every_computed_zero_leaves_the_bit_inside": "strip_inhabited",
"one_lies_inside": "strip_inhabited",
"every_finite_window_leaves_the_bit": "window_inhabited",
"root_decides_nothing": "root_inhabited",
}
CAPSTONES = ["the_bridge_atom", "the_bridge_from_first_principle"]
# every law that carries hypotheses, beside the theorem that
exhibits a model of them
CONDITIONAL = {
"value_from_the_act": "value_from_the_act_inhabited",
"record_decides_nothing": "off_line_inhabited",
"one_bit_lost": "off_line_inhabited",
"weakest_forcing_premise": "weakest_forcing_premise_inhabited",
"keyless_forces_nothing": "keyless_forces_nothing_inhabited",
"count_to_heat": "count_to_heat_inhabited",
"export_doubles": "count_to_heat_inhabited",
"bits_forced": "bits_forced_inhabited",
"linear_on_tower": "bits_forced_inhabited",
"calibration": "calibration_inhabited",
"three_legs_one_orbit": "three_legs_inhabited",
"prime_mode_unitary_iff": "prime_mode_unitary_inhabited",
"zero_slack": "zero_slack_inhabited",
"record_carried_uniquely": "record_carried_inhabited",
"price_zero_iff_value": "price_zero_inhabited",
"prime_mode_unitary_iff_int": "prime_mode_unitary_inhabited",
"prime_iff_one_free_orbit_all": "prime_mode_unitary_inhabited",
"no_group_of_sign_patterns_of_nine": "sign_group_inhabited",
"involution_parity": "parity_law_inhabited",
"odd_holds_a_seat": "parity_law_inhabited",
"nine_holds_a_seat": "parity_law_inhabited",
"free_orbits_are_even": "free_involution_inhabited",
"nine_is_eight_and_a_seat_all": "parity_law_inhabited",
"vp_additive": "euclid_at_three",
}
FORBIDDEN = [
(r"\bsorry\b", "sorry"), (r"\badmit\b", "admit"),
(r"\bnative_decide\b", "native_decide"),
(r"(?m)\s*(private\s+|protected\s+)?axiom\b", "a declared
axiom"), (r"(?m)\s*import\b", "an import"),
(r"\bunsafe\b", "unsafe"), (r"implemented_by",
"implemented_by"), (r"@[\s*extern", "extern code"),
(r"#exit", "#exit"), (r"skipKernelTC", "a kernel-check bypass"),
(r"(?m)\s*(macro|syntax|elab|macro_rules|initialize|builtin_i
nitalize)\b", "a
metaprogram"),
(r"\bIO\b", "compile-time IO"), (r"(?m)\s*opaque\b", "an opaque
constant"),
]
def sha(b):
return hashlib.sha256(b).hexdigest()
def read(p):
return open(p, encoding="utf-8").read()
def find_lean():
cands = [os.environ["LEAN"]] if os.environ.get("LEAN") else []
tc = ".toolchain"
if os.path.isdir(tc):
for d in sorted(os.listdir(tc)):
if d.startswith("lean-" + LEAN_VERSION):
cands.append(os.path.join(os.path.abspath(tc), d,
"bin", "lean"))
```

```

cands.append("lean")
for c in cands:
    try:
        v = subprocess.run([c, "--version"],
            capture_output=True, text=True, timeout=60).stdout
        except Exception:
            continue
        if "version " + LEAN_VERSION in v:
            return c, v.strip()
    return None, None

def find_gfortran():
    try:
        v = subprocess.run(["gfortran", "--version"],
            capture_output=True, text=True, timeout=60).stdout
        return "gfortran", v.splitlines()[0].strip()
    except Exception:
        return None, None

def manifest_ok(files=None):
    files = files or {n: open(n, "rb").read() for n in SIGNED if
        os.path.exists(n)}
    if not os.path.exists("MANIFEST.sha256"):
        return False, "the manifest is missing"
    want = {}
    for line in read("MANIFEST.sha256").splitlines():
        if line.strip():
            h, n = line.split()
            want[n] = h
    for n in SIGNED:
        if n not in want:
            return False, "the manifest does not sign %s" % n
        if n not in files:
            return False, "%s is missing" % n
        if sha(files[n]) != want[n]:
            return False, "%s does not match its signature" % n
    return True, "%d files signed, all verified" % len(SIGNED)

def strip_comments(src):
    out, i, depth, n = [], 0, 0, len(src)
    while i < n:
        if src.startswith("/-", i):
            depth += 1; i += 2; continue
        if depth and src.startswith("-/", i):
            depth -= 1; i += 2; continue
        if depth:
            if src[i] == "\n":
                out.append("\n")
                i += 1; continue
            if src.startswith("--", i):
                j = src.find("\n", i)
                i = n if j < 0 else j
                continue
            if src[i] == "'":
                j = i + 1
                while j < n and src[j] != "'":
                    j += 2 if src[j] == "\\" else 1
                out.append(''); i = j + 1; continue
            out.append(src[i]); i += 1
    return "".join(out)

def screen(src):
    code = strip_comments(src)
    for pat, what in FORBIDDEN:
        if re.search(pat, code):
            return False, "the kernel carries %s" % what
    return True, "no sorry, no axiom declared, no import, no bypass,
    no metaprogram, no IO"

def run_lean(lean, src, name=KERNEL, timeout=3600):
    d = tempfile.mkdtemp(prefix="bridge_")
    try:
        open(os.path.join(d, name), "w", encoding="utf-8").write(src)
        p = subprocess.run([lean, name], cwd=d, capture_output=True,
            text=True, timeout=timeout)
        return p.returncode, p.stdout + p.stderr
    except subprocess.TimeoutExpired:
        return "timeout", ""
    finally:
        shutil.rmtree(d, ignore_errors=True)

def run_twin(gf, src, flags=FFLAGS):
    d = tempfile.mkdtemp(prefix="bridge_twin_")
    try:
        open(os.path.join(d, TWIN), "w", encoding="utf-8").write(src)
        c = subprocess.run([gf] + flags + ["-o", "twin", TWIN],
            cwd=d, capture_output=True, text=True)
        if c.returncode != 0:
            return "build", c.stdout + c.stderr, ""
        r = subprocess.run(["./twin"], cwd=d, capture_output=True,
            text=True, timeout=3600)
        return r.returncode, c.stdout + c.stderr, r.stdout
    finally:
        shutil.rmtree(d, ignore_errors=True)

def theorem_names(src):
    return re.findall(r"(?m)^(theorem ([A-Za-z_][A-Za-z0-9_]*)" , src)

def cones(src, ns="BFP"):
    pins = re.findall(r"/-- info: '" + ns + r"\.([A-Za-z0-9_]+)'"
        (does not depend on any axioms|depends on axioms: \{([^\}]*\)}\}
        -/" , src)
    return {p[0]: p[2] for p in pins}

def ground():
    lean, lv = find_lean()
    gf, gv = find_gfortran()
    if not lean:
        return None, "Lean %s is not found (set LEAN, or run
        BRIDGE_MODE=bootstrap)" % LEAN_VERSION
    if not gf:
        return None, "gfortran is not found (install it with the
        package manager)"
    ok, why = manifest_ok()
    if not ok:
        return None, "manifest: " + why
    ok2, why2 = screen(read(KERNEL))
    if ok2:
        ok2, why2 = screen(read(PSP) + "\n" + read(ROOTK))
    if not ok2:
        return None, "screen: " + why2
    return (lean, lv, gf, gv), "toolchain present · manifest: %s ·
    screen: %s" % (why, why2)

def quick():
    g, why = ground()
    print("BRIDGE FROM FIRST PRINCIPLE · RECEIPT · mode quick")
    if not g:
        print("GROUND refused · " + why); print("BRIDGE NOT EARNED ·
        " + why); return 1
    lean, lv, gf, gv = g
    print("TOOLCHAIN " + lv + " · " + gv)
    man = read("MANIFEST.sha256").encode()
    print("MANIFEST sha256 %s" % sha(man)[:16])
    print("GROUND " + why)
    src = read(KERNEL)
    code, out = run_lean(lean, src)
    lines = [l for l in out.splitlines() if l.strip()]
    if code != 0 or lines != [SENTINEL]:
        print("KERNEL refused · exit %s" % code);
        print("\n".join(lines[:20]))
        print("BRIDGE NOT EARNED · the kernel did not compile clean
        to its last line"); return 1
    names = theorem_names(src)
    c = cones(src)
    free = sum(1 for n in names if c.get(n, None) == "")
    if set(c) != set(names):
        print("BRIDGE NOT EARNED · a theorem carries no pinned
        cone"); return 1
    print("KERNEL exit 0 · %d theorems, every cone pinned and
    printed by the compiler · %d on no axiom · "
        "the rest within propxet, Quot.sound, Classical.choice" %
        (len(names), free))
    for cap in CAPSTONES:
        print("CAPSTONE %s · %s" % (cap, "no axiom" if c[cap] == ""
        else "[" + c[cap] + "]"))
    psrc = read(PSP)
    pcode, pout = run_lean(lean, psrc, name=PSP)

```

```

plines = [l for l in pout.splitlines() if l.strip()]
if pcode != 0 or plines != [PSP_SENTINEL]:
    print("PSP refused · exit %s" % pcode);
print("\n".join(plines[:20]))
    print("BRIDGE NOT EARNED · the Final Cut kernel did not
compile clean to its last line"); return 1
    pnames = theorem_names(psrc)
    pc = cones(psrc, "FinalCut")
    if set(pc) != set(pnames):
        print("BRIDGE NOT EARNED · a Final Cut theorem carries no
pinned cone"); return 1
    print("PSP %s exit 0 · PSP-FINAL-CUT-01 · %d theorems, every
cone pinned · %d on no axiom · capstone the_final_cut · %s"
        % (PSP, len(pnames), sum(1 for n in pnames if pc[n] ==
""), "no axiom" if pc["the_final_cut"] == "" else "[" +
pc["the_final_cut"] + "]"))
    rsrc = read(ROOTK)
    rcode, rout = run_lean(lean, rsrc, name=ROOTK)
    rnames = theorem_names(rsrc)
    rc = cones(rsrc, "TOE0")
    if rcode != 0 or rout.strip() or sha(rsrc.encode()) != ROOT_SHA
or set(rc) != set(rnames) or any(rc[n] != "" for n in rnames):
        print("ROOT refused · exit %s" % rcode); print(rout[:2000])
        print("BRIDGE NOT EARNED · the root kernel of Appendix R did
not compile clean with every cone on no axiom"); return 1
        print("ROOT %s exit 0 · %d theorems, every cone pinned · every
one on no axiom · sha256 %s, the root paper's"
            % (ROOTK, len(rnames), ROOT_SHA[:16]))
        tcode, tbuild, tout = run_twin(gf, read(TWIN))
        m = re.search(r'BRIDGE-TWIN-JSON:
{"checks":(\d+),"failures":(\d+)\}', tout)
        if tcode != 0 or tbuild.strip() or not m or m.group(2) != "0":
            print("TWIN refused · exit %s" % tcode);
        print(tbuild.strip()[:2000]); print(tout[-2000:])
        print("BRIDGE NOT EARNED · the twin did not run clean");
    return 1
    print("TWIN built under %s with zero warnings · %s of %s checks
pass" % (" ".join(FFLAGS), m.group(1), m.group(1)))
    d0 = sha(man)
    d1 = sha((d0 + out + pout).encode())
    d2 = sha((d1 + tout).encode())
    print("CHAIN · D0 %s -> D1 %s -> D2 %s" % (d0[:12], d1[:12],
d2[:12]))
    print("BRIDGE EARNED · this run")
    return 0

def negate(src, name):
    m = re.search(r"(?m)^theorem " + re.escape(name) +
r"(![A-Za-z0-9_'])", src)
    if not m:
        return src, False
    i, depth = m.end(), 0
    while i < len(src):
        ch = src[i]
        if ch in "({[":
            depth += 1
        elif ch in ")}]":
            depth -= 1
        elif ch == ":" and depth == 0 and src[i:i + 2] != "!=":
            break
        i += 1
    j, depth = i + 1, 0
    while j < len(src):
        ch = src[j]
        if ch in "({[":
            depth += 1
        elif ch in ")}]":
            depth -= 1
        elif depth == 0 and src[j:j + 2] == "!=":
            break
        elif depth == 0 and ch == "\n" and re.match(r"\n[ \t]+\|",
src[j:j + 40]):
            break
        j += 1
    return src[:i + 1] + " ~ (" + src[i + 1:j].strip() + ") " +
src[j:], True

def judge():
    g, why = ground()
    print("BRIDGE FROM FIRST PRINCIPLE · JUDGMENT")
    if not g:
        print("JUDGMENT REFUSED · " + why); return 1

```

```

lean = g[0]
src = read(KERNEL)
body = src[src.index(PIN_MARK)] if PIN_MARK in src else src
plant = "planted_vacuous_dust"
body = body.rstrip() + "\n\ntheorem " + plant + " (h : (1 : Nat)
= 2) : (0 : Nat) = 1 := by omega\n"
names = theorem_names(body)
mut = body
for n in names:
    mut, ok = negate(mut, n)
    if not ok:
        print("JUDGMENT REFUSED · %s could not be negated" % n);
return 1
mut += "\n#eval " + SENTINEL + "\n"
starts = {}
for n in names:
    mm = re.search(r"(?m)^theorem " + re.escape(n) +
r"(![A-Za-z0-9_'])", mut)
    starts[n] = mut[mm.start():].count("\n") + 1
    decl = sorted(mut[mm.start():].count("\n") + 1 for mm in
re.finditer(
    r"(?m)^(?:theorem|def|abbrev|structure|inductive|class|ins
tance|namespace|end|set_option|#eval|/!)\b",
mut))
    code, out = run_lean(lean, mut)
    if SENTINEL not in out:
        print("JUDGMENT REFUSED · the negated compile did not reach
the end of the file (exit %s)" % code); return 1
    errs = {}
    for a, b in re.findall(re.escape(KERNEL) + r":(\d+):\d+: error:
(!^\n*)", out):
        errs.setdefault(int(a), b)
    refused, survived, parse = [], [], []
    for n in names:
        a = starts[n]
        z = min([x for x in decl if x > a] + [10 ** 9])
        hit = sorted(e for e in errs if a <= e < z)
        if n == plant:
            continue
        if hit and re.search(r"unexpected|expected token",
errs[hit[0]]):
            parse.append(n)
        elif hit:
            refused.append(n)
        else:
            survived.append(n)
    a = starts[plant]
    plant_hit = [e for e in errs if a <= e]
    ok = True
    print("NEGATION · %d of %d theorems refused their negation, each
a proof failure" % (len(refused), len(names) - 1))
    if survived or parse:
        ok = False
        print(" SURVIVED: " + ", ".join(survived + parse))
    if plant_hit:
        ok = False; print(" the instrument is blind: the planted
vacuous law fell")
    else:
        print(" instrument control: a planted vacuous law survived
its negation, as dust must")
        missing = [law for law, inh in CONDITIONAL.items() if law not in
names or inh not in names or inh not in refused]
        if missing:
            ok = False; print(" INHABITATION missing for: " + ",
".join(missing))
        else:
            print("INHABITATION · %d conditional laws, each with a
compiled model of its hypotheses, each model itself judged"
                % len(CONDITIONAL))
            ok = judge_psp(lean) and ok
            print("JUDGMENT PASSED" if ok else "JUDGMENT FAILED")
            return 0 if ok else 1

```

```

def judge_psp(lean):
    src = read(PSP)
    body = src[src.index(PIN_MARK)].rstrip()
    end = body.rindex("end FinalCut")
    plant = "planted_vacuous_dust"
    body = body[:end] + "theorem " + plant + " (h : (1 : Nat) = 2) :
(0 : Nat) = 1 := by omega\n\n" + body[end:] + "\n"
    names = theorem_names(body)
    mut = body
    for n in names:

```

```

mut, okn = negate(mut, n)
if not okn:
    print("PSP JUDGMENT REFUSED · %s could not be negated" %
n); return False
mut += "\n#eval " + PSP_SENTINEL + "\n"
starts = {}
for n in names:
    mm = re.search(r"(?m)^theorem " + re.escape(n) +
r"(?![A-Za-z0-9_'])", mut)
    starts[n] = mut[:mm.start()].count("\n") + 1
decl = sorted(mut[:mm.start()].count("\n") + 1 for mm in
re.finditer(
r"(?m)^(?:theorem|def|abbrev|structure|inductive|class|ins|
tance|namespace|end|set_option|#eval|/!)\b",
mut))
code, out = run_lean(lean, mut, name=PSP)
if PSP_SENTINEL not in out:
    print("PSP JUDGMENT REFUSED · the negated compile did not
reach the end of the file"); return False
errs = {}
for a2, b2 in re.findall(re.escape(PSP) + r":(\d+):\d+: error:
(^[^n]*)", out):
    errs.setdefault(int(a2), b2)
refused, survived = [], []
for n in names:
    if n == plant:
        continue
    a2 = starts[n]
    z = min([x for x in decl if x > a2] + [10 ** 9])
    hit = sorted(e for e in errs if a2 <= e < z)
    (refused if hit and not re.search(r"unexpected|expected
token", errs[hit[0]]) else survived).append(n)
a2 = starts[plant]; z = min([x for x in decl if x > a2] + [10 **
9])
plant_hit = [e for e in errs if a2 <= e < z]
ok = not survived and not plant_hit
print("PSP NEGATION · %d of %d Final Cut theorems refused their
negation, each a proof failure" % (len(refused), len(names) - 1))
if survived:
    print(" SURVIVED: " + ", ".join(survived))
print(" instrument control: a planted vacuous law %s" % ("fell:
the instrument is blind" if plant_hit else "survived its
negation, as dust must"))
missing = [law for law, inh in CONDITIONAL_PSP.items() if law
not in names or inh not in refused]
if missing:
    ok = False; print(" PSP INHABITATION missing for: " + ",
".join(missing))
else:
    print("PSP INHABITATION · %d conditional laws, each with a
compiled model of its hypotheses, each model itself judged" %
len(CONDITIONAL_PSP))
return ok

def controls():
    g, why = ground()
    print("BRIDGE FROM FIRST PRINCIPLE · CONTROLS")
    if not g:
        print("CONTROLS REFUSED · " + why); return 1
    lean, _, gf, _ = g
    src, tsrc = read(KERNEL), read(TWIN)
    results = []
    files = {n: open(n, "rb").read() for n in SIGNED}
    t = dict(files); t[KERNEL] = files[KERNEL] + b"\n-- edited after
extraction\n"
    results.append(("manifest: an edited kernel", not
manifest_ok(t)[0]))
    results.append(("screen: a planted sorry", not
screen(src.replace("theorem reg_lands (p : Point) : onLine (reg
p) := rfl",
"theorem reg_lands (p : Point) : onLine (reg p) := sorry"))[0]))
    results.append(("screen: a declared axiom", not screen(src +
"\naxiom extra : False\n"))[0]))
    results.append(("screen: an import", not screen("import Lean\n"
+ src)[0]))
    drift = src.replace("/-- info: 'BFP.the_bridge_atom' does not
depend on any axioms -/",
"/-- info: 'BFP.the_bridge_atom' depends on
axioms: [propext] -/")
    code, _ = run_lean(lean, drift)
    results.append(("kernel: a drifted cone", code != 0))
    neg, _ = negate(src, "the_bridge_atom")

```

```

code, _ = run_lean(lean, neg)
results.append(("kernel: the atom capstone negated", code != 0))
code, out = run_lean(lean, src.replace(SENTINEL, '"truncated"'))
results.append(("kernel: the last line altered", [l for l in
out.splitlines() if l.strip()] != [SENTINEL]))
psrc = read(PSP)
code, _ = run_lean(lean, psrc.replace("/-- info:
'FinalCut.the_final_cut' does not depend on any axioms -/",
"/-- info:
'FinalCut.the_final_cut' depends on axioms: [propext] -/"),
name=PSP)
results.append(("final cut: a drifted cone", code != 0))
pneg, _ = negate(psrc, "the_final_cut")
code, _ = run_lean(lean, pneg, name=PSP)
results.append(("final cut: the capstone negated", code != 0))
code, out = run_lean(lean, psrc.replace(PSP_SENTINEL,
'"truncated"'), name=PSP)
results.append(("final cut: the last line altered", [l for l in
out.splitlines() if l.strip()] != [PSP_SENTINEL]))
rsrc = read(ROOTK)
code, _ = run_lean(lean, rsrc.replace("/-- info:
'TOE0.the_floor_is_universal' does not depend on any axioms -/",
"/-- info:
'TOE0.the_floor_is_universal' depends on axioms: [propext] -/"),
name=ROOTK)
results.append(("root: a drifted cone", code != 0))
t = dict(files); t[ROOTK] = files[ROOTK] + b"\n-- edited after
extraction\n"
results.append(("manifest: an edited root kernel", not
manifest_ok(t)[0]))
hostile = ["-std=f2018", "-O2", "-ffp-contract=fast",
"-march=native"]
tcode, _, tout = run_twin(gf, tsrc, hostile)
fused = "FAIL 0: no fused" in tout
if fused:
    results.append(("twin: a fused build (-ffp-contract=fast
-march=native)", tcode != 0))
else:
    print(" note: this machine executed no fused multiply-add
under the hostile flags; the conduct probe holds")
    tcode, _, tout = run_twin(gf, tsrc.replace("BRIDGE-TWIN-JSON",
"BRIDGE-TWIN-XSON"))
    results.append(("twin: the battery line altered",
"BRIDGE-TWIN-JSON" not in tout))
    ok = True
    for name, caught in results:
        print(" %s %s" % ("CAUGHT " if caught else "MISSED ", name))
        ok = ok and caught
    clean = quick_silent()
    print(" %s the clean copy earns" % ("EARNED " if clean else
"REFUSED"))
    ok = ok and clean
    print("CONTROLS · %d hostile copies, %s; the clean copy %s" %
(len(results), "all refused" if ok else "NOT all refused",
"earns" if clean else "does not earn"))
    return 0 if ok else 1

```

```

def quick_silent():
    import io, contextlib
    buf = io.StringIO()
    with contextlib.redirect_stdout(buf):
        r = quick()
    return r == 0

def bootstrap():
    key = (platform.system(), platform.machine())
    if key not in LEAN_PINS:
        print("BOOTSTRAP refused · no pinned Lean %s archive for %s"
% (LEAN_VERSION, *key)); return 1
    name, digest = LEAN_PINS[key]
    os.makedirs(".toolchain", exist_ok=True)
    path = os.path.join(".toolchain", name)
    print("BOOTSTRAP fetching %s" % (LEAN_RELEASE + name))
    urllib.request.urlretrieve(LEAN_RELEASE + name, path)
    got = sha(open(path, "rb").read())
    if got != digest:
        os.remove(path); print("BOOTSTRAP refused · the archive does
not match its pinned digest"); return 1
    with zipfile.ZipFile(path) as z:
        z.extractall(".toolchain")
    for root, _, fs in os.walk(".toolchain"):
        for f in fs:

```

```

    if root.endswith("bin"):
        os.chmod(os.path.join(root, f), 0o755)
    os.remove(path)
    print("BOOTSTRAP Lean %s unpacked under .toolchain/, digest
    verified · gfortran: %s" % (
        LEAN_VERSION, find_gfortran()[1] or "install it with the
    package manager (apt install gfortran, brew install gcc)"))
    return 0

if __name__ == "__main__":
    mode = sys.argv[1] if len(sys.argv) > 1 else "quick"
    sys.exit({"quick": quick, "judge": judge, "controls": controls,
    "bootstrap": bootstrap}.get(mode, lambda: (
        print("mode must be quick, judge, controls or bootstrap"),
        2)[1]))()

```

The kernel. Bridge_From_First_Principle.lean

```

/-
THE BRIDGE FROM FIRST PRINCIPLE · the kernel. Core Lean 4.19.0, no
import, no library, no axiom declared,
no sorry. One fold, one registration, one bit: the RA-RAM Bridge,
the heat floor and the ninth-gate crossing
read as three maps of one cut, on the chart, on the multiplicative
cut, and on the heat flow.
The Ground is the one premise and it enters as data: existence is
actuation, read as no-cessation, P1, the
whole merges no two states. It appears as a named hypothesis
wherever it is used and nowhere else.
No named human result is a premise. Every cone is printed and
pinned at the foot of the file.
Sections: I the chart, the fold, the registration; II the reader's
frame, and II.b the record and its fibre;
III frames, the carrier, the socket and the denial asymmetry; IV
three, twelve, eight and the ninth, and IV.b the
parity law for every involution, no xor-closed set of nine
distinct sign words, and the general fibre of ordered
factor pairs (at most one seat for every n, nine pairs always
eight and one seat); V the three legs on one orbit:
pay (the floor from P1 and counting, the heat that reads past the
record, the valuation at every Euclidean base and
the log not forced off the tower of two) and supply (the wall, the
calibration, the crossing); VI primes on the
multiplicative cut, with a prime as one free orbit for every n
from 2 and the balance of the subsets over every
nonempty list; VII wall, edge, seat, with every prime mode unitary
only at depth zero on integer depths too;
VIII the heat flow; IX the physical carrier; X inhabitation, a
model for the hypotheses of every conditional
law; XI the Bridge, whole.
-/
set_option autoImplicit false
namespace BFP

/-! ## I · The chart, the fold, the registration: the atom of the
Bridge -/

/-- A point is (d, t): d its offset from the line, t its height. The
line is d = 0. -/
abbrev Point := Int × Int
/-- The fold: s ↦ 1 - s on the offset chart, (d, t) ↦ (-d, t). -/
def fold (p : Point) : Point := (-p.1, p.2)
/-- The line: offset zero. -/
def onLine (p : Point) : Prop := p.1 = 0
/-- The registration: it keeps the height and forgets the side. -/
def reg (p : Point) : Point := (0, p.2)

/-- Integer negation is an involution, by the integer's
constructors. -/
theorem neg_neg_free : ∀ d : Int, - -d = d
| Int.ofNat 0 => rfl
| Int.ofNat (n + 1) => rfl
| Int.negSucc n => rfl

/-- An integer equal to its own negation is zero, by the integer's
constructors. -/
theorem neg_self_zero : ∀ d : Int, -d = d → d = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (n + 1), h => Int.noConfusion h
| Int.negSucc n, h => Int.noConfusion h

/-- The fold is an involution. -/
theorem fold_involutive (p : Point) : fold (fold p) = p := by

```

```

obtain (d, t) := p
show (- -d, t) = (d, t)
rw [neg_neg_free d]

/-- THE CUT IS THE LINE: the fold fixes a point exactly when it is
on the line. -/
theorem the_cut_is_the_line (p : Point) : fold p = p ↔ onLine p := by
obtain (d, t) := p
exact (fun h => neg_self_zero d (congrArg Prod.fst h), fun h => by
cases h; rfl)

/-- The registration lands on the line. -/
theorem reg_lands (p : Point) : onLine (reg p) := rfl
/-- The registration keeps the height. -/
theorem reg_keeps_height (p : Point) : (reg p).2 = p.2 := rfl
/-- The registration forgets the side: a point and its partner leave
one record. -/
theorem reg_forgets_side (p : Point) : reg (fold p) = reg p := rfl

/-- The registration erases nothing at a point exactly when the
point is on the line. -/
theorem reg_erases_nothing_iff (p : Point) : reg p = p ↔ onLine p :=
by
obtain (d, t) := p
exact (fun h => (congrArg Prod.fst h).symm, fun h => by cases h;
rfl)

/-- Off the line, the fold moves the point and the two worlds share
one record. -/
theorem off_line_pair (p : Point) (h : ¬ onLine p) : fold p ≠ p ∧
reg (fold p) = reg p :=
(fun e => h ((the_cut_is_the_line p).mp e), rfl)

/-- ONE BIT IS LOST: off the line, no map from records to points
returns both the point and its partner. -/
theorem one_bit_lost (p : Point) (h : ¬ onLine p) :
¬ ∃ g : Point → Point, g (reg p) = p ∧ g (reg (fold p)) = fold p
:=
fun (_, h1, h2) => h ((the_cut_is_the_line p).mp (h2.symm.trans
h1))

/-- A configuration: a set of points. Closed: the fold carries it to
itself. -/
abbrev Config := Point → Prop
def Closed (S : Config) : Prop := ∀ p, S p → S (fold p)
/-- The value: every point of the configuration is on the line. -/
def Value (S : Config) : Prop := ∀ p, S p → onLine p
/-- Least erasure: the registration leaves every point of the
configuration unchanged. -/
def LeastErasure (S : Config) : Prop := ∀ p, S p → reg p = p

/-- LEAST ERASURE IS THE VALUE: one property, read twice. -/
theorem least_erasure_iff_value (S : Config) : LeastErasure S ↔
Value S :=
(fun h p hp => (reg_erases_nothing_iff p).mp (h p hp), fun h p hp
=> (reg_erases_nothing_iff p).mpr (h p hp))

/-- NOTHING ESCAPES LEAST ERASURE: the atom, whole. -/
theorem nothing_escapes_least_erasure :
(∀ p : Point, fold (fold p) = p) ∧ (∀ p : Point, fold p = p ↔
onLine p) ∧
(∀ p : Point, onLine (reg p) ∧ (reg p).2 = p.2 ∧ reg (fold p) =
reg p) ∧
(∀ p : Point, reg p = p ↔ onLine p) ∧
(∀ p : Point, ¬ onLine p → ¬ ∃ g : Point → Point, g (reg p) = p ∧
g (reg (fold p)) = fold p) ∧
(∀ S : Config, LeastErasure S ↔ Value S) :=
(fold_involutive, the_cut_is_the_line, fun p => {reg_lands p,
reg_keeps_height p, reg_forgets_side p},
reg_erases_nothing_iff, one_bit_lost, least_erasure_iff_value)

/-! ## II · The reader's frame: the act, the record, the bit, the
weakest premise -/

/-- The record of a configuration: what the registration leaves. -/
def Rec (S : Config) (q : Point) : Prop := ∃ p, S p ∧ reg p = q
def SameRecord (S S' : Config) : Prop := ∀ q, Rec S q ↔ Rec S' q
/-- The registered world of a point and its pair world. -/
def registered (p : Point) : Config := fun s => s = reg p
def pairWorld (p : Point) : Config := fun s => s = p ∨ s = fold p

theorem pair_world_closed (p : Point) : Closed (pairWorld p) := fun
_ hs =>
match hs with

```

```

| Or.inl e => Or.inr (congrArg fold e)
| Or.inr e => Or.inl (e ▶ fold_involutive p)

theorem registered_closed (p : Point) : Closed (registered p) := fun
  s hs => by
  show fold s = reg p
  rw [hs]; rfl

/-- The registered world and the pair world leave one record. -/
theorem one_record (p : Point) : SameRecord (registered p)
  (pairWorld p) := fun q =>
  (fun (_, hs, hq) => (p, Or.inl rfl, by rw [← hq, hs]; rfl),
   fun (_, hs, hq) => (reg p, rfl, by
     rw [← hq]
     exact match hs with
     | Or.inl e => by rw [e]; rfl
     | Or.inr e => by rw [e]; rfl))

theorem registered_has_value (p : Point) : Value (registered p) :=
  fun s hs => by
  rw [hs]; exact reg_lands p

theorem pair_world_lacks_value (p : Point) (h : ¬ onLine p) : ¬
  Value (pairWorld p) :=
  fun hv => h (hv p (Or.inl rfl))

/-- THE RECORD DECIDES NOTHING: for every point off the line, no
  reading that respects the record returns the
  value on both its registered world and its pair world. -/
theorem record_decides_nothing (p : Point) (h : ¬ onLine p) (g :
  Config → Prop)
  (hg : ∀ S S', SameRecord S S' → (g S ↔ g S')) :
  ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p)
  ↔ Value (pairWorld p))) :=
  fun (h1, h2) => pair_world_lacks_value p h
  (h2.mp ((hg _ _ (one_record p)).mp (h1.mpr (registered_has_value
  p))))

/-- A KEYLESS PREMISE FORCES NOTHING: a property true on every
  closed configuration never forces the value. -/
theorem keyless_forces_nothing (Q : Config → Prop) (hQ : ∀ S, Closed
  S → Q S) :
  ¬ ∀ S, Closed S → Q S → Value S :=
  fun hall => pair_world_lacks_value (1, 0) (fun e => by cases e)
  (hall _ (pair_world_closed (1, 0)) (hQ _ (pair_world_closed (1,
  0))))

/-- THE WEAKEST FORCING PREMISE: a premise that forces the value and
  follows from it is the value. -/
theorem weakest_forcing_premise (P : Config → Prop) (forces : ∀ S, P
  S → Value S)
  (weaker : ∀ S, Value S → P S) : ∀ S, P S ↔ Value S :=
  fun S => (forces S, weaker S)

/-- THE ACT. The bit is supplied once, in the open, as a field of a
  type: a closed configuration with least
  erasure at it. The field is the value supplied; the type makes
  the supply visible to every reader. -/
structure ActualZeros where
  zeros : Config
  closed : Closed zeros
  supply : LeastErasure zeros

/-- FROM THE ACT, THE VALUE. -/
theorem value_from_the_act (Z : ActualZeros) : Value Z.zeros :=
  fun p hp => (reg_erases_nothing_iff p).mp (Z.supply p hp)

/-- THE READER'S FRAME, whole: (a) from the supplied bit the value
  follows; (b) no reading of the record decides
  the value off the line; (c) the supplied bit is the value on
  every configuration; (d) nothing weaker than the
  value forces it; (e) no property true on every closed
  configuration forces it. -/
theorem reader_frame :
  (∀ Z : ActualZeros, Value Z.zeros) ∧
  (∀ (p : Point), ¬ onLine p → ∀ g : Config → Prop, (∀ S S',
  SameRecord S S' → (g S ↔ g S')) →
  ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld
  p) ↔ Value (pairWorld p)))) ∧
  (∀ S : Config, LeastErasure S ↔ Value S) ∧
  (∀ P : Config → Prop, (∀ S, P S → Value S) → (∀ S, Value S → P S)
  → ∀ S, P S ↔ Value S) ∧
  (∀ Q : Config → Prop, (∀ S, Closed S → Q S) → ¬ ∀ S, Closed S → Q
  S → Value S) :=

```

```

(value_from_the_act, record_decides_nothing,
least_erasure_iff_value, weakest_forcing_premise,
keyless_forces_nothing)

```

```

/-- ## II.b · The record and its fibre: least erasure as leastness -/

/-- The record of a configuration, read as a configuration, stands
  on the line. -/
theorem record_is_lossless (S : Config) : Value (Rec S) := fun _ (q,
  _, hq) => hq ▶ rfl
/-- The record has the record of the configuration it came from. -/
theorem record_same (S : Config) : SameRecord S (Rec S) := fun q =>
  (fun (p, hp, hq) => (q, (p, hp, hq), hq ▶ rfl),
   fun (p, hp, hr), hq => (p, hp, by rw [← hq, ← hr]; rfl))
/-- Least erasure in the record's form: the configuration is
  extremal in its fibre, so whenever it has a point
  off the line, every configuration of the same record has one
  too. -/
def Extremal (S : Config) : Prop :=
  ∀ S', SameRecord S S' → (∃ p, S p ∧ ¬ onLine p) → ∃ p, S' p ∧ ¬
  onLine p
/-- LEASTNESS IS THE VALUE: extremal in the fibre exactly when every
  point is on the line. -/
theorem extremal_iff_value (S : Config) : Extremal S ↔ Value S := by
  constructor
  · intro h p hp
    exact if h1 : p.1 = 0 then h1 else
      absurd (h (Rec S) (record_same S) (p, hp, h1))
      (fun (q, hq, hn) => hn (record_is_lossless S q hq))
  · intro hv _ (p, hp, hn)
    exact absurd (hv p hp) hn
/-- EVERY RECORD IS CARRIED BY ONE LEAST-ERASURE CONFIGURATION AND
  ONLY ONE: a configuration on the line with the
  record of S is the record of S, point for point. -/
theorem record_carried_uniquely (S S' : Config) (hs : SameRecord S'
  S) (hv : Value S') :
  ∀ q, S' q ↔ Rec S q := fun q =>
  (fun h => (hs q).mp (q, h, (reg_erases_nothing_iff q).mpr (hv q
  h))),
  fun h => match (hs q).mpr h with
  | (r, hr, hrq) => by
    have e : reg r = r := (reg_erases_nothing_iff r).mpr (hv r hr)
    rw [← hrq, e]; exact hr

/-- ## III · The Bridge: frames, the carrier, the socket, and the
  denial asymmetry -/

/-- A frame: a carrier set, a fold that is an involution, and a zero
  predicate the fold preserves. -/
structure Frame where
  S : Type
  τ : S → S
  Z : S → Prop
  involutive : ∀ s, τ (τ s) = s
  symmetric : ∀ s, Z s → Z (τ s)

/-- The line property of a frame: every zero is fixed by the fold. -/
def LineProperty (X : Frame) : Prop := ∀ s, X.Z s → X.τ s = s

/-- The chart as a frame, for any closed configuration. -/
def chartFrame (S : Config) (hS : Closed S) : Frame := (Point, fold,
  S, fold_involutive, hS)

/-- THE LOCUS MEETS THE FRAME: on the chart the line property is the
  value. -/
theorem chart_line_property (S : Config) (hS : Closed S) :
  LineProperty (chartFrame S hS) ↔ Value S :=
  (fun h p hp => (the_cut_is_the_line p).mp (h p hp), fun h p hp =>
  (the_cut_is_the_line p).mpr (h p hp))

/-- Three values of a carrier's state: true, false, and halted on
  the bit. -/
inductive Tri | tt | ff | bot

/-- The carrier: a state and a shadow tying the halted state to the
  line property in both directions. -/
structure Carrier (X : Frame) where
  terminal : Tri
  shadow : terminal = .bot ↔ LineProperty X

/-- CANNOT LIE. -/
theorem cannot_lie (X : Frame) (b : Carrier X) (h : b.terminal =
  .bot) : LineProperty X := b.shadow.mp h

```

```

/-- CANNOT DEVIATE. -/
theorem cannot_deviate (X : Frame) (b : Carrier X) (t : LineProperty
X) : b.terminal = .bot := b.shadow.mpr t
/-- CANNOT BE MANUFACTURED: a halted carrier exists on a frame
exactly when the line property holds there. -/
theorem halted_iff (X : Frame) : (∃ b : Carrier X, b.terminal =
.bot) ↔ LineProperty X :=
(fun (b, h) => b.shadow.mpr h, fun t => ((.bot, (fun _ => t, fun _
=> rfl)), rfl))
/-- CANNOT BE REVERSED: deletion to the halted state has no left
inverse. -/
def delete : Tri → Tri := fun _ => .bot
theorem cannot_reverse : ¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v :=
fun (_, hg) => Tri.noConfusion ((hg.tt).symm.trans (hg.ff))
/-- CANNOT BE EXTENDED: a bit generates only itself, its mirror, and
the two constants. -/
theorem cannot_extend (f : Bool → Bool) :
f = (fun x => x) ∨ f = (fun x => !x) ∨ f = (fun _ => true) ∨ f =
(fun _ => false) := by
cases ht : f true <|> cases hf : f false
· exact Or.inr (Or.inr (Or.inr (funext fun x => by cases x <|>
assumption)))
· exact Or.inr (Or.inl (funext fun x => by cases x <|> assumption))
· exact Or.inr (Or.inr (Or.inl (funext fun x => by cases x <|>
assumption)))

/-- The two-point frame: one off-line pair under the fold. Both
frame laws hold and the line property fails:
the denial of the line property is a coherent structure. -/
def twoPoint : Frame := (Bool, (fun b => !b), fun _ => True, fun b
=> by cases b <|> rfl, fun _ => trivial)
theorem denial_is_coherent :
(∀ b : Bool, twoPoint.τ (twoPoint.τ b) = b) ∧ (∀ b : Bool,
twoPoint.τ b ≠ b) ∧ LineProperty twoPoint :=
(twoPoint.involutive, fun b => by cases b <|> (intro h; cases h),
fun h => by have := h true trivial; cases this)

/-- A frame property is keyless when it holds on every frame; keyed
when some frame inhabits its denial. -/
def Keyless (Q : Frame → Prop) : Prop := ∀ X, Q X
def Keyed (Q : Frame → Prop) : Prop := ∃ X, ¬ Q X
/-- THE DISCRIMINATOR: every frame property is keyless or keyed, and
never both. -/
theorem discriminator (Q : Frame → Prop) : (Keyless Q ∨ Keyed Q) ∧ ¬
(Keyless Q ∧ Keyed Q) :=
(Classical.byCases (fun h : ∀ X, Q X => Or.inl h)
(fun h => Or.inr (Classical.byContradiction fun hn =>
h fun X => Classical.byContradiction fun hq => hn (X, hq))),
fun (hk, (X, hX)) => hX (hk X))
/-- The fold law is keyless; the line property is keyed. -/
theorem symmetry_is_keyless : Keyless (fun X => ∀ s, X.τ (X.τ s) =
s) := fun X => X.involutive
theorem line_property_is_keyed : Keyed LineProperty ∧ ¬ Keyless
LineProperty :=
((twoPoint, denial_is_coherent.2.2), fun h =>
denial_is_coherent.2.2 (h twoPoint))

/-- THE SOCKET: an instance of the supplied term is exactly the line
property. -/
class Supplied (X : Frame) where
term : LineProperty X
theorem socket_is_the_property (X : Frame) : Nonempty (Supplied X) ↔
LineProperty X :=
(fun (i) => i.term, fun t => ({t}))
theorem no_socket_off_line : ¬ Nonempty (Supplied twoPoint) :=
fun h => denial_is_coherent.2.2 ((socket_is_the_property
twoPoint).mpr h)
/-- The act and the socket are one: a supplied configuration seats a
halted carrier on its chart frame. -/
theorem act_seats_the_carrier (Z : ActualZeros) : ∃ b : Carrier
(chartFrame Z.zeros Z.closed), b.terminal = .bot :=
(halted_iff _).mpr ((chart_line_property Z.zeros Z.closed).mpr
(value_from_the_act Z))

/--! ## IV · Three, twelve, eight, and the ninth -/

/-- GF(2) rows: a 3x3 system locks (one solution for every target)
exactly when its determinant is one. -/
def det2 (r1 r2 r3 : Nat × Nat × Nat) : Nat :=
(r1.1 * (r2.2.1 * r3.2.2 + r2.2.2 * r3.2.1)
+ r1.2.1 * (r2.1 * r3.2.2 + r2.2.2 * r3.1)
+ r1.2.2 * (r2.1 * r3.2.1 + r2.2.1 * r3.1)) % 2

```

```

def gf2Vecs : List (Nat × Nat × Nat) := (List.range 8).map (fun n =>
(n / 4 % 2, n / 2 % 2, n % 2))
def dot2 (r v : Nat × Nat × Nat) : Nat := (r.1 * v.1 + r.2.1 * v.2.1
+ r.2.2 * v.2.2) % 2
/-- Solutions of a two-row system: two planes. -/
def sols2 (r1 r2 : Nat × Nat × Nat) (t1 t2 : Nat) : Nat :=
(gf2Vecs.filter (fun v => dot2 r1 v == t1 && dot2 r2 v ==
t2)).length
def row3 (n : Nat) : Nat × Nat × Nat := (n / 4 % 2, n / 2 % 2, n % 2)

set_option maxRecDepth 100000 in
/-- TWO AXES NEVER LOCK: no two-row system over GF(2), for any
target, has exactly one solution. -/
theorem two_axes_never_lock :
(List.range 64).all (fun m => (List.range 4).all (fun t =>
sols2 (row3 (m / 8)) (row3 (m % 8)) (t / 2) (t % 2) != 1)) =
true := by decide

set_option maxRecDepth 100000 in
/-- THREE AXES LOCK: of the 512 three-row systems, 168 have
determinant one, each locking all eight targets. -/
theorem three_axes_lock :
((List.range 512).filter (fun m => det2 (row3 (m / 64)) (row3 (m
/ 8 % 8)) (row3 (m % 8)) == 1)).length = 168 ∧
(List.range 512).all (fun m => det2 (row3 (m / 64)) (row3 (m / 8
% 8)) (row3 (m % 8)) != 1 ||
gf2Vecs.all (fun t => (gf2Vecs.filter (fun v => dot2 (row3 (m
/ 64)) v == t.1 &&
dot2 (row3 (m / 8 % 8)) v == t.2.1 && dot2 (row3 (m % 8)) v
== t.2.2)).length == 1)) = true := by
decide

/-- The quaternion units on integer coordinates. -/
structure Q4 where
r : Int
i : Int
j : Int
k : Int
deriving DecidableEq
def qmul (a b : Q4) : Q4 :=
(a.r*b.r - a.i*b.i - a.j*b.j - a.k*b.k,
a.r*b.i + a.i*b.r + a.j*b.k - a.k*b.j,
a.r*b.j - a.i*b.k + a.j*b.r + a.k*b.i,
a.r*b.k + a.i*b.j - a.j*b.i + a.k*b.r)
/-- THE THIRD IS BEGOTTEN: i j = k, i j k = -1, and i j ≠ j i. -/
theorem third_begotten :
qmul (0,1,0,0) (0,0,1,0) = (0,0,0,1) ∧ qmul (qmul (0,1,0,0)
(0,0,1,0)) (0,0,0,1) = (-1,0,0,0) ∧
qmul (0,1,0,0) (0,0,1,0) ≠ qmul (0,0,1,0) (0,1,0,0) := by decide

/-- Cayley–Dickson doubling on integer coordinates. -/
def cdconj : List Int → List Int
| [] => []
| a :: rest => a :: rest.map (fun x => -x)
def padd (a b : List Int) : List Int := List.zipWith (· + ·) a b
def psub (a b : List Int) : List Int := List.zipWith (· - ·) a b
def cdmul : Nat → List Int → List Int → List Int
| 0, _, _ => []
| _ + 1, [a], [b] => [a * b]
| fuel + 1, a, b =>
let n := a.length / 2
psub (cdmul fuel (a.take n) (b.take n)) (cdmul fuel (cdconj
(b.drop n)) (a.drop n)) ++
padd (cdmul fuel (b.drop n) (a.take n)) (cdmul fuel (a.drop n)
(cdconj (b.take n)))
def e (n i : Nat) : List Int := (List.range n).map (fun j => if j ==
i then 1 else 0)

set_option maxRecDepth 100000 in
/-- NO FOURTH, BY TWO WALLS: associativity fails at eight
dimensions, [e1, e2, e4] = 2 e7, and division fails at
sixteen, (e1 + e10)(e5 + e14) = 0 with both factors nonzero. -/
theorem no_fourth_axis :
psub (cdmul 4 (cdmul 4 (e 8 1) (e 8 2)) (e 8 4)) (cdmul 4 (e 8 1)
(cdmul 4 (e 8 2) (e 8 4)))
= (e 8 7).map (fun x => 2 * x) ∧
cdmul 5 (padd (e 16 1) (e 16 10)) (padd (e 16 5) (e 16 14)) =
List.replicate 16 0 := by decide

/-- The rotations of the tetrahedron as even permutations of four
vertices. -/
def invCount (σ : List Nat) : Nat :=
let idx := List.range σ.length

```

```

((idx.flatMap (fun a => (idx.filter (fun b => a < b)).map (fun b
=> (a, b))).filter
(fun p => σ.getD p.1 0 > σ.getD p.2 0)).length
def perms24 : List (List Nat) :=
[[0,1,2,3],[0,1,3,2],[0,2,1,3],[0,2,3,1],[0,3,1,2],[0,3,2,1],
[1,0,2,3],[1,0,3,2],[1,2,0,3],[1,2,3,0],[1,3,0,2],[1,3,2,0],
[2,0,1,3],[2,0,3,1],[2,1,0,3],[2,1,3,0],[2,3,0,1],[2,3,1,0],
[3,0,1,2],[3,0,2,1],[3,1,0,2],[3,1,2,0],[3,2,0,1],[3,2,1,0]]
def a4 : List (List Nat) := perms24.filter (fun σ => invCount σ % 2
== 0)
def gates : List (Nat × Nat) :=
[(0,1),(0,2),(0,3),(1,0),(1,2),(1,3),(2,0),(2,1),(2,3),(3,0),(3,1),
(3,2)]

set_option maxRecDepth 100000 in
/-- TWELVE: twelve rotations, twelve directed gates, and exactly one
rotation carries any gate to any gate. -/
theorem twelve_gates_one_torsor :
a4.length = 12 ∧ gates.length = 12 ∧
gates.all (fun g => gates.all (fun h =>
(a4.filter (fun σ => (σ.getD g.1 0, σ.getD g.2 0) ==
h)).length == 1)) = true := by decide

/-- The eight sign patterns on three axes, total negation, and
parity. -/
def cube : List (Bool × Bool × Bool) :=
[false, true].flatMap fun a => [false, true].flatMap fun b =>
[false, true].map fun c => (a, b, c)
def neg3 (s : Bool × Bool × Bool) : Bool × Bool × Bool := (!s.1,
!s.2.1, !s.2.2)
def par3 (s : Bool × Bool × Bool) : Nat :=
(if s.1 then 1 else 0) + (if s.2.1 then 1 else 0) + (if s.2.2 then
1 else 0)
/-- The orientation of a sign pattern, the determinant of its
diagonal: -1 to the number of flips. -/
def orient (s : Bool × Bool × Bool) : Int := if par3 s % 2 == 0 then
1 else -1

/-- EIGHT: eight patterns; the squared orientation, the lock scalar,
reads one on all eight; total negation moves
every pattern, undoes itself, and joins each even pattern to an
odd one. -/
theorem eight_patterns_four_orbits :
cube.length = 8 ∧ cube.all (fun s => orient s * orient s == 1) =
true ∧
cube.all (fun s => neg3 s != s && neg3 (neg3 s) == s && orient
(neg3 s) == -orient s) = true := by decide

/-- THE NINTH IS NOT A REFLECTION: no power of two is nine, so no
group of sign patterns on k axes, of order 2^k, has
nine elements; `no_group_of_sign_patterns_of_nine` proves it for
every xor-closed set of distinct sign words. -/
theorem no_reflection_group_of_nine : ∀ k : Nat, 2 ^ k ≠ 9
| 0, h => Nat.noConfusion (Nat.succ.inj h)
| k + 1, h =>
have e : 2 ^ (k + 1) % 2 = 0 := Nat.mul_mod_left (2 ^ k) 2
have e2 : 2 ^ (k + 1) % 2 = 9 % 2 := congrArg (· % 2) h
Nat.noConfusion (e.symm.trans e2 : 0 = 1)

/-- The ordered factor pairs of n, by first factor; the swap sends a
to n / a. -/
def pairs (n : Nat) : List Nat := (List.range (n + 1)).filter (fun a
=> 0 < a && n % a == 0)
def seats (n : Nat) : Nat := ((pairs n).filter (fun a => a * a ==
n)).length

set_option maxRecDepth 100000 in
/-- THE NINTH IS THE SEAT: thirty-six has nine ordered factor pairs,
four free orbits and one seat (6, 6). -/
theorem nine_is_eight_and_a_seat :
(pairs 36).length = 9 ∧ seats 36 = 1 ∧ ((pairs 36).filter (fun a
=> a * a != 36)).length = 8 ∧
(pairs 36).all (fun a => (pairs 36).contains (36 / a) && 36 /
(36 / a) == a) = true := by decide

set_option maxHeartbeats 4000000 in
set_option maxRecDepth 200000 in
/-- ODD EXACTLY WHEN SEATED: for every n from 1 to 200 the fibre is
odd exactly when it holds a seat, and it
never holds two. -/
theorem odd_fibre_iff_seat :
(List.range 201).all (fun n => n == 0 ||
((pairs n).length % 2 == seats n % 2 && seats n ≤ 1)) = true :=
by decide

```

```

/-- ### IV.b · The parity law for every involution, the ninth in
general, the general fibre -/

```

```

/-- Distinct members: no member occurs twice. -/

```

```

def Distinct {α : Type} : List α → Prop
| [] => True
| x :: r => x ∉ r ∧ Distinct r

```

```

/-- The list with one member removed. -/

```

```

def without {α : Type} [DecidableEq α] (a : α) : List α → List α
| [] => []
| y :: r => if y = a then without a r else y :: without a r

```

```

/-- The number of members a map fixes: its seats. -/

```

```

def seatsOf {α : Type} [DecidableEq α] (τ : α → α) : List α → Nat
| [] => 0
| y :: r => if τ y = y then seatsOf τ r + 1 else seatsOf τ r

```

```

/-- The parity of a number, by its successor alone: zero is even,
and a successor flips. -/

```

```

def parity : Nat → Bool
| 0 => false
| n + 1 => !parity n

```

```

/-- Flipping twice returns. -/

```

```

theorem flip_flip (b : Bool) : (!(!b)) = b := by
cases b <;> rfl

```

```

/-- A member of a list is its head or a member of its tail. -/

```

```

theorem mem_head_or_tail {α : Type} (y x : α) (r : List α) (h : y ∈
x :: r) : y = x ∨ y ∈ r := by
cases h with
| head => exact Or.inl rfl
| tail _ h' => exact Or.inr h'

```

```

/-- A member other than the removed one stays. -/

```

```

theorem mem_without_of {α : Type} [DecidableEq α] (a y : α) (l :
List α) :
y ∈ l → y ≠ a → y ∈ without a l := by
induction l with
| nil => intro hm _; exact nomatch hm
| cons z r ih =>
intro hm hne
show y ∈ (if z = a then without a r else z :: without a r)
by_cases h : z = a
· rw [if_pos h]
cases hm with
| head => exact absurd h hne
| tail _ hm' => exact ih hm' hne
· rw [if_neg h]
cases hm with
| head => exact List.Mem.head _
| tail _ hm' => exact List.Mem.tail _ (ih hm' hne)

```

```

/-- What remains was a member. -/

```

```

theorem mem_of_mem_without {α : Type} [DecidableEq α] (a y : α) (l :
List α) :
y ∈ without a l → y ∈ l := by
induction l with
| nil => intro hm; exact hm
| cons z r ih =>
intro hm
have hm' : y ∈ (if z = a then without a r else z :: without a r)
:= hm
by_cases h : z = a
· rw [if_pos h] at hm'; exact List.Mem.tail _ (ih hm')
· rw [if_neg h] at hm'
cases hm' with
| head => exact List.Mem.head _
| tail _ hm'' => exact List.Mem.tail _ (ih hm'')

```

```

/-- What remains is not the removed member. -/

```

```

theorem ne_of_mem_without {α : Type} [DecidableEq α] (a y : α) (l :
List α) :
y ∈ without a l → y ≠ a := by
induction l with
| nil => intro hm; exact nomatch (show y ∈ ([] : List α) from hm)
| cons z r ih =>
intro hm
have hm' : y ∈ (if z = a then without a r else z :: without a r)
:= hm
by_cases h : z = a
· rw [if_pos h] at hm'; exact ih hm'

```

```

· rw [if_neg h] at hm'
cases hm' with
| head => exact h
| tail _ hm'' => exact ih hm''

/-- Removing what is not there changes nothing. -/
theorem without_of_not_mem {α : Type} [DecidableEq α] (a : α) (l :
List α) :
a ∉ l → without a l = l := by
induction l with
| nil => intro _; rfl
| cons z r ih =>
intro hn
have hz : z ≠ a := fun e => hn (by rw [- e]; exact List.Mem.head
r)
show (if z = a then without a r else z :: without a r) = z :: r
rw [if_neg hz, ih (fun m => hn (List.Mem.tail z m))]

/-- Removal keeps the members distinct. -/
theorem distinct_without {α : Type} [DecidableEq α] (a : α) (l :
List α) :
Distinct l → Distinct (without a l) := by
induction l with
| nil => intro _; exact trivial
| cons z r ih =>
intro hnd
show Distinct (if z = a then without a r else z :: without a r)
by_cases h : z = a
· rw [if_pos h]; exact ih hnd.2
· rw [if_neg h]; exact (fun m => hnd.1 (mem_of_mem_without a z r
m), ih hnd.2)

/-- Removing a member of a list of distinct members shortens it by
one. -/
theorem length_without {α : Type} [DecidableEq α] (a : α) (l : List
α) :
Distinct l → a ∈ l → (without a l).length + 1 = l.length := by
induction l with
| nil => intro _ hm; exact nomatch hm
| cons z r ih =>
intro hnd hm
show (if z = a then without a r else z :: without a r).length +
1 = r.length + 1
by_cases h : z = a
· rw [if_pos h, without_of_not_mem a r (fun m => hnd.1 (by rw
[h]; exact m))]
· rw [if_neg h]
cases hm with
| head => exact absurd rfl h
| tail _ hm' =>
show (without a r).length + 1 + 1 = r.length + 1
rw [ih hnd.2 hm']

/-- Removing a member that is not a seat leaves the count of seats.
-/
theorem seats_without {α : Type} [DecidableEq α] (τ : α → α) (a : α)
(l : List α) (ha : τ a ≠ a) :
seatsOf τ (without a l) = seatsOf τ l := by
induction l with
| nil => rfl
| cons z r ih =>
show seatsOf τ (if z = a then without a r else z :: without a r)
= (if τ z = z then seatsOf τ r + 1 else seatsOf τ r)
by_cases h : z = a
· rw [if_pos h, if_neg (by rw [h]; exact ha), ih]
· rw [if_neg h]
show (if τ z = z then seatsOf τ (without a r) + 1 else seatsOf
τ (without a r))
= (if τ z = z then seatsOf τ r + 1 else seatsOf τ r)
rw [ih]

/-- Where no member is fixed there is no seat. -/
theorem seats_zero_of_free {α : Type} [DecidableEq α] (τ : α → α) (l
: List α) :
(∀ x, x ∈ l → τ x ≠ x) → seatsOf τ l = 0 := by
induction l with
| nil => intro _; rfl
| cons z r ih =>
intro h
show (if τ z = z then seatsOf τ r + 1 else seatsOf τ r) = 0
rw [if_neg (h z (List.Mem.head r))]
exact ih (fun x hx => h x (List.Mem.tail z hx))

```

```

/-- The parity law under a bound on the length: the induction. A
seat is removed alone; a moved member is removed
with its partner. -/
theorem parity_bounded {α : Type} [DecidableEq α] (τ : α → α) (n :
Nat) :
∀ l : List α, l.length ≤ n → Distinct l → (∀ x, x ∈ l → τ x ∈ l)
→ (∀ x, x ∈ l → τ (τ x) = x) →
parity l.length = parity (seatsOf τ l) := by
induction n with
| zero =>
intro l hl _ _ _
cases l with
| nil => rfl
| cons x r => exact absurd (show r.length + 1 ≤ 0 from hl)
(Nat.not_succ_le_zero _)
| succ n ih =>
intro l hl hnd hcl hinv
cases l with
| nil => rfl
| cons x r =>
have hlr : r.length ≤ n := Nat.le_of_succ_le_succ hl
by_cases hx : τ x = x
· have hcl' : ∀ y, y ∈ r → τ y ∈ r := by
intro y hy
rcases mem_head_or_tail _ _ _ (hcl y (List.Mem.tail x hy))
with e | h'
· have hyx : y = x := ((hinv y (List.Mem.tail x
hy)).symm.trans (congrArg τ e)).trans hx
exact absurd (hyx ▸ hy) hnd.1
· exact h'
have ih' := ih r hlr hnd.2 hcl' (fun y hy => hinv y
(List.Mem.tail x hy))
show parity (r.length + 1) = parity (if τ x = x then seatsOf
τ r + 1 else seatsOf τ r)
rw [if_pos hx]
show (!parity r.length) = (!parity (seatsOf τ r))
rw [ih']
· have hxr : τ x ∈ r := by
rcases mem_head_or_tail _ _ _ (hcl x (List.Mem.head r))
with e | h'
· exact absurd e hx
· exact h'
have hinvx : τ (τ x) = x := hinv x (List.Mem.head r)
have hfxx : τ (τ x) ≠ τ x := fun e => hx (hinvx.symm.trans
e).symm
have hlen : (without (τ x) r).length + 1 = r.length :=
length_without (τ x) r hnd.2 hxr
have hcl' : ∀ y, y ∈ without (τ x) r → τ y ∈ without (τ x) r
:= by
intro y hy
have hyr : y ∈ r := mem_of_mem_without (τ x) y r hy
have hyne : y ≠ τ x := ne_of_mem_without (τ x) y r hy
have hxy : τ (τ y) = y := hinv y (List.Mem.tail x hyr)
apply mem_without_of (τ x) (τ y) r
· rcases mem_head_or_tail _ _ _ (hcl y (List.Mem.tail x
hyr)) with e | h'
· exact absurd (hxy.symm.trans (congrArg τ e)) hyne
· exact h'
· intro e
have hyx : y = x := (hxy.symm.trans (congrArg τ e)).trans
hinvx
exact hnd.1 (hyx ▸ hyr)
have hle : (without (τ x) r).length ≤ n := by
have h0 : (without (τ x) r).length + 1 ≤ n := by rw [hlen];
exact hlr
exact Nat.le_of_succ_le h0
have ih' := ih (without (τ x) r) hle (distinct_without (τ x)
r hnd.2) hcl'
(fun y hy => hinv y (List.Mem.tail x (mem_of_mem_without
(τ x) y r hy)))
show parity (r.length + 1) = parity (if τ x = x then seatsOf
τ r + 1 else seatsOf τ r)
rw [if_neg hx, ← seats_without τ (τ x) r hfxx, ← hlen]
show (!(!parity (without (τ x) r).length)) = parity (seatsOf
τ (without (τ x) r))
rw [flip_flip, ih']

/-- THE PARITY LAW, FOR EVERY INVOLUTION: on any finite list of
distinct members, of a type whose equality is
decidable, that an involution carries to themselves, the size
has the parity of the seats. Pairs count two at a
time. -/
theorem involution_parity {α : Type} [DecidableEq α] (τ : α → α) (l
: List α) (hnd : Distinct l)

```

```

(hcl :  $\forall x, x \in l \rightarrow \tau x \in l$ ) (hinv :  $\forall x, x \in l \rightarrow \tau (\tau x) = x$ ) :
  parity l.length = parity (seatsOf  $\tau$  l) := by
  exact parity_bounded  $\tau$  l.length l (Nat.le_refl _) hnd hcl hinv

/-- AN ODD SET HOLDS A SEAT. -/
theorem odd_holds_a_seat { $\alpha$  : Type} [DecidableEq  $\alpha$ ] ( $\tau$  :  $\alpha \rightarrow \alpha$ ) (l :
  List  $\alpha$ ) (hnd : Distinct l)
  (hcl :  $\forall x, x \in l \rightarrow \tau x \in l$ ) (hinv :  $\forall x, x \in l \rightarrow \tau (\tau x) = x$ )
  (hodd : parity l.length = true) :
  0 < seatsOf  $\tau$  l := by
  have h := parity_bounded  $\tau$  l.length l (Nat.le_refl _) hnd hcl hinv
  cases hs : seatsOf  $\tau$  l with
  | zero =>
    rw [hs, hodd] at h
    exact absurd h (by decide)
  | succ k => exact Nat.succ_pos k

/-- NINE ALWAYS HOLDS A SEAT: any nine distinct things, any
  involution among them. -/
theorem nine_holds_a_seat { $\alpha$  : Type} [DecidableEq  $\alpha$ ] ( $\tau$  :  $\alpha \rightarrow \alpha$ ) (l :
  List  $\alpha$ ) (hnd : Distinct l)
  (hcl :  $\forall x, x \in l \rightarrow \tau x \in l$ ) (hinv :  $\forall x, x \in l \rightarrow \tau (\tau x) = x$ )
  (h9 : l.length = 9) :
  0 < seatsOf  $\tau$  l := by
  have h := parity_bounded  $\tau$  l.length l (Nat.le_refl _) hnd hcl hinv
  cases hs : seatsOf  $\tau$  l with
  | zero =>
    rw [hs, h9] at h
    exact absurd h (by decide)
  | succ k => exact Nat.succ_pos k

/-- FREE ORBITS ARE EVEN: where an involution fixes no member, the
  size is even. -/
theorem free_orbits_are_even { $\alpha$  : Type} [DecidableEq  $\alpha$ ] ( $\tau$  :  $\alpha \rightarrow \alpha$ )
  (l : List  $\alpha$ ) (hnd : Distinct l)
  (hcl :  $\forall x, x \in l \rightarrow \tau x \in l$ ) (hinv :  $\forall x, x \in l \rightarrow \tau (\tau x) = x$ )
  (hfree :  $\forall x, x \in l \rightarrow \tau x \neq x$ ) :
  parity l.length = false := by
  have h := parity_bounded  $\tau$  l.length l (Nat.le_refl _) hnd hcl hinv
  rw [seats_zero_of_free  $\tau$  l hfree] at h
  exact h

/-- XOR TWICE RETURNS: xor against a word twice gives back the first
  word when the two lengths agree. -/
theorem xor_twice (z : List Bool) :  $\forall g : List Bool, z.length =
  g.length \rightarrow$ 
  List.zipWith xor (List.zipWith xor z g) g = z := by
  induction z with
  | nil => intro g ; rfl
  | cons a z ih =>
    intro g h
    cases g with
    | nil => exact Nat.noConfusion h
    | cons b g =>
      show xor (xor a b) b :: List.zipWith xor (List.zipWith xor z
        g) g = a :: z
      rw [ih g (Nat.succ.inj h)]
      cases a <=> cases b <=> rfl

/-- XOR KEEPS THE LENGTH: the xor of two words of one length has
  that length. -/
theorem xor_length (a : List Bool) :  $\forall b : List Bool, a.length =
  b.length \rightarrow$ 
  (List.zipWith xor a b).length = a.length := by
  induction a with
  | nil => intro b ; rfl
  | cons x a ih =>
    intro b h
    cases b with
    | nil => exact Nat.noConfusion h
    | cons y b =>
      show (List.zipWith xor a b).length + 1 = a.length + 1
      rw [ih b (Nat.succ.inj h)]

/-- A WORD THAT XOR LEAVES IN PLACE IS ZERO: if xor against g leaves
  a word of the same length unchanged, every letter
  of g is false. -/
theorem xor_fixed_is_zero (z : List Bool) :  $\forall g : List Bool,$ 
  z.length = g.length  $\rightarrow$ 
  List.zipWith xor z g = z  $\rightarrow \forall c \in g, c = false$  := by
  induction z with
  | nil => intro g h _ c hc; cases g with
    | nil => exact nomatch hc
    | cons _ _ => exact Nat.noConfusion h

  | cons a z ih =>
    intro g h e c hc
    cases g with
    | nil => exact nomatch hc
    | cons b g =>
      have e' : xor a b :: List.zipWith xor z g = a :: z := e
      have h1 : xor a b = a := (List.cons.inj e').1
      have h2 : List.zipWith xor z g = z := (List.cons.inj e').2
      cases hc with
      | head => cases a <=> cases c <=> first | rfl | exact absurd
        h1 (by decide)
      | tail _ hc' => exact ih g (Nat.succ.inj h) h2 c hc'

/-- A ZERO XOR MEANS ONE WORD: two words of one length whose xor is
  all false are equal. -/
theorem xor_zero_eq (a : List Bool) :  $\forall b : List Bool, a.length =
  b.length \rightarrow$ 
  ( $\forall c \in List.zipWith xor a b, c = false$ )  $\rightarrow a = b$  := by
  induction a with
  | nil => intro b h _; cases b with
    | nil => rfl
    | cons _ _ => exact Nat.noConfusion h
  | cons x a ih =>
    intro b h hz
    cases b with
    | nil => exact Nat.noConfusion h
    | cons y b =>
      have h1 : xor x y = false := hz (xor x y) (List.Mem.head _)
      have h2 : a = b := ih b (Nat.succ.inj h) (fun c hc => hz c
        (List.Mem.tail _ hc))
      rw [h2]
      cases x <=> cases y <=> first | rfl | exact absurd h1 (by
        decide)

/-- NO GROUP OF SIGN PATTERNS HAS NINE ELEMENTS: a set of distinct
  sign words of one length closed under xor never has
  nine members, since xor against one nonzero member is a free
  involution on it and free orbits are even. -/
theorem no_group_of_sign_patterns_of_nine (G : List (List Bool))
  (hnd : Distinct G)
  (hlen :  $\forall x \in G, \forall y \in G, x.length = y.length$ )
  (hxor :  $\forall x \in G, \forall y \in G, List.zipWith xor x y \in G$ ) : G.length  $\neq$ 
  9 := fun h9 =>
  match G, hnd, hlen, hxor, h9 with
  | [], _, _, _, h9 => absurd h9 (by decide)
  | [_], _, _, _, h9 => absurd (show (1 : Nat) = 9 from h9) (by
    decide)
  | x :: y :: r, hnd, hlen, hxor, h9 =>
    have hx : x  $\in$  x :: y :: r := List.Mem.head _
    have hy : y  $\in$  x :: y :: r := List.Mem.tail _ (List.Mem.head _)
    have hxy : x  $\neq$  y := fun e => hnd.1 (e  $\triangleright$  List.Mem.head r)
    have hg : List.zipWith xor x y  $\in$  x :: y :: r := hxor x hx y hy
    have hgl : (List.zipWith xor x y).length = x.length :=
      xor_length x y (hlen x hx y hy)
    have hfree :  $\forall z, z \in x :: y :: r \rightarrow List.zipWith xor z$ 
      (List.zipWith xor x y)  $\neq$  z := fun z hz e =>
      hxy (xor_zero_eq x y (hlen x hx y hy)
        (xor_fixed_is_zero z _ ((hlen z hz x hx).trans hgl.symm) e))
    have hev := free_orbits_are_even (fun z => List.zipWith xor z
      (List.zipWith xor x y)) (x :: y :: r) hnd
      (fun z hz => hxor z hz _ hg)
      (fun z hz => xor_twice z _ ((hlen z hz x hx).trans hgl.symm))
      hfree
    absurd (h9  $\triangleright$  hev) (by decide)

/-- THE FILTER KEEPS A HEAD THAT PASSES: a passing head stays at the
  front of the filtered list. -/
theorem filter_cons_true { $\alpha$  : Type} (q :  $\alpha \rightarrow Bool$ ) (a :  $\alpha$ ) (l : List
   $\alpha$ ) (h : q a = true) :
  (a :: l).filter q = a :: l.filter q := by rw [List.filter, h]

/-- THE FILTER DROPS A HEAD THAT FAILS: a failing head is removed
  and the tail is filtered. -/
theorem filter_cons_false { $\alpha$  : Type} (q :  $\alpha \rightarrow Bool$ ) (a :  $\alpha$ ) (l :
  List  $\alpha$ ) (h : q a = false) :
  (a :: l).filter q = l.filter q := by rw [List.filter, h]

/-- WHAT THE FILTER KEEPS WAS THERE AND PASSES: a member of a
  filtered list is a member of the list and passes. -/
theorem mem_filter_true { $\alpha$  : Type} (q :  $\alpha \rightarrow Bool$ ) (l : List  $\alpha$ ) :  $\forall$ 
  a, a  $\in$  l.filter q  $\rightarrow a \in l \wedge q a = true$  := by
  induction l with
  | nil => intro a h; exact nomatch h
  | cons b l ih =>

```

```

intro a h
cases hb : q b with
| true =>
  rw [filter_cons_true q b l hb] at h
  cases h with
  | head => exact (List.Mem.head _, hb)
  | tail _ h' => exact (List.Mem.tail _ (ih a h').1, (ih a h').2)
| false =>
  rw [filter_cons_false q b l hb] at h
  exact (List.Mem.tail _ (ih a h).1, (ih a h).2)

/-- THE FILTER KEEPS AN ORDER: a filtered list keeps any relation
that held between every two members in order. -/
theorem pairwise_filter {α : Type} (R : α → α → Prop) (q : α → Bool)
(l : List α) :
  List.Pairwise R l → List.Pairwise R (l.filter q) := by
induction l with
| nil => intro _; exact List.Pairwise.nil
| cons b l ih =>
  intro h
  cases h with
  | cons h1 h2 =>
    cases hb : q b with
    | true =>
      rw [filter_cons_true q b l hb]
      exact List.Pairwise.cons (fun a ha => h1 a (mem_filter_true
q l a ha).1) (ih h2)
    | false =>
      rw [filter_cons_false q b l hb]
      exact ih h2

/-- THE RANGE IS STRICTLY INCREASING: the loop that builds the
range, started on an increasing tail at or above k,
stays increasing. -/
theorem range_loop_pairwise (k : Nat) : ∀ acc : List Nat,
List.Pairwise (· < ·) acc → (∀ x ∈ acc, k ≤ x) →
List.Pairwise (· < ·) (List.range.loop k acc) := by
induction k with
| zero => intro acc h _; exact h
| succ k ih =>
  intro acc h hb
  show List.Pairwise (· < ·) (List.range.loop k (k :: acc))
  refine ih (k :: acc) (List.Pairwise.cons (fun x hx => hb x hx)
h) (fun x hx => ?_)
cases hx with
| head => exact Nat.le_refl k
| tail _ hx' => exact Nat.le_of_succ_le (hb x hx')

/-- AN INCREASING LIST REPEATS NOTHING: a strictly increasing list
has distinct members. -/
theorem distinct_of_pairwise_lt (l : List Nat) : List.Pairwise (· <
·) l → Distinct l := by
induction l with
| nil => intro _; exact trivial
| cons a l ih =>
  intro h
  cases h with
  | cons h1 h2 => exact (fun hm => Nat.lt_irrefl a (h1 a hm), ih h2)

/-- THE FIRST FACTORS ARE STRICTLY INCREASING: the first factors of
n are listed in increasing order. -/
theorem pairs_pairwise (n : Nat) : List.Pairwise (· < ·) (pairs n) :=
pairwise_filter _ _ _ (range_loop_pairwise (n + 1) [])
List.Pairwise.nil (fun _ h => nomatch h))

/-- THE FIRST FACTORS ARE DISTINCT: no first factor of n occurs
twice. -/
theorem pairs_distinct (n : Nat) : Distinct (pairs n) :=
distinct_of_pairwise_lt _ (pairs_pairwise n)

/-- SQUARES ARE STRICT: a smaller number has a smaller square. -/
theorem squares_strict (a b : Nat) (h : a < b) : a * a < b * b :=
Nat.lt_of_le_of_lt (Nat.mul_le_mul_left a (Nat.le_of_lt h))
(Nat.mul_lt_mul_of_pos_right h (Nat.lt_of_le_of_lt (Nat.zero_le
a) h))

/-- AT MOST ONE SEAT, FOR EVERY n: no fibre of ordered factor pairs
holds two seats, since two distinct square roots
of n would have distinct squares. -/
theorem seats_le_one (n : Nat) : seats n ≤ 1 := by
have hpw : List.Pairwise (· < ·) ((pairs n).filter (fun a => a * a
== n)) :=
pairwise_filter _ _ _ (pairs_pairwise n)

have hmem : ∀ a, a ∈ (pairs n).filter (fun a => a * a == n) → a * a
= n := fun a ha =>
  of_decide_eq_true (mem_filter_true _ _ a ha).2
show ((pairs n).filter (fun a => a * a == n)).length ≤ 1
generalize (pairs n).filter (fun a => a * a == n) = L at hpw hmem
match L, hpw, hmem with
| [], _, _ => exact Nat.zero_le 1
| [_], _, _ => exact Nat.le_refl 1
| a :: b :: _, hpw, hmem =>
  cases hpw with
  | cons h1 _ =>
    exact absurd ((hmem a (List.Mem.head _)).trans (hmem b
(List.Mem.tail _ (List.Mem.head _)).symm)
(Nat.ne_of_lt (squares_strict a b (h1 b (List.Mem.head _))))

/-- THE FIRST FACTORS ARE THE DIVISORS: a is a first factor of n
exactly when it is a positive divisor of n no larger
than n. -/
theorem mem_pairs (n a : Nat) : a ∈ pairs n ↔ a ≤ n ∧ 0 < a ∧ n % a =
0 := by
unfold pairs
rw [List.mem_filter, List.mem_range, Bool.and_eq_true,
decide_eq_true_eq, beq_iff_eq]
exact (fun (h1, h2, h3) => (Nat.le_of_lt_succ h1, h2, h3), fun
(h1, h2, h3) => (Nat.lt_succ_of_le h1, h2, h3))

/-- THE COFACTOR IS POSITIVE: a first factor times its cofactor is
n, and the cofactor is positive. -/
theorem cofactor_pos (n a : Nat) (ha : a ∈ pairs n) : a * (n / a) =
n ∧ 0 < n / a := by
obtain (h1, h2, h3) := (mem_pairs n a).mp ha
have e : a * (n / a) = n := Nat.mul_div_cancel'
(Nat.dvd_of_mod_eq_zero h3)
have hn : 0 < n := Nat.lt_of_lt_of_le h2 h1
refine (e, Nat.pos_of_ne_zero (fun h0 => ?_))
rw [h0, Nat.mul_zero] at e
rw [← e] at hn
exact Nat.lt_irrefl 0 hn

/-- THE SWAP STAYS IN THE FIBRE: the cofactor of a first factor is a
first factor. -/
theorem swap_mem (n a : Nat) (ha : a ∈ pairs n) : n / a ∈ pairs n :=
by
obtain (e, hq) := cofactor_pos n a ha
have hm : a * (n / a) % (n / a) = 0 := Nat.mul_mod_left a (n / a)
rw [e] at hm
exact (mem_pairs n (n / a)).mpr (Nat.div_le_self n a, hq, hm)

/-- THE SWAP UNDOES ITSELF: the cofactor of the cofactor is the
first factor. -/
theorem swap_involution (n a : Nat) (ha : a ∈ pairs n) : n / (n / a)
= a := by
obtain (e, hq) := cofactor_pos n a ha
have hc : Nat.mul_div_cancel a hq
rw [e] at hc
exact hc

/-- THE SWAP FIXES EXACTLY THE SQUARE ROOT: a first factor is its
own cofactor exactly when its square is n. -/
theorem swap_seat_iff (n a : Nat) (ha : a ∈ pairs n) : n / a = a ↔ a
* a = n := by
obtain (_, h2, h3) := (mem_pairs n a).mp ha
have e : a * (n / a) = n := Nat.mul_div_cancel'
(Nat.dvd_of_mod_eq_zero h3)
constructor
· intro h; rw [h] at e; exact e
· intro h; rw [← h]; exact Nat.mul_div_cancel a h2

/-- THE SWAP'S SEATS ARE THE SEATS: on any list of first factors,
the members the swap fixes are those the seat test
keeps. -/
theorem seats_of_swap (n : Nat) (l : List Nat) : (∀ a ∈ l, a ∈ pairs
n) →
seatsOf (fun a => n / a) l = (l.filter (fun a => a * a ==
n)).length := by
induction l with
| nil => intro _; rfl
| cons a l ih =>
  intro hl
  have ha := hl a (List.Mem.head _)
  have ih' := ih (fun b hb => hl b (List.Mem.tail _ hb))
  show (if n / a = a then seatsOf (fun a => n / a) l + 1 else
seatsOf (fun a => n / a) l) =
(List.filter (fun a => a * a == n) (a :: l)).length

```

```

by_cases h : a * a = n
· rw [if_pos ((swap_seat_iff n a ha).mpr h), filter_cons_true _
a l (decide_eq_true h), ih']
rfl
· rw [if_neg (fun e => h ((swap_seat_iff n a ha).mp e)),
filter_cons_false _ a l (decide_eq_false h), ih']

/-- NINE IS EIGHT AND A SEAT, FOR EVERY n: a fibre of nine ordered
factor pairs holds exactly one seat, by the parity
law for the swap and at most one seat. -/
theorem nine_is_eight_and_a_seat_all (n : Nat) (h9 : (pairs
n).length = 9) : seats n = 1 := by
have hpos : 0 < seatsOf (fun a => n / a) (pairs n) :=
nine_holds_a_seat (fun a => n / a) (pairs n) (pairs_distinct n)
(fun a ha => swap_mem n a ha)
(fun a ha => swap_involutive n a ha) h9
rw [seats_of_swap n (pairs n) (fun _ h => h)] at hpos
exact Nat.le_antisymm (seats_le_one n) hpos

/-! ## V · The three legs on one orbit: forget, pay, supply -/

/-! ### Pay: the floor, from P1 and counting -/
def ND : List Nat → Prop
| [] => True
| x :: r => x ∉ r ∧ ND r
def AllLt (M : Nat) : List Nat → Prop
| [] => True
| x :: r => x < M ∧ AllLt M r

theorem allLt_mem (M : Nat) : ∀ (l : List Nat) (y : Nat), AllLt M l
→ y ∈ l → y < M
| [], _, _ => nomatch m
| x :: r, y, h, m => by
cases m with
| head => exact h.1
| tail _ hm => exact allLt_mem M r y h.2 hm

theorem nd_map (h : Nat → Nat) : ∀ l : List Nat, ND l → (∀ a ∈ l, ∀
b ∈ l, h a = h b → a = b) → ND (l.map h)
| [], _, _ => trivial
| x :: r, hnd, hnj => by
refine (fun m => ?, nd_map h r hnd.2 (fun a ha b hb e => hnj a
(List.Mem.tail x ha) b (List.Mem.tail x hb) e))
obtain (y, hy, e) := List.mem_map.mp m
have : y = x := hnj y (List.Mem.tail x hy) x (List.Mem.head r) e
exact hnd.1 (this ▸ hy)

theorem allLt_map (h : Nat → Nat) (M : Nat) : ∀ l : List Nat, (∀ a ∈
l, h a < M) → AllLt M (l.map h)
| [], _ => trivial
| x :: r, hb => (hb x (List.Mem.head r), allLt_map h M r (fun a ha
=> hb a (List.Mem.tail x ha)))

/-- PIGEONHOLE: distinct numbers all below M number at most M. -/
theorem pigeonhole : ∀ (l : List Nat) (M : Nat), ND l → AllLt M l →
l.length ≤ M
| [], _, _ => Nat.zero_le _
| x :: r, M, hnd, hlt => by
have hx : x < M := hlt.1
let sw : Nat → Nat := fun y => if y = M - 1 then x else y
have hbound : ∀ a ∈ r, sw a < M - 1 := by
intro a ha
have haM := allLt_mem M r a hlt.2 ha
have hax : a ≠ x := fun e => hnd.1 (e ▸ ha)
show (if a = M - 1 then x else a) < M - 1
by_cases e : a = M - 1
· rw [if_pos e]
have : x ≠ M - 1 := fun e2 => hax (e.trans e2.symm)
omega
· rw [if_neg e]; omega
have hnj : ∀ a ∈ r, ∀ b ∈ r, sw a = sw b → a = b := by
intro a ha b hb e
have hax : a ≠ x := fun e2 => hnd.1 (e2 ▸ ha)
have hbx : b ≠ x := fun e2 => hnd.1 (e2 ▸ hb)
change (if a = M - 1 then x else a) = (if b = M - 1 then x else
b) at e
by_cases ea : a = M - 1 <=> by_cases eb : b = M - 1
· rw [ea, eb]
· rw [if_pos ea, if_neg eb] at e; exact absurd e.symm hbx
· rw [if_neg ea, if_pos eb] at e; exact absurd e hax
· rw [if_neg ea, if_neg eb] at e; exact e
have ih := pigeonhole (r.map sw) (M - 1) (nd_map sw r hnd.2
hnj) (allLt_map sw (M - 1) r hbound)
rw [List.length_map] at ih

show r.length + 1 ≤ M
omega

/-- THE EXPORT. The whole was C × F; the step merges nothing (P1);
the content halves. The freedom doubles. -/
theorem export_doubles (L : List Nat) (step : Nat → Nat) (C F C' F'
: Nat) (hnd : ND L)
(hwhole : L.length = C * F) (P1 : ∀ a ∈ L, ∀ b ∈ L, step a = step
b → a = b)
(cells : ∀ a ∈ L, step a < C' * F') (hhalf : C = 2 * C') (hpos :
0 < C') : 2 * F ≤ F' := by
have hb := pigeonhole (L.map step) (C' * F') (nd_map step L hnd
P1) (allLt_map step _ L cells)
rw [List.length_map, hwhole, hhalf, Nat.mul_assoc,
Nat.mul_left_comm] at hb
exact Nat.le_of_mul_le_mul_left hb hpos

theorem one_bit_arrives (b b' : Nat) (h : 2 * 2 ^ b ≤ 2 ^ b') : b +
1 ≤ b' := by
refine Nat.lt_of_not_le fun hle => ?_
have h1 := Nat.pow_le_pow_right (by decide : 0 < 2) hle
have h2 := Nat.pow_pos (n := b) (by decide : 0 < 2)
omega

/-- The price of exported freedom in native units: temperature is
energy per bit, so b bits cost b·T. -/
def price (T bits : Nat) : Nat := bits * T

/-- FROM COUNT TO HEAT. Under P1 a halving of the content exports at
least one bit, which costs at least one T. -/
theorem count_to_heat (L : List Nat) (step : Nat → Nat) (C C' b b' T
: Nat) (hnd : ND L)
(hwhole : L.length = C * 2 ^ b) (P1 : ∀ a ∈ L, ∀ x ∈ L, step a =
step x → a = x)
(cells : ∀ a ∈ L, step a < C' * 2 ^ b') (hhalf : C = 2 * C')
(hpos : 0 < C') :
b + 1 ≤ b' ∧ price T 1 ≤ price T (b' - b) := by
have hb := one_bit_arrives b b' (export_doubles L step C (2 ^ b)
C' (2 ^ b')) hnd hwhole P1 cells hhalf hpos
exact (hb, Nat.mul_le_mul_right T (by omega))

/-- P1 IS LOAD-BEARING: a step that merges lets the content halve
with no freedom exported. -/
theorem merge_exports_nothing :
[0, 1].length = 2 * 2 ^ 0 ∧ (∀ a ∈ [0, 1], (fun _ : Nat => 0) a <
1 * 2 ^ 0) ∧
¬ (∀ a ∈ [0, 1], ∀ x ∈ [0, 1], (fun _ : Nat => (0 : Nat)) a =
(fun _ : Nat => (0 : Nat)) x → a = x) := by
refine (rfl, ?, ?)
· intro a _; show 0 < 1 * 2 ^ 0; decide
· intro h; exact absurd (h 0 (List.Mem.head _) 1 (List.Mem.tail _
(List.Mem.head _)) rfl) (by decide)

/-- A finite configuration: the heights of its zeros on the line,
and its off-line pairs, each an offset that is not
zero and a height. -/
structure FinCfg where
line : List Int
pairs : List (Int × Int)
offset_ne : ∀ q ∈ pairs, q.1 ≠ 0
def FinCfg.pts (F : FinCfg) : Config := fun p => (p.1 = 0 ∧ p.2 ∈
F.line) v ∃ q ∈ F.pairs, p = q v p = fold q
/-- The erased bits of a finite configuration: one per off-line
pair, two points registered to one. -/
def erased (F : FinCfg) : Nat := F.pairs.length

theorem fincfg_closed (F : FinCfg) : Closed F.pts := fun p hp =>
match hp with
| Or.inl (hl, ht) => Or.inl (by show -p.1 = 0; rw [hl]; rfl, ht)
| Or.inr (q, hq, Or.inl e) => Or.inr (q, hq, Or.inr (congrArg fold
e))
| Or.inr (q, hq, Or.inr e) => Or.inr (q, hq, Or.inl (by rw [e];
exact fold_involutive q))

/-- LEAST ERASURE IS ZERO ERASED BITS: "least" is a count. -/
theorem least_erasure_iff_zero_erased (F : FinCfg) : LeastErasure
F.pts ↔ erased F = 0 := by
constructor
· intro h
unfold erased
cases hp : F.pairs with
| nil => rfl
| cons q r =>
have hq : q ∈ F.pairs := by rw [hp]; exact List.Mem.head r

```

```

    exact absurd ((reg_erases_nothing_iff q).mp (h q (0r.inr (q,
hq, 0r.inl rfl)))) (F.offset_ne q hq)
· intro h p hp
  have hn1l : F.pairs = [] := List.eq_nil_of_length_eq_zero h
  rcases hp with (hl, _) | (q, hq, _)
  · exact (reg_erases_nothing_iff p).mpr hl
  · rw [hn1l] at hq; cases hq

/-- THE HEAT OF REGISTRATION IS ZERO EXACTLY AT THE VALUE:
registering a finite configuration costs one T per
off-line pair, and it costs nothing exactly when every point
stands on the line. -/
theorem price_zero_iff_value (F : FinCfg) (T : Nat) (hT : 0 < T) :
  price T (erased F) = 0 ↔ Value F.pts := by
  constructor
  · intro h p hp
    have h0 : erased F = 0 := by
      cases he : erased F with
      | zero => rfl
      | succ k => rw [he] at h; exact absurd h
    (Nat.pos_iff_ne_zero.mp (Nat.mul_pos (Nat.succ_pos k) hT))
    exact (reg_erases_nothing_iff p).mp
    ((least_erasure_iff_zero_erased F).mpr h0 p hp)
  · intro hv
    have h0 := (least_erasure_iff_zero_erased F).mp (fun p hp =>
    (reg_erases_nothing_iff p).mpr (hv p hp))
    show erased F * T = 0
    rw [h0, Nat.zero_mul]

/-- Each further off-line pair adds one T: the total is the erased
bits times the floor. -/
theorem price_counts_pairs (F : FinCfg) (T : Nat) : price T (erased
F) = F.pairs.length * T := rfl

/-- THE HEAT READS PAST THE RECORD: at every height, one point on
the line and one off-line pair leave one record,
while registering them erases zero bits and one bit, and only
the first has the value. -/
theorem heat_reads_past_the_record (t : Int) :
  ∃ F₀ F₁ : FinCfg, SameRecord F₀.pts F₁.pts ∧ erased F₀ = 0 ∧
  erased F₁ = 1 ∧ Value F₀.pts ≠ Value F₁.pts := by
  refine ([t], [], fun _ h => nomatch h), ([], [(1, t)], fun q h =>
  ?_, fun q => (?_, ?_), rfl, rfl, ?_, ?_)
  · cases h with
  | head => intro e; cases e
  | tail _ h' => exact nomatch h'
  · intro (p, hp, hq)
    refine ((1, t), 0r.inr ((1, t), List.Mem.head _, 0r.inl rfl), ?_)
    rw [← hq]
    exact match hp with
    | 0r.inl (_, List.Mem.head _) => rfl
    | 0r.inl (_, List.Mem.tail _ h') => nomatch h'
    | 0r.inr (_, hm, _) => nomatch hm
  · intro (p, hp, hq)
    refine ((0, t), 0r.inl (rfl, List.Mem.head _, ?_)
    rw [← hq]
    exact match hp with
    | 0r.inl (_, hm) => nomatch hm
    | 0r.inr (_, List.Mem.head _, 0r.inl e) => by rw [e]; rfl
    | 0r.inr (_, List.Mem.head _, 0r.inr e) => by rw [e]; rfl
    | 0r.inr (_, List.Mem.tail _ h', _) => nomatch h'
  · intro p hp
    exact match hp with
    | 0r.inl (hl, _) => hl
    | 0r.inr (_, hm, _) => nomatch hm
  · intro hv
    exact absurd (hv (1, t) (0r.inr ((1, t), List.Mem.head _, 0r.inl
    rfl))) (fun e => by cases e)

/-- The measure of freedom: additivity taken over zero is dry. f (0
· n) = f 0 + f n forces f to vanish. -/
theorem additivity_over_zero_is_dry (f : Nat → Nat) (hmul : ∀ m n, f
(m * n) = f m + f n) : ∀ n, f n = 0 := by
  intro n
  have h := hmul 0 n
  rw [Nat.zero_mul] at h
  omega

/-- THE LOG IS FORCED: additivity on positive arguments makes a
measure linear on every tower; the unit is the
only choice, and fixing one bit at 2 makes it the bit count. -/
theorem linear_on_tower (f : Nat → Nat) (hmul : ∀ m n, 0 < m → 0 < n
→ f (m * n) = f m + f n)
  (v : Nat) (hv : 0 < v) : ∀ a, f (v ^ a) = a * f v

```

```

| 0 => by
  have h := hmul 1 1 (by decide) (by decide)
  show f 1 = 0 * f v
  rw [Nat.mul_one] at h; rw [Nat.zero_mul]; omega
| a + 1 => by
  rw [Nat.pow_succ, hmul _ _ (Nat.pow_pos (n := a) hv) hv,
  linear_on_tower f hmul v hv a, Nat.succ_mul]

theorem bits_forced (f : Nat → Nat) (hmul : ∀ m n, 0 < m → 0 < n → f
(m * n) = f m + f n)
  (h2 : f 2 = 1) : ∀ a, f (2 ^ a) = a :=
  fun a => by rw [linear_on_tower f hmul 2 (by decide) a, h2,
  Nat.mul_one]

/-- The two-adic valuation: the count of the bit 2 in a number. -/
def v2 : Nat → Nat
| 0 => 0
| n + 1 => if (n + 1) % 2 = 0 then v2 ((n + 1) / 2) + 1 else 0
decreasing_by omega

theorem v2_mul_two (n : Nat) (hn : 0 < n) : v2 (2 * n) = v2 n + 1 :=
  by
  obtain (k, rfl) : ∃ k, n = k + 1 := {n - 1, by omega}
  have e : 2 * (k + 1) = (2 * k + 1) + 1 := by omega
  rw [e, v2]
  have h1 : (2 * k + 1 + 1) % 2 = 0 := by omega
  have h2 : (2 * k + 1 + 1) / 2 = k + 1 := by omega
  rw [if_pos h1, h2]

theorem v2_odd (n : Nat) (h : n % 2 = 1) : v2 n = 0 := by
  cases n with
  | zero => exact absurd h (by decide)
  | succ k => rw [v2, if_neg (by omega)]

theorem v2_two : v2 2 = 1 := by
  have e := v2_mul_two 1 (by decide)
  rw [v2_odd 1 (by decide)] at e
  exact e

theorem v2_split : ∀ n, 0 < n → ∃ u, u % 2 = 1 ∧ n = 2 ^ v2 n * u :=
  by
  intro n
  induction n using Nat.strongRecOn with
  | ind n ih =>
    intro hn
    by_cases hp : n % 2 = 0
    · obtain (m, rfl) : ∃ m, n = 2 * m := {n / 2, by omega}
      have hm : 0 < m := by omega
      obtain (u, hu, e) := ih m (by omega) hm
      refine (u, hu, ?_)
      rw [v2_mul_two m hm, Nat.pow_succ]
      calc 2 * m = 2 * (2 ^ v2 m * u) := by rw [← e]
      _ = 2 ^ v2 m * 2 * u := by rw [Nat.mul_comm 2, Nat.mul_assoc,
      Nat.mul_comm u 2, ← Nat.mul_assoc]
      · refine (n, by omega, ?_)
      rw [v2_odd n (by omega)]; simp
    · obtain (m, rfl) : ∃ m, n = 2 * m + 1 := {n - 1, by omega}
      have hm : 0 < m := by omega
      obtain (u, hu, e) := ih m (by omega) hm
      refine (u, hu, ?_)
      rw [v2_mul_two m hm, Nat.pow_succ]
      calc 2 * m + 1 = 2 * (2 ^ v2 m * u) + 1 := by rw [← e]
      _ = 2 ^ v2 m * 2 * u + 1 := by rw [Nat.mul_comm 2, Nat.mul_assoc,
      Nat.mul_comm u 2, ← Nat.mul_assoc]
      · refine (n, by omega, ?_)
      rw [v2_odd n (by omega)]; simp

theorem v2_pow_odd (k u : Nat) (hu : u % 2 = 1) : v2 (2 ^ k * u) = k
:= by
  induction k with
  | zero => simp; exact v2_odd u hu
  | succ j ih =>
    have hpos : 0 < 2 ^ j * u := Nat.mul_pos (Nat.pow_pos (n := j)
    (by decide : 0 < 2)) (by omega)
    rw [Nat.pow_succ, Nat.mul_comm (2 ^ j) 2, Nat.mul_assoc,
    v2_mul_two _ hpos, ih]

/-- The two-adic valuation is additive on positive arguments. -/
theorem v2_additive (m n : Nat) (hm : 0 < m) (hn : 0 < n) : v2 (m *
n) = v2 m + v2 n := by
  obtain (u, hu, em) := v2_split m hm
  obtain (w, hw, en) := v2_split n hn
  have huw : (u * w) % 2 = 1 := by rw [Nat.mul_mod, hu, hw]
  have e : m * n = 2 ^ (v2 m + v2 n) * (u * w) := by
    calc m * n = (2 ^ v2 m * u) * (2 ^ v2 n * w) := by rw [← em, ← en]
    _ = (2 ^ v2 m * 2 ^ v2 n) * (u * w) := by rw [Nat.mul_assoc,
    Nat.mul_left_comm u, ← Nat.mul_assoc]
    _ = 2 ^ (v2 m + v2 n) * (u * w) := by rw [Nat.pow_add]
  rw [e, v2_pow_odd _ huw]

/-- The valuation at a base p: the count of the factor p in a
number. -/
def vp (p : Nat) : Nat → Nat
| 0 => 0

```

```

| n + 1 => if h : 2 ≤ p ∧ (n + 1) % p = 0 then vp p ((n + 1) / p) +
1 else 0
decreasing_by exact Nat.div_lt_self (Nat.succ_pos n) h.1

/-- ONE MORE FACTOR, ONE MORE COUNT: multiplying a positive number
by the base raises its valuation by one. -/
theorem vp_mul (p : Nat) (hp : 2 ≤ p) (n : Nat) (hn : 0 < n) : vp p
(p * n) = vp p n + 1 := by
obtain (k, hk) : ∃ k, p * n = k + 1 := (p * n - 1, by have :=
Nat.mul_pos (by omega : 0 < p) hn; omega)
rw [hk, vp]
have h1 : 2 ≤ p ∧ (k + 1) % p = 0 := (hp, by rw [← hk]; exact
Nat.mul_mod_right p n)
have h2 : (k + 1) / p = n := by rw [← hk]; exact
Nat.mul_div_cancel_left n (by omega)
rw [dif_pos h1, h2]

/-- NO FACTOR, NO COUNT: a number the base does not divide has
valuation zero. -/
theorem vp_coprime (p : Nat) (h : n % p ≠ 0) : vp p n = 0 := by
cases n with
| zero => exact absurd (Nat.zero_mod p) h
| succ k => rw [vp, dif_neg (fun hc => h hc.2)]

/-- EVERY POSITIVE NUMBER SPLITS: it is the base to its valuation
times a part the base does not divide. -/
theorem vp_split (p : Nat) (hp : 2 ≤ p) : ∀ n, 0 < n → ∃ u, u % p ≠ 0
∧ n = p ^ vp p n * u := by
intro n
induction n using Nat.strongRecOn with
| ind n ih =>
intro hn
by_cases hd : n % p = 0
· obtain (m, rfl) : ∃ m, n = p * m := (n / p,
(Nat.mul_div_cancel' (Nat.dvd_of_mod_eq_zero hd)).symm)
have hm : 0 < m := Nat.pos_of_ne_zero (fun e => by rw [e,
Nat.mul_zero] at hn; exact absurd hn (by decide))
have hlt : m < p * m := by
have := Nat.mul_lt_mul_of_pos_right (show 1 < p by omega) hm
rw [Nat.one_mul] at this; exact this
obtain (u, hu, e) := ih m hlt hm
refine (u, hu, ?_)
rw [vp_mul p hp m hm, Nat.pow_succ]
calc p * m = p * (p ^ vp p m * u) := by rw [← e]
_ = p ^ vp p m * p * u := by rw [Nat.mul_comm p,
Nat.mul_assoc, Nat.mul_comm u p, ← Nat.mul_assoc]
· exact (n, hd, by rw [vp_coprime p n hd, Nat.pow_zero,
Nat.one_mul])

/-- THE VALUATION READS THE EXPONENT: the base to the k times a part
the base does not divide has valuation k. -/
theorem vp_pow_coprime (p : Nat) (hp : 2 ≤ p) (k u : Nat) (hu : u %
p ≠ 0) : vp p (p ^ k * u) = k := by
induction k with
| zero => rw [Nat.pow_zero, Nat.one_mul]; exact vp_coprime p u hu
| succ j ih =>
have hu0 : 0 < u := Nat.pos_of_ne_zero (fun e => hu (by rw [e];
exact Nat.zero_mod p))
have hpos : 0 < p ^ j * u := Nat.mul_pos (Nat.pow_pos (n := j)
(by omega : 0 < p)) hu0
rw [Nat.pow_succ, Nat.mul_comm (p ^ j) p, Nat.mul_assoc, vp_mul
p hp _ hpos, ih]

/-- THE VALUATION IS ADDITIVE AT EVERY EUCLIDEAN BASE: where the
base divides no product of two numbers it divides
neither of, the valuation of a product of positive numbers is
the sum of the valuations. -/
theorem vp_additive (p : Nat) (hp : 2 ≤ p) (euclid : ∀ u w, u % p ≠
0 → w % p ≠ 0 → (u * w) % p ≠ 0)
(m n : Nat) (hm : 0 < m) (hn : 0 < n) : vp p (m * n) = vp p m +
vp p n := by
obtain (u, hu, em) := vp_split p hp m hm
obtain (w, hw, en) := vp_split p hp n hn
have e : m * n = p ^ (vp p m + vp p n) * (u * w) := by
calc m * n = (p ^ vp p m * u) * (p ^ vp p n * w) := by rw [← em,
← en]
_ = (p ^ vp p m * p ^ vp p n) * (u * w) := by rw
[Nat.mul_assoc, Nat.mul_left_comm u, ← Nat.mul_assoc]
_ = p ^ (vp p m + vp p n) * (u * w) := by rw [Nat.pow_add]
rw [e, vp_pow_coprime p hp _ _ (euclid u w hu hw)]

/-- THREE IS EUCLIDEAN: three divides no product of two numbers it
divides neither of. -/

```

```

theorem euclid_at_three (u w : Nat) (hu : u % 3 ≠ 0) (hw : w % 3 ≠
0) : (u * w) % 3 ≠ 0 := by
rw [Nat.mul_mod]
have hu' : u % 3 = 1 ∨ u % 3 = 2 := by omega
have hw' : w % 3 = 1 ∨ w % 3 = 2 := by omega
rcases hu' with h1 | h1 <|> rcases hw' with h2 | h2 <|> rw [h1,
h2] <|> decide

/-- THE LOG IS NOT FORCED OFF THE TOWER OF TWO: the two-adic
valuation and its sum with the three-adic valuation are
both additive on positive arguments and both read one bit at 2,
yet they part at 3, so additivity and the unit
fix the bit count on the powers of two and nowhere else. -/
theorem log_not_forced_off_tower : ∃ f g : Nat → Nat, (∀ m n, 0 < m
→ 0 < n → f (m * n) = f m + f n) ∧
(∀ m n, 0 < m → 0 < n → g (m * n) = g m + g n) ∧ f 2 = 1 ∧ g 2 =
1 ∧ f 3 ≠ g 3 := by
refine (v2, fun n => v2 n + vp 3 n, v2_additive, fun m n hm hn =>
?_, v2_two, ?_, ?_)
· show v2 (m * n) + vp 3 (m * n) = (v2 m + vp 3 m) + (v2 n + vp 3 n)
rw [v2_additive m n hm hn, vp_additive 3 (by decide)
euclid_at_three m n hm hn]
omega
· show v2 2 + vp 3 2 = 1
rw [v2_two, vp_coprime 3 2 (by decide)]
· show v2 3 ≠ v2 3 + vp 3 3
have h3 : vp 3 3 = 1 := by
have := vp_mul 3 (by decide) 1 (by decide)
rw [vp_coprime 3 1 (by decide)] at this
exact this
rw [h3, v2_odd 3 (by decide)]
decide

/-! ### Supply: the wall, the calibration, the crossing -/
def Even {α : Type} (σ : α → α) (f : α → Bool) : Prop := ∀ x, f (σ x)
= f x

/-- THE WALL: an even reading never equals a target odd at a point.
-/
theorem wall {α : Type} (σ : α → α) (f d : α → Bool) (x : α) (he :
Even σ f) (ho : d (σ x) ≠ d x) : f ≠ d :=
fun h => by subst h; exact ho (he x)

/-- THE CALIBRATION: one supplied odd witness fixes the bit, and
fixes it uniquely. -/
theorem calibration {α : Type} (σ : α → α) (s d : α → Bool) (x : α)
(hs : s (σ x) = !s x) (hd : d (σ x) = !d x) :
∃ c : Bool, (d x = xor (s x) c ∧ d (σ x) = xor (s (σ x)) c) ∧
∀ c', (d x = xor (s x) c' ∧ d (σ x) = xor (s (σ x)) c') → c' =
c := by
refine (xor (d x) (s x), (by cases s x <|> cases d x <|> rfl, by
rw [hs, hd]; cases s x <|> cases d x <|> rfl), ?_)
intro c' hc; obtain (h1, -) := hc
generalize hsx : s x = sv at h1; generalize hdx : d x = dv at h1
cases sv <|> cases dv <|> cases c' <|> first | rfl | exact absurd
h1 (by decide)

/-- The executed frame: a formal content and the bit of whether it
ran. Reading the content is formal. -/
abbrev Exec (Q : Type) := Bool × Q
def run {Q : Type} (s : Exec Q) : Exec Q := (!s.1, s.2)
def formalRead {Q : Type} (s : Exec Q) : Q := s.2
def ran {Q : Type} (s : Exec Q) : Bool := s.1

/-- EVERY FORMAL READING IS EVEN. -/
theorem formal_never_odd {Q : Type} (g : Q → Bool) : Even (run (Q :=
Q)) (fun s => g (formalRead s)) :=
fun _ => rfl

/-- A CONSTANT SUPPLIES NOTHING. -/
theorem constant_no_crossing {Q : Type} (b : Bool) (s : Exec Q) : ¬
((fun _ : Exec Q => b) (run s) ≠ b) :=
fun h => h rfl

/-- THE CROSSING WALL: no reading of the formal content returns
whether it ran. -/
theorem no_reading_returns_the_deed {Q : Type} (q : Q) : ¬ ∃ g : Q →
Bool, ∀ s : Exec Q, g (formalRead s) = ran s :=
fun (_, hg) => Bool.noConfusion ((hg (true, q)).symm.trans (hg
(false, q)))

/-- THE THREE LEGS ON ONE ORBIT. The orbit {{1, t}, {-1, t}}, the
seat (0, t). FORGET: both members register to the
seat as one record. PAY: two distinct states sent injectively
below F' need F' at least two. SUPPLY: for any

```

```

    target odd on the orbit, one calibration bit against the side
    reconstructs it, and only one. -/
def side (p : Point) : Bool := decide (0 < p.1)
theorem three_legs_one_orbit (t : Int) (d : Point → Bool) (hd : d
  (fold (1, t)) = !d (1, t)) :
  (online (reg (1, t)) ∧ reg (fold (1, t)) = reg (1, t) ∧ fold (1,
  t) ≠ (1, t)) ∧
  (∀ (F' : Nat) (step : Point → Nat), step (1, t) ≠ step (fold (1,
  t)) →
    step (1, t) < F' → step (fold (1, t)) < F' → 2 ≤ F') ∧
  (∃ c : Bool, (d (1, t) = xor (side (1, t)) c ∧ d (fold (1, t)) =
  xor (side (fold (1, t))) c) ∧
    ∀ c', (d (1, t) = xor (side (1, t)) c' ∧ d (fold (1, t)) = xor
  (side (fold (1, t))) c') → c' = c) := by
  refine ((rfl, rfl, fun h => by cases (congrArg Prod.fst h)), fun
  F' step hne h1 h2 => ?, ?)
· match F', h1, h2 with
| 0, h1, _ => exact absurd h1 (Nat.not_lt_zero _)
| 1, h1, h2 => exact absurd ((Nat.le_zero.mp (Nat.le_of_lt_succ
  h1)).trans
    (Nat.le_zero.mp (Nat.le_of_lt_succ h2)).symm) hne
| n + 2, _, _ => exact Nat.le_add_left 2 n
· exact calibration fold side d (1, t) rfl hd

/-! ## VI · Primes: the Bridge on the multiplicative cut -/
def fib (n : Nat) : Nat := (pairs n).length
def isPrimeTrial (n : Nat) : Bool := 2 ≤ n && ((List.range n).filter
  (fun a => 2 ≤ a && n % a == 0)).isEmpty

set_option maxHeartbeats 4000000 in
set_option maxRecDepth 200000 in
/-! A PRIME IS ONE FREE ORBIT: for every n from 2 to 200, n is prime
  exactly when its ordered factor fibre has two
  points, and then no seat. -/
theorem prime_iff_one_free_orbit :
  (List.range 201).all (fun n => n < 2 || ((isPrimeTrial n == (fib
  n == 2)) && (!isPrimeTrial n || seats n == 0))) = true := by
  decide

/-! TWO MEMBERS, LENGTH TWO: a list of distinct members whose
  members are exactly two distinct things has length
  two. -/
theorem two_members {α : Type} [DecidableEq α] (l : List α) (a b :
  α) (hnd : Distinct l) (hab : a ≠ b)
  (hm : ∀ x, x ∈ l ↔ x = a ∨ x = b) : l.length = 2 := by
  have ha : a ∈ l := (hm a).mpr (Or.inl rfl)
  have hb : b ∈ l := (hm b).mpr (Or.inr rfl)
  have h1 := length_without a l hnd ha
  have h2 := length_without b (without a l) (distinct_without a l
  hnd) hb
  have h3 : without b (without a l) = [] := by
    cases hw : without b (without a l) with
    | nil => rfl
    | cons y r =>
      have hy : y ∈ without b (without a l) := by rw [hw]; exact
      List.Mem.head _
      have hyb := ne_of_mem_without b y _ hy
      have hy' := mem_of_mem_without b y _ hy
      have hya := ne_of_mem_without a y l hy'
      exact absurd ((hm y).mp (mem_of_mem_without a y l hy')) (fun h
      => h.elim hya hyb)
  rw [h3] at h2
  rw [← h1, ← h2]
  rfl

/-! THREE MEMBERS, LENGTH AT LEAST THREE: a list of distinct members
  holding three distinct things has length at least
  three. -/
theorem three_members {α : Type} [DecidableEq α] (l : List α) (a b c :
  α) (hnd : Distinct l)
  (ha : a ∈ l) (hb : b ∈ l) (hc : c ∈ l) (hab : a ≠ b) (hac : a ≠
  c) (hbc : b ≠ c) : 3 ≤ l.length := by
  have hb' : b ∈ without a l := mem_without_of a b l hb (fun e =>
  hab e.symm)
  have hc' : c ∈ without b (without a l) :=
  mem_without_of b c _ (mem_without_of a c l hc (fun e => hac
  e.symm)) (fun e => hbc e.symm)
  have h1 := length_without a l hnd ha
  have h2 := length_without b (without a l) (distinct_without a l
  hnd) hb'
  have h3 : 1 ≤ (without b (without a l)).length := by
    cases hw : without b (without a l) with
    | nil => rw [hw] at hc'; exact nomatch hc'

```

```

    | cons _ _ => exact Nat.succ_le_succ (Nat.zero_le _)
  rw [← h1, ← h2]
  exact Nat.succ_le_succ (Nat.succ_le_succ h3)

/-! THE TRIAL TEST READS PRIMAILTY: for n at least two, the trial
  test passes exactly when no number from two to
  n - 1 divides n. -/
theorem prime_trial_iff (n : Nat) (hn : 2 ≤ n) :
  isPrimeTrial n = true ↔ ∀ a, 2 ≤ a → a < n → n % a ≠ 0 := by
  unfold isPrimeTrial
  rw [decide_eq_true hn, Bool.true_and]
  constructor
· intro h a ha1 ha2 hmod
  have hmem : a ∈ (List.range n).filter (fun a => 2 ≤ a && n % a
  == 0) := by
    rw [List.mem_filter, List.mem_range, Bool.and_eq_true,
    decide_eq_true_eq, beq_iff_eq]
    exact (ha2, ha1, hmod)
  cases hf : (List.range n).filter (fun a => 2 ≤ a && n % a == 0)
  with
  | nil => rw [hf] at hmem; exact nomatch hmem
  | cons _ _ => rw [hf] at h; exact Bool.noConfusion h
· intro h
  cases hf : (List.range n).filter (fun a => 2 ≤ a && n % a == 0)
  with
  | nil => rfl
  | cons x r =>
    have hx : x ∈ (List.range n).filter (fun a => 2 ≤ a && n % a ==
    0) := by rw [hf]; exact List.Mem.head _
    rw [List.mem_filter, List.mem_range, Bool.and_eq_true,
    decide_eq_true_eq, beq_iff_eq] at hx
    exact absurd hx.2.2 (h x hx.2.1 hx.1)

/-! A PRIME IS ONE FREE ORBIT, FOR EVERY n FROM 2: n is prime
  exactly when its ordered factor fibre has two points,
  and then it holds no seat. -/
theorem prime_iff_one_free_orbit_all (n : Nat) (hn : 2 ≤ n) :
  (isPrimeTrial n = true ↔ fib n = 2) ∧ (isPrimeTrial n = true →
  seats n = 0) := by
  have h1n : (1 : Nat) ≠ n := fun e => by omega
  have h1 : 1 ∈ pairs n := (mem_pairs n 1).mpr (by omega, by decide,
  Nat.mod_one n)
  have hnn : n ∈ pairs n := (mem_pairs n n).mpr (Nat.le_refl n, by
  omega, Nat.mod_self n)
  have only : isPrimeTrial n = true → ∀ x, x ∈ pairs n → x = 1 ∨ x =
  n := by
    intro hp x
    have hp' := (prime_trial_iff n hn).mp hp
    constructor
· intro hx
  obtain (hx1, hx2, hx3) := (mem_pairs n x).mp hx
  by_cases hx4 : x = n
  · exact Or.inr hx4
  · by_cases hx5 : x = 1
    · exact Or.inl hx5
    · exact absurd hx3 (hp' x (by omega) (by omega))
· intro h
  rcases h with h | h
  · rw [h]; exact h1
  · rw [h]; exact hnn
  refine ((fun hp => two_members (pairs n) 1 n (pairs_distinct n)
  h1n (only hp), fun hf => ?_), fun hp => ?_)
· apply (prime_trial_iff n hn).mpr
  intro a ha1 ha2 hmod
  have ha : a ∈ pairs n := (mem_pairs n a).mpr (by omega, by
  omega, hmod)
  have h3 := three_members (pairs n) 1 a n (pairs_distinct n) h1
  ha hnn (by omega) (by omega) (by omega)
  have hf' : (pairs n).length = 2 := hf
  omega
· have hn1l : ∀ a, a ∈ (pairs n).filter (fun a => a * a == n) →
  False := fun a ha => by
  have hm := mem_filter_true _ _ a ha
  have hsq : a * a = n := of_decide_eq_true hm.2
  rcases (only hp a).mp hm.1 with e | e
  · rw [e] at hsq; omega
  · rw [e] at hsq
    have := Nat.mul_le_mul_right n hn
    omega
  show ((pairs n).filter (fun a => a * a == n)).length = 0
  cases hf : (pairs n).filter (fun a => a * a == n) with
  | nil => rfl
  | cons x r => exact (hn1l x (by rw [hf]; exact List.Mem.head
  _)).elim

```

```

/-- The six pairs of the parity frame: a prime and a semiprime with
    equal residues mod 420. -/
def k4 : List (Nat × Nat) := [(11, 851), (13, 1273), (17, 437), (19,
    2119), (23, 1703), (29, 869)]
def lg2 : Nat → Nat → Nat
| 0, _ => 0
| k + 1, n => if n < 2 then 0 else lg2 k (n / 2) + 1

set_option maxRecDepth 100000 in
/-- THE PARITY FRAME: residues mod 420 merge each pair; the bits
    paid by registering the product separate it, one
    against two, and the parity of the bits paid is the sign: odd at
    the prime, even at the semiprime. -/
theorem parity_frame_paid_bits :
  k4.all (fun p => p.1 % 420 == p.2 % 420 && lg2 64 (fib p.1) == 1
    && lg2 64 (fib p.2) == 2) = true := by
  decide

def divsN (n : Nat) : List Nat := (List.range (n + 1)).filter (fun d
  => 0 < d && n % d == 0)
def omegaIn (ps : List Nat) (d : Nat) : Nat := (ps.filter (fun p =>
  d % p == 0)).length
def toggle (p d : Nat) : Nat := if d % p == 0 then d / p else d * p
def sig30 (d : Nat) : Bool × Bool × Bool := (d % 2 == 0, d % 3 == 0,
  d % 5 == 0)

set_option maxRecDepth 100000 in
/-- THE EIGHT ARE THE DIVISOR CUBE: the eight divisors of 30 are the
    eight sign patterns, one to one, and the
    Möbius sign is the orientation of the pattern. -/
theorem divisor_cube_is_sign_cube :
  divsN 30 = [1, 2, 3, 5, 6, 10, 15, 30] ∧ ((divsN 30).map
  sig30).eraseDups.length = 8 ∧
  (divsN 30).all (fun d => orient (sig30 d) == (if omegaIn [2, 3,
  5] d % 2 == 0 then 1 else -1)) = true := by
  decide

set_option maxRecDepth 200000 in
/-- THE FIBRE OVER 2310 BALANCES: on the 32 divisors of 2·3·5·7·11
    the toggle of 2 is a free involution that flips
    the sign, so the signs balance sixteen against sixteen;
    `every_fibre_over_more_than_one_balances` carries the
    balance to the subsets of every nonempty list of primes. -/
theorem fibre_balance :
  (divsN 2310).length = 32 ∧
  (divsN 2310).all (fun d => (divsN 2310).contains (toggle 2 d) &&
  toggle 2 d != d &&
  toggle 2 (toggle 2 d) == d &&
  (omegaIn [2, 3, 5, 7, 11] d + omegaIn [2, 3, 5, 7, 11]
  (toggle 2 d)) % 2 == 1) = true ∧
  ((divsN 2310).filter (fun d => omegaIn [2, 3, 5, 7, 11] d % 2 ==
  0)).length = 16 := by decide

/-- THE FILTER SPLITS OVER A JOIN: the filtered length of a join is
    the sum of the filtered lengths. -/
theorem filter_append_length {α : Type} (q : α → Bool) (l m : List
  α) :
  ((l ++ m).filter q).length = (l.filter q).length + (m.filter
  q).length := by
  induction l with
| nil => exact (Nat.zero_add _).symm
| cons a l ih =>
  cases h : q a with
| true =>
  show (List.filter q (a :: (l ++ m))).length = (List.filter q
  (a :: l)).length + _
  rw [filter_cons_true q a _ h, filter_cons_true q a _ h]
  show (List.filter q (l ++ m)).length + 1 = (List.filter q
  l).length + 1 + _
  rw [ih, Nat.add_right_comm]
| false =>
  show (List.filter q (a :: (l ++ m))).length = (List.filter q
  (a :: l)).length + _
  rw [filter_cons_false q a _ h, filter_cons_false q a _ h]
  exact ih

/-- PREFIXING SHIFTS THE TEST: filtering the lists prefixed by p
    under one test counts the lists that pass the
    shifted test. -/
theorem filter_map_cons_length (q₁ q₂ : List Nat → Bool) (p : Nat)
  (hq : ∀ s, q₁ (p :: s) = q₂ s)
  (l : List (List Nat)) : ((l.map (p :: ·)).filter q₁).length =
  (l.filter q₂).length := by

```

```

induction l with
| nil => rfl
| cons s l ih =>
  cases h : q₂ s with
| true =>
  show (List.filter q₁ ((p :: s) :: l.map (p :: ·))).length =
  (List.filter q₂ (s :: l)).length
  rw [filter_cons_true q₁ _ _ ((hq s).trans h), filter_cons_true
  q₂ s l h]
  show (List.filter q₁ (l.map (p :: ·))).length + 1 =
  (List.filter q₂ l).length + 1
  rw [ih]
| false =>
  show (List.filter q₁ ((p :: s) :: l.map (p :: ·))).length =
  (List.filter q₂ (s :: l)).length
  rw [filter_cons_false q₁ _ _ ((hq s).trans h),
  filter_cons_false q₂ s l h]
  exact ih

/-- ONE MORE MEMBER FLIPS THE PARITY: the successor of n is even
    exactly when n is odd, and odd exactly when n is
    even. -/
theorem mod_two_flips (n : Nat) :
  ((n + 1) % 2 == 0) = (n % 2 == 1) ∧ ((n + 1) % 2 == 1) = (n % 2
  == 0) := by
  have h : n % 2 = 0 ∨ n % 2 = 1 := by omega
  rcases h with h | h
  · have h' : (n + 1) % 2 = 1 := by omega
    rw [h, h']; exact (rfl, rfl)
  · have h' : (n + 1) % 2 = 0 := by omega
    rw [h, h']; exact (rfl, rfl)

/-- The subsets of a list, each member left out or kept: over a list
    of distinct primes, the squarefree divisors of
    their product. -/
def subs : List Nat → List (List Nat)
| [] => [[]]
| p :: r => subs r ++ (subs r).map (p :: ·)

/-- EVERY FIBRE OVER MORE THAN ONE BALANCES: over every nonempty
    list, the subsets of even size and the subsets of odd
    size are equally many, since keeping or leaving out the first
    member flips the parity of the size. -/
theorem every_fibre_over_more_than_one_balances (p : Nat) (r : List
  Nat) :
  ((subs (p :: r)).filter (fun s => s.length % 2 == 0)).length =
  ((subs (p :: r)).filter (fun s => s.length % 2 == 1)).length
  := by
  show ((subs r ++ (subs r).map (p :: ·)).filter (fun s => s.length
  % 2 == 0)).length =
  ((subs r ++ (subs r).map (p :: ·)).filter (fun s => s.length % 2
  == 1)).length
  rw [filter_append_length, filter_append_length,
  filter_map_cons_length _ (fun s => s.length % 2 == 1) p (fun s
  => (mod_two_flips s.length).1),
  filter_map_cons_length _ (fun s => s.length % 2 == 0) p (fun s
  => (mod_two_flips s.length).2)]
  exact Nat.add_comm _ _

/-! ## VII · Wall, edge, seat -/

/-- The edge and the wall on the offset chart, d = 2 Re s - 1: the
    wall Re s = 1 is d = 1, the edge Re s = 0 is
    d = -1, and the fold exchanges them; the seat d = 0 is their
    midpoint. -/
theorem wall_and_edge_exchanged (t : Int) : fold (1, t) = (-1, t) ∧
  fold (-1, t) = (1, t) ∧ onLine (0, t) :=
  (rfl, rfl, rfl)

/-- Every prime's own mode p^(-s), at the depth Re s (here the
    exponent, not the chart offset), is unitary at exactly one
    depth: the exponent zero, the edge Re s = 0. -/
theorem prime_mode_unitary_iff (p d : Nat) (hp : 2 ≤ p) : p ^ d = 1
  ↔ d = 0 := by
  constructor
  · intro h
  cases d with
  | zero => rfl
  | succ k =>
    have h1 : p ^ 1 ≤ p ^ (k + 1) := Nat.pow_le_pow_right (by
    omega) (by omega)
    rw [Nat.pow_one] at h1
    omega
  · intro h; rw [h]; rfl

```

```

/-- NO POSITIVE POWER OF A BASE ABOVE ONE IS ONE: p to the k + 1 is
    at least p, and p is at least two. -/
theorem pow_succ_ne_one (p k : Nat) (hp : 2 ≤ p) : p ^ (k + 1) ≠ 1
:= fun h =>
  match hk : p ^ k with
  | 0 => by rw [Nat.pow_succ, hk, Nat.zero_mul] at h; exact
    Nat.noConfusion h
  | m + 1 => by
    rw [Nat.pow_succ, hk, Nat.succ_mul] at h
    exact absurd (h ▸ Nat.le_trans hp (Nat.le_add_left p (m * p)))
    (by decide)

/-- EVERY PRIME MODE IS UNITARY ONLY AT DEPTH ZERO, ON INTEGER
    DEPTHS: p to the absolute depth is one exactly when the
    integer depth is zero, on either side of the edge. -/
theorem prime_mode_unitary_iff_int (p : Nat) (d : Int) (hp : 2 ≤ p)
: p ^ d.natAbs = 1 ↔ d = 0 :=
  (fun h => match d, h with
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (k + 1), h => absurd h (pow_succ_ne_one p k hp)
  | Int.negSucc k, h => absurd h (pow_succ_ne_one p k hp),
  fun h => by subst h; rfl)

/-- At s units of the floor per bit, the state 2^a weighs 2^(-a*s):
    s is the price per bit, and s = 1 is the
    floor itself. -/
theorem price_per_bit_tower (a s : Nat) : (2 ^ a) ^ s = 2 ^ (a * s)
:= (Nat.pow_mul 2 a s).symm

theorem len_le_sum : ∀ l : List Nat, (∀ x ∈ l, 1 ≤ x) → l.length ≤
  l.sum
| [], _ => Nat.le_refl 0
| x :: r, h => by
  have hx := h x (List.Mem.head r)
  have ih := len_le_sum r (fun y hy => h y (List.Mem.tail x hy))
  simp only [List.length_cons, List.sum_cons]
  omega

/-- THE WALL OF COUNTING: the block (2^j, 2^(j+1)] of shares
    one-in-n holds, in units of 1/2^(j+1), at least
    2^j units, a half. Every block holds a half, so the shares of
    all states have no finite total at s = 1. -/
theorem wall_block_share (j : Nat) :
  2 ^ j ≤ ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^ j
  + 1 + i))).sum := by
  have hlen := ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^
  j + 1 + i))).length = 2 ^ j := by simp
  have key := len_le_sum ((List.range (2 ^ j)).map (fun i => 2 ^ (j
  + 1) / (2 ^ j + 1 + i))) (by
  intro x hx
  obtain {i, hi, rfl} := List.mem_map.mp hx
  have hi' : i < 2 ^ j := List.mem_range.mp hi
  have hle : 2 ^ j + 1 + i ≤ 2 ^ (j + 1) := by rw [Nat.pow_succ];
  omega
  exact (Nat.le_div_iff_mul_le (by omega)).mpr (by omega))
  rw [hlen] at key
  exact key

/-! ## VIII · The heat flow: the fold pulls every pair onto its seat
-/

/-- Conjugation of the flow variable is the fold: with s = 1/2 + i z
    and Im z = y, the offset is d = -2y, and
    y ↦ -y sends d to -d. -/
theorem conjugation_is_the_fold (y : Int) : -2 * -y = -(-2 * y) :=
  Int.mul_neg (-2) y

/-- The quadratic z^2 + b z + c under the flow c ↦ c - 2t. -/
def flow (t : Int) (q : Int × Int) : Int × Int := (q.1, q.2 - 2 * t)
theorem flow_inverse (t : Int) (q : Int × Int) : flow (-t) (flow t
q) = q := by
  obtain (b, c) := q
  simp only [flow, Prod.mk.injEq, true_and]
  omega

/-- THE FLOW MERGES NOTHING: it is injective, so under the floor it
    exports nothing and costs nothing. -/
theorem flow_injective (t : Int) (p q : Int × Int) (h : flow t p =
flow t q) : p = q := by
  rw [← flow_inverse t p, ← flow_inverse t q, h]

/-- The discriminant rises by exactly eight per unit of time. -/
theorem disc_flow (D c t : Int) : D - 4 * (c - 2 * t) = (D - 4 * c) +
8 * t := by omega

/-- The pull in squared form: the squared imaginary part of the pair
    of z^2 + c is c - 2t, falling by two per unit
    of time, which is the partner's pull -1/b on b. -/
theorem pull_in_square (c t : Int) : c - 2 * (t + 1) = (c - 2 * t) -
2 := by omega

/-- Collision, doubled time u = 2t: a pair while u < c, the seat at
    u = c, two fixed points after. -/
theorem collision_sign (c u : Int) :
  (u < c → 4 * u - 4 * c < 0) ∧ (u = c → 4 * u - 4 * c = 0) ∧ (c <
  u → 0 < 4 * u - 4 * c) :=
  (fun _ => by omega, fun _ => by omega, fun _ => by omega)

/-- Zero slack: the double root on the seat leaves it under every
    backward step. -/
theorem zero_slack (t : Int) (ht : t < 0) : 0 - 4 * (0 - 2 * t) < 0
:= by omega

/-- The only merge is the certificate: real-rootedness at a positive
    time takes one value on two states. -/
def realAt (t : Int) (q : Int × Int) : Bool := decide (0 ≤ q.1 * q.1
- 4 * (q.2 - 2 * t))
theorem certificate_merges : realAt 1 (0, 0) = realAt 1 (0, 1) ∧
  ((0, 0) : Int × Int) ≠ (0, 1) ∧
  realAt 0 (0, 0) ≠ realAt 0 (0, 1) := by decide

/-! ## IX · The physical carrier -/

/-- Charge in thirds of the electron's charge; charge conjugation is
    negation; the carrier puts charge on the
    offset, d = 0, with Re s counted in twelfths: the Notation's
    chart at resolution six. -/
def conj (q : Int) : Int := -q
def carrier (q : Int) : Point := (q, 0)
/-- The carrier is equivariant, neutral lands on the line, and the
    electron and positron are one off-line pair of
    one record. -/
theorem charge_carrier :
  (∀ q : Int, carrier (conj q) = fold (carrier q)) ∧ online
  (carrier 0) ∧ ¬ online (carrier (-3)) ∧
  fold (carrier (-3)) = carrier 3 ∧ reg (carrier 3) = reg (carrier
  (-3)) :=
  (fun _ => rfl, rfl, fun h => (by decide : ¬ ((-3 : Int) = 0)) h,
  rfl, rfl)

/-! ## X · Inhabitation: every conditional law the harness lists has
    a model of its hypotheses -/
theorem value_from_the_act_inhabited : Nonempty ActualZeros :=
  ((online, fun p h => by show -p.1 = 0; rw [show p.1 = 0 from h];
  rfl, fun p h => (reg_erases_nothing_iff p).mpr h))
theorem off_line_inhabited : ∃ p : Point, ¬ online p := ((1, 0), fun
h => by cases h)
theorem weakest_forcing_premise_inhabited :
  ∃ P : Config → Prop, (∀ S, P S → Value S) ∧ (∀ S, Value S → P S)
:= (Value, fun _ h => h, fun _ h => h)
theorem keyless_forces_nothing_inhabited : ∃ Q : Config → Prop, ∀ S,
Closed S → Q S :=
  (fun _ => True, fun _ _ => trivial)
theorem count_to_heat_inhabited :
  ND [0, 1] ∧ [0, 1].length = 2 * 2 ^ 0 ∧ (∀ a ∈ [0, 1], ∀ x ∈ [0,
  1], (fun n : Nat => n) a = (fun n : Nat => n) x → a = x) ∧
  (∀ a ∈ [0, 1], (fun n : Nat => n) a < 1 * 2 ^ 1) ∧ 2 = 2 * 1 ∧ 0
  < 1 := by
  refine ((by decide, by decide, trivial), rfl, fun a _ x _ h => h,
  ?, rfl, by decide)
  intro a ha
  cases ha with
  | head => decide
  | tail _ h => cases h with
  | head => decide
  | tail _ h2 => cases h2
theorem bits_forced_inhabited :
  ∃ f : Nat → Nat, (∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n) ∧
  f 2 = 1 := (v2, v2_additive, v2_two)
theorem calibration_inhabited :
  (fun b : Bool => b) ((fun b : Bool => !b) true) = !((fun b :
  Bool => b) true) := rfl
theorem three_legs_inhabited (t : Int) : side (fold (1, t)) = !side
(1, t) := rfl
theorem prime_mode_unitary_inhabited : 2 ≤ 2 := Nat.le_refl 2
theorem zero_slack_inhabited : (-1 : Int) < 0 := by decide
theorem record_carried_inhabited (S : Config) : SameRecord (Rec S) S
  ∧ Value (Rec S) :=
  (fun q => ((record_same S) q).symm, record_is_lossless S)
theorem price_zero_inhabited : 0 < 1 := Nat.succ_pos 0
theorem sign_group_inhabited :

```

```

Distinct [[false], [true]] ∧ (∀ x ∈ [[false], [true]], ∀ y ∈
[[false], [true]], x.length = y.length) ∧
  (∀ x ∈ [[false], [true]], ∀ y ∈ [[false], [true]],
List.zipWith xor x y ∈ [[false], [true]]) := by
have two : ∀ x, x ∈ [[false], [true]] → x = [false] ∨ x = [true]
:= fun x hx =>
  match hx with
  | List.Mem.head _ => Or.inl rfl
  | List.Mem.tail _ (List.Mem.head _) => Or.inr rfl
  | List.Mem.tail _ (List.Mem.tail _ h) => nomatch h
refine (And.intro (fun h => ?_) (And.intro (fun h => nomatch h)
trivial), fun x hx y hy => ?_, fun x hx y hy => ?_)
· cases h with
| tail _ h' => cases h'
· rcases two x hx with rfl | rfl <=> rcases two y hy with rfl |
rfl <=> rfl
· rcases two x hx with rfl | rfl <=> rcases two y hy with rfl |
rfl <=>
  first | exact List.Mem.head _ | exact List.Mem.tail _
(List.Mem.head _)
theorem parity_law_inhabited :
  Distinct (pairs 36) ∧ (∀ x, x ∈ pairs 36 → 36 / x ∈ pairs 36) ∧
  (∀ x, x ∈ pairs 36 → 36 / (36 / x) = x) ∧
  (pairs 36).length = 9 :=
(pairs_distinct 36, swap_mem 36, swap_involutive 36,
nine_is_eight_and_a_seat.1)

/-- A model of the free-involution law: negation on the two
one-letter words is distinct, closed, an involution,
fixes nothing, and the size, two, is even. -/
theorem free_involution_inhabited :
  Distinct [[false], [true]] ∧
  (∀ x, x ∈ [[false], [true]] → List.zipWith xor x [true] ∈
[[false], [true]]) ∧
  (∀ x, x ∈ [[false], [true]] → List.zipWith xor (List.zipWith xor
x [true]) [true] = x) ∧
  (∀ x, x ∈ [[false], [true]] → List.zipWith xor x [true] ≠ x) ∧
  parity ([[false], [true]] : List (List Bool)).length = false :=
(sign_group_inhabited.1, by decide, by decide, by decide, rfl)

/-! ## XI · The Bridge, whole -/

/-- What the Bridge says on no axiom at all. -/
def BridgeAtom : Prop :=
  (∀ p : Point, fold (fold p) = p) ∧ (∀ p : Point, fold p = p ↔
online p) ∧
  (∀ p : Point, onLine (reg p) ∧ (reg p).2 = p.2 ∧ reg (fold p) =
reg p) ∧
  (∀ p : Point, ¬ onLine p → ¬ ∃ g : Point → Point, g (reg p) = p ∧
g (reg (fold p)) = fold p) ∧
  (∀ S : Config, LeastErasure S ↔ Value S) ∧
  (∀ Z : ActualZeros, Value Z.zeros) ∧
  (∀ (p : Point), ¬ onLine p → ∀ g : Config → Prop, (∀ S S',
SameRecord S S' → (g S ↔ g S')) →
¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld
p) ↔ Value (pairWorld p)))) ∧
  (∀ Q : Config → Prop, (∀ S, Closed S → Q S) → ¬ ∀ S, Closed S → Q
S → Value S) ∧
  (∀ P : Config → Prop, (∀ S, P S → Value S) → (∀ S, Value S → P S)
→ ∀ S, P S ↔ Value S) ∧
  (∀ X : Frame, (∃ b : Carrier X, b.terminal = .bot) ↔
LineProperty X) ∧
  (¬ ∃ g : Tri → Tri, ∀ v, g (delete v) = v) ∧
  ((∀ b : Bool, twoPoint.τ (twoPoint.τ b) = b) ∧ (∀ b : Bool,
twoPoint.τ b ≠ b) ∧ ¬ LineProperty twoPoint) ∧
  (∀ X : Frame, Nonempty (Supplied X) ↔ LineProperty X) ∧
  (∀ {Q : Type} ( _ : Q), ¬ ∃ g : Q → Bool, ∀ s : Exec Q, g
(formalRead s) = ran s) ∧
  (∀ (t : Int) (d : Point → Bool), d (fold (1, t)) = !d (1, t) →
onLine (reg (1, t)) ∧ reg (fold (1, t)) = reg (1, t) ∧ fold
(1, t) ≠ (1, t)) ∧
  (∀ (F' : Nat) (step : Point → Nat), step (1, t) ≠ step (fold
(1, t)) →
step (1, t) < F' → step (fold (1, t)) < F' → 2 ≤ F') ∧
  (∃ c : Bool, (d (1, t) = xor (side (1, t)) c) ∧ d (fold (1, t))
= xor (side (fold (1, t))) c) ∧
  ∀ c', (d (1, t) = xor (side (1, t)) c') ∧ d (fold (1, t)) =
xor (side (fold (1, t))) c' → c' = c) ∧
  ((∀ q : Int, carrier (conj q) = fold (carrier q)) ∧ onLine
(carrier 0) ∧ ¬ onLine (carrier (-3))) ∧
  fold (carrier (-3)) = carrier 3 ∧ reg (carrier 3) = reg
(carrier (-3))) ∧
  (List.range 64).all (fun m => (List.range 4).all (fun t =>
sols2 (row3 (m / 8)) (row3 (m % 8)) (t / 2) (t % 2) != 1)) =
true ∧
  a4.length = 12 ∧ cube.length = 8 ∧ (pairs 36).length = 9 ∧ seats
36 = 1 ∧
  (List.range 201).all (fun n => n == 0 || ((pairs n).length % 2
== seats n % 2 && seats n ≤ 1)) = true ∧
  (List.range 201).all (fun n => n < 2 || ((isPrimeTrial n == (fib
n == 2)) && (!isPrimeTrial n || seats n == 0))) = true ∧
  k4.all (fun p => p.1 % 420 == p.2 % 420 && lg2 64 (fib p.1) == 1
&& lg2 64 (fib p.2) == 2) = true ∧
  ((divsN 2310).filter (fun d => omegaIn [2, 3, 5, 7, 11] d % 2 ==
0)).length = 16 ∧
  (realAt 1 (0, 0) = realAt 1 (0, 1) ∧ ((0, 0) : Int × Int) ≠ (0,
1) ∧ realAt 0 (0, 0) ≠ realAt 0 (0, 1)) ∧
  (∀ S : Config, Extremal S ↔ Value S) ∧
  (∀ S S' : Config, SameRecord S' S → Value S' → ∀ q, S' q ↔ Rec S
q) ∧
  (∀ F : FinCfg, LeastErasure F.pts ↔ erased F = 0) ∧
  (∀ (F : FinCfg) (T : Nat), 0 < T → (price T (erased F) = 0 ↔
Value F.pts))

/-- THE ATOM OF THE BRIDGE, ON NO AXIOM: the fold and its line; the
registration; the lost bit; least erasure as
the value; the act; the record, the keyless premise and the
weakest premise; the carrier, its irreversibility,
the coherent denial and the socket; the deed no reading returns;
the three legs on one orbit; the charge
carrier; two axes never lock; twelve, eight, and nine as eight
and a seat; odd exactly when seated; a prime as
one free orbit; the parity frame paid in bits; the balance of a
fibre; the certificate's merge; leastness in the
fibre as the value; the record carried uniquely; least erasure
as zero erased bits; and the heat of registration
zero exactly at the value. -/
theorem the_bridge_atom : BridgeAtom :=
  (fold_involutive, the_cut_is_the_line, fun p => (reg_lands p,
reg_keeps_height p, reg_forgets_side p),
one_bit_lost, least_erasure_iff_value, value_from_the_act,
record_decides_nothing, keyless_forces_nothing,
weakest_forcing_premise, halted_iff, cannot_reverse,
denial_is_coherent, socket_is_the_property,
fun q => no_reading_returns_the_deed q, three_legs_one_orbit,
charge_carrier, two_axes_never_lock,
twelve_gates_one_torsor.1, eight_patterns_four_orbits.1,
nine_is_eight_and_a_seat.1,
nine_is_eight_and_a_seat.2.1, odd_fibre_iff_seat,
prime_iff_one_free_orbit, parity_frame_paid_bits,
fibre_balance.2.2, certificate_merges, extremal_iff_value,
record_carried_uniquely,
least_erasure_iff_zero_erased, price_zero_iff_value)

/-- THE BRIDGE FROM FIRST PRINCIPLE: the atom, and on the standard
axioms the rest. The line property is keyed and
the fold law keyless, and every frame property is one or the
other; no ninth reflection on any number of
axes; under P1 a halving exports a bit priced at one T, and P1
is load-bearing; additivity over zero is dry and
on positive arguments forces the bit count, with the two-adic
valuation as its model; every prime mode is
unitary only at the edge; the counting wall; the price per bit;
the flow merges nothing, collides on the seat,
pulls by two per unit time and leaves the seat backward;
conjugation is the fold. -/
theorem the_bridge_from_first_principle :
  BridgeAtom ∧
  (Keyed LineProperty ∧ ¬ Keyless LineProperty) ∧ Keyless (fun X
=> ∀ s, X.τ (X.τ s) = s) ∧
  (∀ Q : Frame → Prop, (Keyless Q ∨ Keyed Q) ∧ ¬ (Keyless Q ∧
Keyed Q)) ∧
  (∀ k : Nat, 2 ^ k ≠ 9) ∧
  (∀ (L : List Nat) (step : Nat → Nat) (C C' b b' T : Nat), ND L →
L.length = C * 2 ^ b →
  (∀ a ∈ L, ∀ x ∈ L, step a = step x → a = x) → (∀ a ∈ L, step a
< C' * 2 ^ b' → C = 2 * C' → 0 < C' →
  b + 1 ≤ b' ∧ price T 1 ≤ price T (b' - b)) ∧
  ¬ (∀ a ∈ [0, 1], ∀ x ∈ [0, 1], (fun _ : Nat => (0 : Nat)) a =
(fun _ : Nat => (0 : Nat)) x → a = x) ∧
  (∀ f : Nat → Nat, (∀ m n, f (m * n) = f m + f n) → ∀ n, f n = 0) ∧
  (∀ f : Nat → Nat, (∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n)
→ f 2 = 1 → ∀ a, f (2 ^ a) = a) ∧
  (∃ f : Nat → Nat, (∀ m n, 0 < m → 0 < n → f (m * n) = f m + f n)
∧ f 2 = 1) ∧
  (∀ p d : Nat, 2 ≤ p → (p ^ d = 1 ↔ d = 0)) ∧
  (∀ a s : Nat, (2 ^ a) ^ s = 2 ^ (a * s)) ∧

```

```

(∀ j : Nat, 2 ^ j ≤ ((List.range (2 ^ j)).map (fun i => 2 ^ (j + 1) / (2 ^ j + 1 + i))).sum) ∧
(∀ (t : Int) (p q : Int × Int), flow t p = flow t q → p = q) ∧
(∀ c u : Int, (u < c → 4 * u - 4 * c < 0) ∧ (u = c → 4 * u - 4 * c = 0) ∧ (c < u → 0 < 4 * u - 4 * c)) ∧
(∀ c t : Int, c - 2 * (t + 1) = (c - 2 * t) - 2) ∧
(∀ t : Int, t < 0 → 0 - 4 * (0 - 2 * t) < 0) ∧
(∀ y : Int, -2 * -y = -(2 * y)) :=
(the_bridge_atom, line_property_is_keyed, symmetry_is_keyless,
discriminator, no_reflection_group_of_nine,
count_to_heat, merge_exports_nothing.2.2,
additivity_over_zero_is_dry, bits_forced, bits_forced_inhabited,
prime_mode_unitary_iff, price_per_bit_tower, wall_block_share,
flow_injective, collision_sign, pull_in_square,
zero_slack, conjugation_is_the_fold)

end BFP

/-! ## The cones, pinned. Each line is the compiler's own; a drifted cone fails the compile. -/
/-- info: 'BFP.neg_neg_free' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.neg_neg_free
/-- info: 'BFP.neg_self_zero' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.neg_self_zero
/-- info: 'BFP.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.fold_involutive
/-- info: 'BFP.the_cut_is_the_line' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.the_cut_is_the_line
/-- info: 'BFP.reg_lands' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_lands
/-- info: 'BFP.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_keeps_height
/-- info: 'BFP.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_forgets_side
/-- info: 'BFP.reg_erases_nothing_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reg_erases_nothing_iff
/-- info: 'BFP.off_line_pair' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.off_line_pair
/-- info: 'BFP.one_bit_lost' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.one_bit_lost
/-- info: 'BFP.least_erase_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.least_erase_iff_value
/-- info: 'BFP.nothing_escapes_least_erase' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.nothing_escapes_least_erase
/-- info: 'BFP.pair_world_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pair_world_closed
/-- info: 'BFP.registered_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.registered_closed
/-- info: 'BFP.one_record' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.one_record
/-- info: 'BFP.registered_has_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.registered_has_value
/-- info: 'BFP.pair_world_lacks_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pair_world_lacks_value
/-- info: 'BFP.record_decides_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_decides_nothing
/-- info: 'BFP.keyless_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.keyless_forces_nothing
/-- info: 'BFP.weakest_forcing_premise' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.weakest_forcing_premise
/-- info: 'BFP.value_from_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.value_from_the_act
/-- info: 'BFP.reader_frame' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.reader_frame
/-- info: 'BFP.record_is_lossless' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_is_lossless
/-- info: 'BFP.record_same' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_same
/-- info: 'BFP.extremal_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.extremal_iff_value
/-- info: 'BFP.record_carried_uniquely' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.record_carried_uniquely
/-- info: 'BFP.chart_line_property' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.chart_line_property
/-- info: 'BFP.cannot_lie' does not depend on any axioms -/

#guard_msgs in #print axioms BFP.cannot_lie
/-- info: 'BFP.cannot_deviate' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.cannot_deviate
/-- info: 'BFP.halted_iff' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.halted_iff
/-- info: 'BFP.cannot_reverse' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.cannot_reverse
/-- info: 'BFP.cannot_extend' depends on axioms: [Quot.sound] -/
#guard_msgs in #print axioms BFP.cannot_extend
/-- info: 'BFP.denial_is_coherent' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.denial_is_coherent
/-- info: 'BFP.discriminator' depends on axioms: [propxet, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BFP.discriminator
/-- info: 'BFP.symmetry_is_keyless' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.symmetry_is_keyless
/-- info: 'BFP.line_property_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.line_property_is_keyed
/-- info: 'BFP.socket_is_the_property' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.socket_is_the_property
/-- info: 'BFP.no_socket_off_line' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.no_socket_off_line
/-- info: 'BFP.act_seats_the_carrier' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.act_seats_the_carrier
/-- info: 'BFP.two_axes_never_lock' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.two_axes_never_lock
/-- info: 'BFP.three_axes_lock' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.three_axes_lock
/-- info: 'BFP.third_begotten' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.third_begotten
/-- info: 'BFP.no_fourth_axis' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.no_fourth_axis
/-- info: 'BFP.twelve_gates_one_torsor' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.twelve_gates_one_torsor
/-- info: 'BFP.eight_patterns_four_orbits' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.eight_patterns_four_orbits
/-- info: 'BFP.no_reflection_group_of_nine' depends on axioms: [propxet] -/
#guard_msgs in #print axioms BFP.no_reflection_group_of_nine
/-- info: 'BFP.nine_is_eight_and_a_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.nine_is_eight_and_a_seat
/-- info: 'BFP.odd_fibre_iff_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.odd_fibre_iff_seat
/-- info: 'BFP.flip_flip' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.flip_flip
/-- info: 'BFP.mem_head_or_tail' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.mem_head_or_tail
/-- info: 'BFP.mem_without_of' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.mem_without_of
/-- info: 'BFP.mem_of_mem_without' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.mem_of_mem_without
/-- info: 'BFP.ne_of_mem_without' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.ne_of_mem_without
/-- info: 'BFP.without_of_not_mem' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.without_of_not_mem
/-- info: 'BFP.distinct_without' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.distinct_without
/-- info: 'BFP.length_without' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.length_without
/-- info: 'BFP.seats_without' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.seats_without
/-- info: 'BFP.seats_zero_of_free' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.seats_zero_of_free
/-- info: 'BFP.parity_bounded' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.parity_bounded
/-- info: 'BFP.involution_parity' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.involution_parity
/-- info: 'BFP.odd_holds_a_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.odd_holds_a_seat
/-- info: 'BFP.nine_holds_a_seat' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.nine_holds_a_seat
/-- info: 'BFP.free_orbits_are_even' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.free_orbits_are_even
/-- info: 'BFP.xor_twice' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.xor_twice
/-- info: 'BFP.xor_length' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.xor_length

```

```

/-- info: 'BFP.xor_fixed_is_zero' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.xor_fixed_is_zero
/-- info: 'BFP.xor_zero_eq' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.xor_zero_eq
/-- info: 'BFP.no_group_of_sign_patterns_of_nine' does not depend on
any axioms -/
#guard_msgs in #print axioms BFP.no_group_of_sign_patterns_of_nine
/-- info: 'BFP.filter_cons_true' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.filter_cons_true
/-- info: 'BFP.filter_cons_false' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.filter_cons_false
/-- info: 'BFP.mem_filter_true' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.mem_filter_true
/-- info: 'BFP.pairwise_filter' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pairwise_filter
/-- info: 'BFP.range_loop_pairwise' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.range_loop_pairwise
/-- info: 'BFP.distinct_of_pairwise_lt' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.distinct_of_pairwise_lt
/-- info: 'BFP.pairs_pairwise' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pairs_pairwise
/-- info: 'BFP.pairs_distinct' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pairs_distinct
/-- info: 'BFP.squares_strict' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.squares_strict
/-- info: 'BFP.seats_le_one' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.seats_le_one
/-- info: 'BFP.mem_pairs' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.mem_pairs
/-- info: 'BFP.cofactor_pos' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.cofactor_pos
/-- info: 'BFP.swap_mem' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.swap_mem
/-- info: 'BFP.swap_involutive' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.swap_involutive
/-- info: 'BFP.swap_seat_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.swap_seat_iff
/-- info: 'BFP.seats_of_swap' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.seats_of_swap
/-- info: 'BFP.nine_is_eight_and_a_seat_all' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.nine_is_eight_and_a_seat_all
/-- info: 'BFP.allLt_mem' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.allLt_mem
/-- info: 'BFP.nd_map' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.nd_map
/-- info: 'BFP.allLt_map' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.allLt_map
/-- info: 'BFP.pigeonhole' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.pigeonhole
/-- info: 'BFP.export_doubles' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.export_doubles
/-- info: 'BFP.one_bit_arrives' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.one_bit_arrives
/-- info: 'BFP.count_to_heat' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.count_to_heat
/-- info: 'BFP.merge_exports_nothing' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.merge_exports_nothing
/-- info: 'BFP.fincfg_closed' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.fincfg_closed
/-- info: 'BFP.least_erasure_iff_zero_erased' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.least_erasure_iff_zero_erased
/-- info: 'BFP.price_zero_iff_value' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.price_zero_iff_value
/-- info: 'BFP.price_counts_pairs' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.price_counts_pairs
/-- info: 'BFP.heat_reads_past_the_record' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.heat_reads_past_the_record
/-- info: 'BFP.additivity_over_zero_is_dry' depends on axioms:
[propext, Quot.sound] -/

```

```

#guard_msgs in #print axioms BFP.additivity_over_zero_is_dry
/-- info: 'BFP.linear_on_tower' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.linear_on_tower
/-- info: 'BFP.bits_forced' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.bits_forced
/-- info: 'BFP.v2_mul_two' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.v2_mul_two
/-- info: 'BFP.v2_odd' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_odd
/-- info: 'BFP.v2_two' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_two
/-- info: 'BFP.v2_split' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.v2_split
/-- info: 'BFP.v2_pow_odd' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.v2_pow_odd
/-- info: 'BFP.v2_additive' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.v2_additive
/-- info: 'BFP.vp_mul' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.vp_mul
/-- info: 'BFP.vp_coprime' depends on axioms: [propext] -/
#guard_msgs in #print axioms BFP.vp_coprime
/-- info: 'BFP.vp_split' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.vp_split
/-- info: 'BFP.vp_pow_coprime' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.vp_pow_coprime
/-- info: 'BFP.vp_additive' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.vp_additive
/-- info: 'BFP.euclid_at_three' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.euclid_at_three
/-- info: 'BFP.log_not_forced_off_tower' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.log_not_forced_off_tower
/-- info: 'BFP.wall' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.wall
/-- info: 'BFP.calibration' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.calibration
/-- info: 'BFP.formal_never_odd' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.formal_never_odd
/-- info: 'BFP.constant_no_crossing' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.constant_no_crossing
/-- info: 'BFP.no_reading_returns_the_deed' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.no_reading_returns_the_deed
/-- info: 'BFP.three_legs_one_orbit' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.three_legs_one_orbit
/-- info: 'BFP.prime_iff_one_free_orbit' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.prime_iff_one_free_orbit
/-- info: 'BFP.two_members' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.two_members
/-- info: 'BFP.three_members' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.three_members
/-- info: 'BFP.prime_trial_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.prime_trial_iff
/-- info: 'BFP.prime_iff_one_free_orbit_all' depends on axioms:
[propext, Quot.sound] -/
#guard_msgs in #print axioms BFP.prime_iff_one_free_orbit_all
/-- info: 'BFP.parity_frame_paid_bits' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.parity_frame_paid_bits
/-- info: 'BFP.divisor_cube_is_sign_cube' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.divisor_cube_is_sign_cube
/-- info: 'BFP.fibre_balance' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.fibre_balance
/-- info: 'BFP.filter_append_length' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.filter_append_length
/-- info: 'BFP.filter_map_cons_length' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.filter_map_cons_length
/-- info: 'BFP.mod_two_flips' depends on axioms: [propext,
Quot.sound] -/

```

```

#guard_msgs in #print axioms BFP.mod_two_flips
/-- info: 'BFP.every_fibre_over_more_than_one_balances' depends on
axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
BFP.every_fibre_over_more_than_one_balances
/-- info: 'BFP.wall_and_edge_exchanged' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.wall_and_edge_exchanged
/-- info: 'BFP.prime_mode_unitary_iff' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.prime_mode_unitary_iff
/-- info: 'BFP.pow_succ_ne_one' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.pow_succ_ne_one
/-- info: 'BFP.prime_mode_unitary_iff_int' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.prime_mode_unitary_iff_int
/-- info: 'BFP.price_per_bit_tower' depends on axioms: [propext] -/
#guard_msgs in #print axioms BFP.price_per_bit_tower
/-- info: 'BFP.len_le_sum' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.len_le_sum
/-- info: 'BFP.wall_block_share' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.wall_block_share
/-- info: 'BFP.conjugation_is_the_fold' depends on axioms: [propext]
-/
#guard_msgs in #print axioms BFP.conjugation_is_the_fold
/-- info: 'BFP.flow_inverse' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.flow_inverse
/-- info: 'BFP.flow_injective' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.flow_injective
/-- info: 'BFP.disc_flow' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.disc_flow
/-- info: 'BFP.pull_in_square' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.pull_in_square
/-- info: 'BFP.collision_sign' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.collision_sign
/-- info: 'BFP.zero_slack' depends on axioms: [propext, Quot.sound]
-/
#guard_msgs in #print axioms BFP.zero_slack
/-- info: 'BFP.certificate_merges' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.certificate_merges
/-- info: 'BFP.charge_carrier' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.charge_carrier
/-- info: 'BFP.value_from_the_act_inhabited' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.value_from_the_act_inhabited
/-- info: 'BFP.off_line_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.off_line_inhabited
/-- info: 'BFP.weakest_forcing_premise_inhabited' does not depend on
any axioms -/
#guard_msgs in #print axioms BFP.weakest_forcing_premise_inhabited
/-- info: 'BFP.keyless_forces_nothing_inhabited' does not depend on
any axioms -/
#guard_msgs in #print axioms BFP.keyless_forces_nothing_inhabited
/-- info: 'BFP.count_to_heat_inhabited' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.count_to_heat_inhabited
/-- info: 'BFP.bits_forced_inhabited' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.bits_forced_inhabited
/-- info: 'BFP.calibration_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.calibration_inhabited
/-- info: 'BFP.three_legs_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.three_legs_inhabited
/-- info: 'BFP.prime_mode_unitary_inhabited' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.prime_mode_unitary_inhabited
/-- info: 'BFP.zero_slack_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.zero_slack_inhabited
/-- info: 'BFP.record_carried_inhabited' does not depend on any
axioms -/
#guard_msgs in #print axioms BFP.record_carried_inhabited
/-- info: 'BFP.price_zero_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.price_zero_inhabited

```

```

/-- info: 'BFP.sign_group_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms BFP.sign_group_inhabited
/-- info: 'BFP.parity_law_inhabited' depends on axioms: [propext,
Quot.sound] -/
#guard_msgs in #print axioms BFP.parity_law_inhabited
/-- info: 'BFP.free_involution_inhabited' depends on axioms:
[propext] -/
#guard_msgs in #print axioms BFP.free_involution_inhabited
/-- info: 'BFP.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms BFP.the_bridge_atom
/-- info: 'BFP.the_bridge_from_first_principle' depends on axioms:
[propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms BFP.the_bridge_from_first_principle

#eval "BRIDGE FROM FIRST PRINCIPLE · the end of the file was reached"

```

The Final Cut kernel, PSP-FINAL-CUT-01. Final_Cut.lean

```

/-
THE FINAL CUT · PSP-FINAL-CUT-01 · core Lean 4.19.0, no import, no
axiom declared, no sorry.
The non-trivial zero is the formal name of a kinetic object, and
the classical statement never discloses the cut
between them. Across the cut, the formal class and the actual
class coincide under the Root Axiom read across the
cut and its converse inclusion. So the hypothesis
is least erasure of the actual zeros, and the prime side carries
the same bit. The unicorn is never registered, and the
one bit is spent by the act, at premise grade, beside the Root
Axiom. Every record, every computed zero and every
finite prime count leaves that bit where it found it.
Each sentence after the first is a theorem below; the first names
the cut those theorems cross. Four things enter by
name: the identification across the cut as the hypothesis `ra`
(every formal zero is an actual zero), its converse as
the hypothesis `hsub` (every actual zero is a formal one), the
explicit formula as the hypothesis `explicit` (the
prime side is equivalent to the value), and the act as the field
`supply`. Every cone is printed and pinned at the
foot of the file.
Sections: the chart, the fold, the registration; across the cut,
the formal class and the actual class; every
record, every computed zero (on the whole line and inside any
strip of offsets below N), every finite prime count
and every finite window of points leaves the bit, with the empty
world and the off-line pair world as the two
closed witnesses; the bit is keyed (no fixed truth value is the
value, the identification of the formal with the
actual class alone does not make the classes coincide and is
itself keyed, the root alone forces nothing);
inhabitation, a model for the hypotheses of every conditional law;
the Final Cut, whole.
-/
set_option autoImplicit false
namespace FinalCut

/--! ## The chart, the fold, the registration -/
abbrev Point := Int × Int
def fold (p : Point) : Point := (-p.1, p.2)
def onLine (p : Point) : Prop := p.1 = 0
def reg (p : Point) : Point := (0, p.2)

theorem neg_neg_free : ∀ d : Int, -d = d
| Int.ofNat 0 => rfl
| Int.ofNat ( _ + 1 ) => rfl
| Int.negSucc _ => rfl
theorem neg_self_zero : ∀ d : Int, -d = d → d = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat ( _ + 1 ), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h
theorem fold_involutive (p : Point) : fold (fold p) = p := by
obtain (d, t) := p
show (- -d, t) = (d, t)
rw [neg_neg_free d]
theorem the_cut_is_the_line (p : Point) : fold p = p ↔ onLine p := by
obtain (d, t) := p
exact (fun h => neg_self_zero d (congrArg Prod.fst h), fun h => by
cases h; rfl)
theorem reg_lands (p : Point) : onLine (reg p) := rfl
theorem reg_erases_nothing_iff (p : Point) : reg p = p ↔ onLine p :=
by

```

```

obtain (d, t) := p
exact (fun h => (congrArg Prod.fst h).symm, fun h => by cases h; rfl)

abbrev Config := Point → Prop
def Closed (S : Config) : Prop := ∀ p, S p → S (fold p)
def Value (S : Config) : Prop := ∀ p, S p → onLine p
def LeastErasure (S : Config) : Prop := ∀ p, S p → reg p = p

theorem least_erasure_iff_value (S : Config) : LeastErasure S ↔ Value S :=
  (fun h p hp => (reg_erases_nothing_iff p).mp (h p hp), fun h p hp => (reg_erases_nothing_iff p).mpr (h p hp))

/-! ## Across the cut: the formal class and the actual class -/

/-- ACROSS THE CUT THE CLASSES COINCIDE: if every formal zero is an actual zero (`ra`, the Root Axiom read at the zero set) and every actual zero is a formal one, the value over the formal class is the value over the actual class. -/
theorem classes_coincide_under_RA (Zall Zact : Config) (ra : ∀ s, Zall s → Zact s) (hsub : ∀ s, Zact s → Zall s) : Value Zall ↔ Value Zact :=
  (fun hv s hs => hv s (hsub s hs), fun hv s hs => hv s (ra s hs))

/-- THE HYPOTHESIS IS LEAST ERASURE OF THE ACTUAL ZEROS. -/
theorem hypothesis_is_least_erasure_of_the_actual (Zall Zact : Config) (ra : ∀ s, Zall s → Zact s) (hsub : ∀ s, Zact s → Zall s) : Value Zall ↔ LeastErasure Zact :=
  (classes_coincide_under_RA Zall Zact ra hsub).trans (least_erasure_iff_value Zact).symm

/-- THE PRIME SIDE CARRIES THE SAME BIT: whatever the prime side asserts, if the explicit formula makes it equivalent to the value (`explicit`), it is least erasure of the actual zeros. -/
theorem prime_side_carries_the_bit (Zall Zact : Config) (ra : ∀ s, Zall s → Zact s) (hsub : ∀ s, Zact s → Zall s) (W : Prop) (explicit : W ↔ Value Zall) : W ↔ LeastErasure Zact :=
  explicit.trans (hypothesis_is_least_erasure_of_the_actual Zall Zact ra hsub)

/-- THE UNICORN IS NEVER REGISTERED: no point off the line is the registration of anything. -/
theorem unicorn_never_registered (p : Point) (h : ¬ onLine p) (q : Point) : reg q ≠ p :=
  fun e => h (e ▶ reg_lands q)

/-- THE ACT: the bit supplied once, as a field of a type, on the actual class. -/
structure ActualZeros where
  zeros : Config
  closed : Closed zeros
  supply : LeastErasure zeros

/-- THE ONE BIT IS SPENT BY THE ACT: the act's least erasure on the actual class, carried across the cut by `ra`, is the value over the formal class. -/
theorem the_act_spends_the_bit (Z : ActualZeros) (Zall : Config) (ra : ∀ s, Zall s → Z.zeros s) : Value Zall :=
  fun s hs => (reg_erases_nothing_iff s).mp (Z.supply s (ra s hs))

/-! ## Every record, every computed zero, every finite prime count, every finite window leaves the bit -/
def Rec (S : Config) (q : Point) : Prop := ∃ p, S p ∧ reg p = q
def SameRecord (S S' : Config) : Prop := ∀ q, Rec S q ↔ Rec S' q
def registered (p : Point) : Config := fun s => s = reg p
def pairWorld (p : Point) : Config := fun s => s = p ∨ s = fold p

theorem one_record (p : Point) : SameRecord (registered p) (pairWorld p) := fun q =>
  (fun (_, hs, hq) => (p, Or.inl rfl, by rw [← hq, hs]; rfl), fun (_, hs, hq) => (reg p, rfl, by rw [← hq] exact match hs with | Or.inl e => by rw [e]; rfl | Or.inr e => by rw [e]; rfl))

/-- The empty world: no zero at all. -/
def emptyWorld : Config := fun _ => False

/-- THE EMPTY WORLD IS CLOSED AND HAS THE VALUE: it holds no point, so nothing leaves the line. -/

```

```

theorem empty_world_has_value : Closed emptyWorld ∧ Value emptyWorld :=
  (fun _ h => h, fun _ h => h.elim)

/-- THE PAIR WORLD IS CLOSED: the fold carries a point and its partner to each other. -/
theorem pair_world_closed (p : Point) : Closed (pairWorld p) := fun _ hs =>
  match hs with
  | Or.inl e => Or.inr (congrArg fold e)
  | Or.inr e => Or.inl (e ▶ fold_involutive p)

/-- THE PAIR WORLD AT OFFSET ONE LACKS THE VALUE: at every height its first point stands off the line. -/
theorem pair_world_lacks_value (t : Int) : ¬ Value (pairWorld (1, t)) :=
  fun hv => absurd (hv (1, t) (Or.inl rfl)) (fun e => by cases e)

/-- EVERY RECORD LEAVES THE BIT: no reading that respects the record returns the value on both worlds of one record. -/
theorem every_record_leaves_the_bit (p : Point) (h : ¬ onLine p) (g : Config → Prop) (hg : ∀ S S', SameRecord S S' → (g S ↔ g S')) : ¬ ((g (registered p) ↔ Value (registered p)) ∧ (g (pairWorld p) ↔ Value (pairWorld p))) :=
  fun (h1, h2) => h ((h2.mp ((hg _ _ (one_record p)).mp (h1.mpr (fun s hs => by rw [hs]; exact reg_lands p)))) (p (Or.inl rfl))

/-- EVERY COMPUTED ZERO LEAVES THE BIT: any finite certificate of zeros on the line extends to a closed world with the value and to a closed world without it. -/
theorem every_computed_zero_leaves_the_bit (L : List Point) (hL : ∀ p ∈ L, onLine p) : ∃ S1 S2 : Config, Closed S1 ∧ Closed S2 ∧ (∀ p ∈ L, S1 p ∧ S2 p) ∧ Value S1 ∧ ¬ Value S2 := by
  refine (fun p => p ∈ L, fun p => p ∈ L ∨ p = (1, 0) ∨ p = (-1, 0), ?_, ?_, ?_, ?_, ?_)
  · intro p hp
  show fold p ∈ L
  rw [(the_cut_is_the_line p).mpr (hL p hp)]; exact hp
  · intro p hp
  rcases hp with hp | e | e
  · exact Or.inl (by show fold p ∈ L; rw [(the_cut_is_the_line p).mpr (hL p hp)]; exact hp)
  · exact Or.inr (Or.inr (by rw [e]; rfl))
  · exact Or.inr (Or.inl (by rw [e]; rfl))
  · intro p hp; exact (hp, Or.inl hp)
  · exact hL
  · intro hv; exact absurd (hv (1, 0) (Or.inr (Or.inl rfl))) (fun e => by cases e)

/-- ONE LIES INSIDE EVERY STRIP OF HALF-WIDTH AT LEAST TWO: the offset one sits strictly between -N and N. -/
theorem one_lies_inside (N : Int) (hN : 2 ≤ N) : -N < 1 ∧ 1 < N :=
  match N, hN with
  | Int.ofNat (n + 2), h => (Int.NonNeg.mk _, h)
  | Int.ofNat 0, h => absurd h (by decide)
  | Int.ofNat 1, h => absurd h (by decide)
  | Int.negSucc _, h => nomatch h

/-- EVERY COMPUTED ZERO LEAVES THE BIT INSIDE THE STRIP: any finite certificate of zeros on the line extends to a closed world on the line and to a closed world with an off-line zero strictly inside the strip of offsets below N. -/
theorem every_computed_zero_leaves_the_bit_inside (N : Int) (hN : 2 ≤ N) (L : List Point) (hL : ∀ p ∈ L, onLine p) : ∃ S1 S2 : Config, Closed S1 ∧ Closed S2 ∧ (∀ p ∈ L, S1 p ∧ S2 p) ∧ (∀ p, S1 p → onLine p) ∧ (∃ p, S2 p ∧ ¬ onLine p ∧ -N < p.1 ∧ p.1 < N) := by
  refine (fun p => p ∈ L, fun p => p ∈ L ∨ p = (1, 0) ∨ p = (-1, 0), ?_, ?_, ?_, hL, ?_)
  · intro p hp
  show fold p ∈ L
  rw [(the_cut_is_the_line p).mpr (hL p hp)]; exact hp
  · intro p hp
  rcases hp with hp | e | e
  · exact Or.inl (by show fold p ∈ L; rw [(the_cut_is_the_line p).mpr (hL p hp)]; exact hp)
  · exact Or.inr (Or.inr (by rw [e]; rfl))
  · exact Or.inr (Or.inl (by rw [e]; rfl))
  · intro p hp; exact (hp, Or.inl hp)

```

```

· refine ((1, 0), Or.inr (Or.inl rfl), ?_, (one_lies_inside N
hN).1, (one_lies_inside N hN).2)
intro e; cases e

/-- EVERY FINITE PRIME COUNT LEAVES THE BIT: a prime-side reading
equivalent to the value on every world is decided by no
finite certificate of zeros on the line. -/
theorem every_finite_prime_count_leaves_the_bit (W : Config → Prop)
(explicit : ∀ S, W S ↔ Value S)
(L : List Point) (hL : ∀ p ∈ L, onLine p) :
  ∃ S₁ S₂ : Config, Closed S₁ ∧ Closed S₂ ∧ (∀ p ∈ L, S₁ p ∧ S₂ p) ∧
  W S₁ ∧ ¬ W S₂ :=
match every_computed_zero_leaves_the_bit L hL with
| (S₁, S₂, c₁, c₂, ext, v₁, v₂) => (S₁, S₂, c₁, c₂, ext, (explicit
S₁).mpr v₁, fun w => v₂ ((explicit S₂).mpr w))

/-- A height bound for a finite window: one more than the sum of the
absolute heights. -/
def heightBound : List Point → Nat
| [] => 0
| p :: r => Nat.succ (p.2.natAbs + heightBound r)

/-- EVERY HEIGHT IN THE WINDOW LIES BELOW THE BOUND: each point's
absolute height is less than the bound. -/
theorem height_below_bound (K : List Point) : ∀ p ∈ K, p.2.natAbs <
heightBound K := by
induction K with
| nil => intro _ h; exact nomatch h
| cons q r ih =>
  intro p h
  cases h with
  | head => exact Nat.lt_succ_of_le (Nat.le_add_right _ _)
  | tail _ h' => exact Nat.lt_trans (ih p h') (Nat.lt_succ_of_le
(Nat.le_add_left _ _))

/-- EVERY FINITE WINDOW LEAVES THE BIT: no reading that sees only a
finite list of points returns the value on every
closed world, since the empty world and the pair world above the
window agree on it and differ in the value. -/
theorem every_finite_window_leaves_the_bit (K : List Point) (g :
Config → Prop)
(hg : ∀ S S' : Config, (∀ p ∈ K, (S p ↔ S' p)) → (g S ↔ g S')) :
  ¬ ∀ S : Config, Closed S → (g S ↔ Value S) :=
fun h =>
  have far : ∀ p ∈ K, ¬ pairWorld (1, Int.ofNat (heightBound K)) p
:= fun p hp hs =>
  have e : p.2 = Int.ofNat (heightBound K) := match hs with
  | Or.inl e => congrArg Prod.snd e
  | Or.inr e => congrArg Prod.snd e
  have h1 : (Int.ofNat (heightBound K)).natAbs < heightBound K
:= e ▸ height_below_bound K p hp
  Nat.lt_irrefl _ h1
  have agree : ∀ p ∈ K, (emptyWorld p ↔ pairWorld (1, Int.ofNat
(heightBound K)) p) :=
  fun p hp => (False.elim, fun hs => absurd hs (far p hp))
  pair_world_lacks_value _ ((h _ (pair_world_closed _)).mpr
((hg _ agree).mpr ((h emptyWorld empty_world_has_value.1).mpr
empty_world_has_value.2)))

/-! ## The bit is keyed: no fixed truth value, the identification
alone, the root alone -/

/-- NO FIXED TRUTH VALUE IS THE VALUE: whatever proposition is
proposed, the empty world and the off-line pair world are
both closed and disagree on the value, so it is not equivalent
to the value on both. -/
theorem no_fixed_count_is_the_value (c : Prop) : ¬ ∀ S : Config,
Closed S → (c ↔ Value S) :=
fun h => pair_world_lacks_value 0
((h _ (pair_world_closed (1, 0))).mpr ((h emptyWorld
empty_world_has_value.1).mpr empty_world_has_value.2))

/-- THE IDENTIFICATION ALONE DOES NOT MAKE THE CLASSES COINCIDE:
every formal zero may be an actual zero while the
formal class has the value and the actual class lacks it, so the
converse inclusion is load-bearing. -/
theorem ra_alone_does_not_make_the_classes_coincide :
  ∃ Zall Zact : Config, (∀ s, Zall s → Zact s) ∧ Value Zall ∧ ¬
Value Zact :=
(emptyWorld, pairWorld (1, 0), fun _ h => False.elim h,
empty_world_has_value.2, pair_world_lacks_value 0)

/-- THE IDENTIFICATION IS KEYED: the inclusion of the formal class
in the actual class fails for some pair of

```

```

configurations, so it holds by supply and never by logic. -/
theorem ra_is_keyed : ¬ ∀ Zall Zact : Config, (∀ s, Zall s → Zact s)
:=
fun h => h (fun _ => True) emptyWorld (0, 0) trivial

/-- THE ROOT DECIDES NOTHING: a premise that holds, whatever it is,
never forces the value on every closed world. -/
theorem root_decides_nothing (R : Prop) (r : R) : ¬ ∀ S : Config,
Closed S → R → Value S :=
fun h => pair_world_lacks_value 0 (h _ (pair_world_closed (1, 0))
r)

/-! ## Inhabitation: every conditional law the harness lists has a
model of its hypotheses -/
theorem cut_hypotheses_inhabited : ∃ Zall Zact : Config, (∀ s, Zall
s → Zact s) ∧ (∀ s, Zact s → Zall s) :=
(onLine, onLine, fun _ h => h, fun _ h => h)
theorem explicit_inhabited : ∃ W : Config → Prop, ∀ S, W S ↔ Value S
:= (Value, fun _ => Iff.rfl)
theorem act_inhabited : Nonempty ActualZeros :=
((onLine, fun p h => by show -p.1 = 0; rw [show p.1 = 0 from h];
rfl, fun p h => (reg_erases_nothing_iff p).mpr h))
theorem certificate_inhabited : ∀ p ∈ [(0, 14)] : List Point,
onLine p := by
intro p hp
cases hp with
| head => rfl
| tail _ h => cases h
theorem unicorn_inhabited : ¬ onLine (1, 0) := fun e => by cases e
theorem strip_inhabited : (2 : Int) ≤ 2 := by decide
theorem window_inhabited :
  ∃ g : Config → Prop, ∀ S S' : Config, (∀ p ∈ [(0, 14)] : List
Point), (S p ↔ S' p) → (g S ↔ g S') :=
(fun _ => True, fun _ _ => Iff.rfl)
theorem root_inhabited : ∃ R : Prop, R := (True, trivial)

/-! ## The Final Cut, whole -/
theorem the_final_cut :
  (∀ Zall Zact : Config, (∀ s, Zall s → Zact s) → (∀ s, Zact s →
Zall s) → (Value Zall ↔ Value Zact)) ∧
  (∀ Zall Zact : Config, (∀ s, Zall s → Zact s) → (∀ s, Zact s →
Zall s) → (Value Zall ↔ LeastErasure Zact)) ∧
  (∀ (Zall Zact : Config) (W : Prop), (∀ s, Zall s → Zact s) → (∀
s, Zact s → Zall s) →
  (W ↔ Value Zall) → (W ↔ LeastErasure Zact)) ∧
  (∀ p : Point, ¬ onLine p → ∀ q, reg q ≠ p) ∧
  (∀ (Z : ActualZeros) (Zall : Config), (∀ s, Zall s → Z.zeros s)
→ Value Zall) ∧
  (∀ (p : Point), ¬ onLine p → ∀ g : Config → Prop, (∀ S S',
SameRecord S S' → (g S ↔ g S')) →
  ¬ ((g (registered p) → Value (registered p)) ∧ (g (pairWorld
p) → Value (pairWorld p)))) ∧
  (∀ L : List Point, (∀ p ∈ L, onLine p) →
  ∃ S₁ S₂ : Config, Closed S₁ ∧ Closed S₂ ∧ (∀ p ∈ L, S₁ p ∧ S₂ p)
  ∧ Value S₁ ∧ ¬ Value S₂) ∧
  (∀ W : Config → Prop, (∀ S, W S ↔ Value S) → ∀ L : List Point, (∀
p ∈ L, onLine p) →
  ∃ S₁ S₂ : Config, Closed S₁ ∧ Closed S₂ ∧ (∀ p ∈ L, S₁ p ∧ S₂ p)
  ∧ W S₁ ∧ ¬ W S₂) :=
(classes_coincide_under_RA,
hypothesis_is_least_erasure_of_the_actual,
fun Zall Zact W ra hsub ex => prime_side_carries_the_bit Zall
Zact ra hsub W ex,
unicorn_never_registered, the_act_spends_the_bit,
every_record_leaves_the_bit,
every_computed_zero_leaves_the_bit,
every_finite_prime_count_leaves_the_bit)

end FinalCut

/-! ## The cones, pinned. Each line is the compiler's own; a drifted
cone fails the compile. -/
/-! info: 'FinalCut.neg_neg_free' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.neg_neg_free
/-! info: 'FinalCut.neg_self_zero' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.neg_self_zero
/-! info: 'FinalCut.fold_involutive' does not depend on any axioms
-/
#guard_msgs in #print axioms FinalCut.fold_involutive
/-! info: 'FinalCut.the_cut_is_the_line' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.the_cut_is_the_line
/-! info: 'FinalCut.reg_lands' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.reg_lands

```

```

/-- info: 'FinalCut.reg_erases_nothing_iff' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.reg_erases_nothing_iff
/-- info: 'FinalCut.least_erasure_iff_value' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.least_erasure_iff_value
/-- info: 'FinalCut.classes_coincide_under_RA' does not depend on
any axioms -/
#guard_msgs in #print axioms FinalCut.classes_coincide_under_RA
/-- info: 'FinalCut.hypothesis_is_least_erasure_of_the_actual' does
not depend on any axioms -/
#guard_msgs in #print axioms
FinalCut.hypothesis_is_least_erasure_of_the_actual
/-- info: 'FinalCut.prime_side_carries_the_bit' does not depend on
any axioms -/
#guard_msgs in #print axioms FinalCut.prime_side_carries_the_bit
/-- info: 'FinalCut.unicorn_never_registered' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.unicorn_never_registered
/-- info: 'FinalCut.the_act_spends_the_bit' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.the_act_spends_the_bit
/-- info: 'FinalCut.one_record' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.one_record
/-- info: 'FinalCut.empty_world_has_value' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.empty_world_has_value
/-- info: 'FinalCut.pair_world_closed' does not depend on any axioms
-/
#guard_msgs in #print axioms FinalCut.pair_world_closed
/-- info: 'FinalCut.pair_world_lacks_value' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.pair_world_lacks_value
/-- info: 'FinalCut.every_record_leaves_the_bit' does not depend on
any axioms -/
#guard_msgs in #print axioms FinalCut.every_record_leaves_the_bit
/-- info: 'FinalCut.every_computed_zero_leaves_the_bit' does not
depend on any axioms -/
#guard_msgs in #print axioms
FinalCut.every_computed_zero_leaves_the_bit
/-- info: 'FinalCut.one_lies_inside' does not depend on any axioms
-/
#guard_msgs in #print axioms FinalCut.one_lies_inside
/-- info: 'FinalCut.every_computed_zero_leaves_the_bit_inside' does
not depend on any axioms -/
#guard_msgs in #print axioms
FinalCut.every_computed_zero_leaves_the_bit_inside
/-- info: 'FinalCut.every_finite_prime_count_leaves_the_bit' does
not depend on any axioms -/
#guard_msgs in #print axioms
FinalCut.every_finite_prime_count_leaves_the_bit
/-- info: 'FinalCut.height_below_bound' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.height_below_bound
/-- info: 'FinalCut.every_finite_window_leaves_the_bit' does not
depend on any axioms -/
#guard_msgs in #print axioms
FinalCut.every_finite_window_leaves_the_bit
/-- info: 'FinalCut.no_fixed_count_is_the_value' does not depend on
any axioms -/
#guard_msgs in #print axioms FinalCut.no_fixed_count_is_the_value
/-- info: 'FinalCut.ra_alone_does_not_make_the_classes_coincide'
does not depend on any axioms -/
#guard_msgs in #print axioms
FinalCut.ra_alone_does_not_make_the_classes_coincide
/-- info: 'FinalCut.ra_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.ra_is_keyed
/-- info: 'FinalCut.root_decides_nothing' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.root_decides_nothing
/-- info: 'FinalCut.cut_hypotheses_inhabited' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.cut_hypotheses_inhabited
/-- info: 'FinalCut.explicit_inhabited' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.explicit_inhabited
/-- info: 'FinalCut.act_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.act_inhabited
/-- info: 'FinalCut.certificate_inhabited' does not depend on any
axioms -/
#guard_msgs in #print axioms FinalCut.certificate_inhabited
/-- info: 'FinalCut.unicorn_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms FinalCut.unicorn_inhabited

```

```

/-- info: 'FinalCut.strip_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms FinalCut.strip_inhabited
/-- info: 'FinalCut.window_inhabited' does not depend on any axioms
-/
#guard_msgs in #print axioms FinalCut.window_inhabited
/-- info: 'FinalCut.root_inhabited' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.root_inhabited
/-- info: 'FinalCut.the_final_cut' does not depend on any axioms -/
#guard_msgs in #print axioms FinalCut.the_final_cut

#eval "FINAL CUT · the end of the file was reached"

```

The twin. Bridge_Twin.f90

```

! Bridge_Twin.f90 . the executed twin of
Bridge_From_First_Principle.lean.
! Every check is a finite computation that ends. No named human
result is a premise.
! Build (sealed): gfortran -std=f2018 -O2 -fno-fast-math
-ffp-contract=off -Wall -Wextra
program bridge_twin
use, intrinsic :: iso_fortran_env, only: dp => real64, i8 => int64
implicit none
integer :: nchk = 0, nfail = 0
call conduct()
call atom()
call three_twelve_eight()
call pay()
call supply()
call primes()
call wall_edge_seat()
call heat_flow()
write(*,'(a,i0,a,i0,a)') 'BRIDGE-TWIN-JSON: {"checks":', nchk,
', "failures":', nfail, '}'
if (nfail /= 0) error stop 1
contains
subroutine check(name, ok)
character(*), intent(in) :: name
logical, intent(in) :: ok
nchk = nchk + 1
if (.not. ok) nfail = nfail + 1
write(*,'(a,a)') merge(' PASS ', ' FAIL ', ok), name
end subroutine check

! 0 . conduct: the arithmetic this twin runs on is the arithmetic
it claims
subroutine conduct()
real(dp), volatile :: one, x, y, r, z
one = 1.0_dp
x = one + scale(one, -30); y = one - scale(one, -30)
r = x*y - one
z = ieee_nan()
write(*,'(a)') '0 CONDUCT'
call check('0: no fused multiply-add executed (a fused path
reads -2^60)', abs(r) <= 0.0_dp)
call check('0: NaN is ordered against nothing, itself included',
.not. (z <= z.or. z > z))
end subroutine conduct
function ieee_nan() result(z)
use, intrinsic :: ieee_arithmetic, only: ieee_value,
ieee_quiet_nan
real(dp) :: z
z = ieee_value(1.0_dp, ieee_quiet_nan)
end function ieee_nan

! I . the atom on a window: fold, line, registration, the lost
bit, least erasure equal to the value
pure function foldp(p) result(q)
integer, intent(in) :: p(2)
integer :: q(2)
q = [-p(1), p(2)]
end function foldp
pure function regp(p) result(q)
integer, intent(in) :: p(2)
integer :: q(2)
q = [0, p(2)]
end function regp
subroutine atom()
integer :: d, t, k, m, g1, g2, g3, h, nboth, none_, ma, mb
integer :: g(0:2), pd(9), pt(9), p(2), ra(2), rb(2)
logical :: ok1, ok2, ok3, le, va
write(*,'(a)') 'I THE ATOM'
ok1 = .true.; ok2 = .true.; ok3 = .true.

```

```

do d = -50, 50
  do t = -5, 5
    p = [d, t]
    if (any(foldp(foldp(p)) /= p)) ok1 = .false.
    if (all(foldp(p) == p) .neqv. (d == 0)) ok1 = .false.
    if (all(regp(p) == p) .neqv. (d == 0)) ok2 = .false.
    if (any(regp(p) /= [0, t])) ok2 = .false.
    if (d /= 0) then
      if (all(foldp(p) == p)) ok3 = .false.
      if (any(regp(foldp(p)) /= regp(p))) ok3 = .false.
    end if
  end do
end do
call check('I: the fold undoes itself and fixes exactly the
line, 1111 points', ok1)
call check('I: the registration lands on the line, keeps the
height, erases nothing exactly on it', ok2)
call check('I: every off-line point has a distinct partner of
one record, 1100 points', ok3)
! window: d in {-1,0,1}, t in {0,1,2}; the records are the three
heights; 9^3 = 729 maps from records to points
k = 0
do t = 0, 2
  do d = -1, 1
    k = k + 1; pd(k) = d; pt(k) = t
  end do
end do
nboth = 0; none_ = 0
do g1 = 1, 9
  do g2 = 1, 9
    do g3 = 1, 9
      g = [g1, g2, g3]
      do h = 0, 2
        ! the two worlds at height h are {(1,h)} and {(-1,h)};
each map reads its world's record, computed by
! the registration, and returns a point; it returns the
side when it gives (1,h) on the first world and
! (-1,h) on the second
ra = regp([1, h]); rb = regp([-1, h])
ma = g(ra(2)); mb = g(rb(2))
if (pt(ma) == h .and. abs(pd(ma)) == 1) none_ = none_ + 1
if (pt(ma) == h .and. pd(ma) == 1 .and. pt(mb) == h .and.
pd(mb) == -1) nboth = nboth + 1
      end do
    end do
  end do
end do
write(*, '(a,i0,a,i0)') ' 729 maps from computed records to
points: returning the side ', nboth, &
'; returning one member ', none_
call check('I: no map from the computed records returns the side
of a pair', nboth == 0 .and. none_ == 486)
! least erasure, read through the registration, against the
value, read through the offset
ok1 = .true.
do m = 0, 4095
  le = .true.; va = .true.
  do k = 0, 11
    if (btest(m, k)) then
      p = [mod(k, 3) - 1, k / 3]
      if (any(regp(p) /= p)) le = .false.
      if (p(1) /= 0) va = .false.
    end if
  end do
  if (le .neqv. va) ok1 = .false.
end do
call check('I: least erasure equals the value on all 4096
subsets of twelve points', ok1)
end subroutine atom

! II . three, twelve, eight, the ninth
subroutine three_twelve_eight()
  integer :: m, t, v, cnt, nlock, a, b, c, i, j, n, nperm, ninv,
s, ok_cnt
  integer :: r(3,3), perm(4), even_perms(4,12)
  logical :: ok, used(4)
  write(*, '(a)') 'II THREE, TWELVE, EIGHT, THE NINTH'
  ok = .true.
  do m = 0, 63
    do t = 0, 3
      cnt = 0
      do v = 0, 7
        if (mod(rowdot(m/8, v), 2) == t/2 .and.
mod(rowdot(mod(m,8), v), 2) == mod(t,2)) cnt = cnt + 1

```

```

      end do
    end do
    if (cnt == 1) ok = .false.
  end do
end do
call check('II: no two-axis system over GF(2) has exactly one
solution, 256 systems', ok)
nlock = 0; ok = .true.
do m = 0, 511
  do i = 1, 3
    r(i,1) = mod(ishft(m, -(3*(3-i)+2)), 2); r(i,2) =
mod(ishft(m, -(3*(3-i)+1)), 2)
    r(i,3) = mod(ishft(m, -(3*(3-i))), 2)
  end do
  if (det2(r) == 1) then
    nlock = nlock + 1
    do t = 0, 7
      cnt = 0
      do v = 0, 7
        if
(mod(r(1,1)*bit(v,2)+r(1,2)*bit(v,1)+r(1,3)*bit(v,0),2) ==
bit(t,2) .and. &
mod(r(2,1)*bit(v,2)+r(2,2)*bit(v,1)+r(2,3)*bit(v,0),2) ==
bit(t,1) .and. &
mod(r(3,1)*bit(v,2)+r(3,2)*bit(v,1)+r(3,3)*bit(v,0),2) ==
bit(t,0)) cnt = cnt + 1
        end do
        if (cnt /= 1) ok = .false.
      end do
    end if
  end do
  call check('II: 168 three-axis systems have determinant one and
each locks all eight targets', nlock == 168 .and. ok)
! twelve: even permutations of four, acting on the twelve
directed gates
nperm = 0
do a = 1, 4
  do b = 1, 4
    do c = 1, 4
      do s = 1, 4
        perm = [a, b, c, s]
        used = .false.; ok = .true.
        do i = 1, 4
          if (used(perm(i))) ok = .false.
          used(perm(i)) = .true.
        end do
        if (.not. ok) cycle
        ninv = 0
        do i = 1, 3
          do j = i+1, 4
            if (perm(i) > perm(j)) ninv = ninv + 1
          end do
        end do
        if (mod(ninv, 2) == 0) then
          nperm = nperm + 1; even_perms(:, nperm) = perm
        end if
      end do
    end do
  end do
end do
ok = (nperm == 12)
do a = 1, 4
  do b = 1, 4
    if (a == b) cycle
    do c = 1, 4
      do s = 1, 4
        if (c == s) cycle
        ok_cnt = 0
        do n = 1, nperm
          if (even_perms(a, n) == c .and. even_perms(b, n) == s)
ok_cnt = ok_cnt + 1
          end do
          if (ok_cnt /= 1) ok = .false.
        end do
      end do
    end do
  end do
end do
call check('II: twelve rotations, and exactly one carries any
directed gate to any directed gate', ok)
! eight: sign patterns, total negation free, orientation flipped
ok = .true.
do m = 0, 7
  if (ieor(m, 7) == m) ok = .false.

```

```

    if (mod(popcnt(m) + popcnt(ieor(m, 7)), 2) /= 1) ok = .false.
end do
call check('II: eight patterns, total negation moves each and
joins an even to an odd', ok)
! the ninth: 2^k is never 9, and every odd fibre holds exactly
one seat
ok = .true.
do i = 0, 62
    if (2_i8**i == 9_i8) ok = .false.
end do
call check('II: 2^k is never nine, k = 0..62', ok)
end subroutine three_twelve_eight
integer function rowdot(rr, v)
    integer, intent(in) :: rr, v
    rowdot = bit(rr,2)*bit(v,2) + bit(rr,1)*bit(v,1) +
    bit(rr,0)*bit(v,0)
end function rowdot
integer function bit(x, k)
    integer, intent(in) :: x, k
    bit = mod(ishft(x, -k), 2)
end function bit
integer function det2(r)
    integer, intent(in) :: r(3,3)
    det2 = mod(r(1,1)*(r(2,2)*r(3,3)+r(2,3)*r(3,2)) +
    r(1,2)*(r(2,1)*r(3,3)+r(2,3)*r(3,1)) + &
    r(1,3)*(r(2,1)*r(3,2)+r(2,2)*r(3,1)), 2)
end function det2

! III . pay: pigeonhole and the export, exhaustively on small
wholes
logical function injective_exists(n, m)
    integer, intent(in) :: n, m
    integer :: f(8), i, j
    logical :: inj
    integer(i8) :: code, total, cc
    injective_exists = .false.
    total = int(m, i8)**n
    do code = 0_i8, total - 1_i8
        cc = code
        do i = 1, n
            f(i) = int(mod(cc, int(m, i8))); cc = cc / int(m, i8)
        end do
        inj = .true.
        do i = 1, n - 1
            do j = i + 1, n
                if (f(i) == f(j)) inj = .false.
            end do
        end do
        if (inj) then
            injective_exists = .true.; return
        end if
    end do
end function injective_exists
subroutine pay()
    integer :: n, m, cp, f, fp, k
    integer, parameter :: cps(5) = [1, 1, 1, 2, 3], fs(5) = [1, 2,
    3, 1, 1], fpmx(5) = [10, 8, 8, 8, 3]
    logical :: ok
    integer(i8) :: a, b
    write(*, '(a)') 'III PAY'
    ok = .true.
    do n = 1, 6
        do m = 1, 6
            if (injective_exists(n, m) .neqv. (n <= m)) ok = .false.
        end do
    end do
    call check('III: n distinct states fit injectively in m cells
    exactly when n <= m, every n, m <= 6', ok)
    ok = .true.
    do k = 1, 5
        cp = cps(k); f = fs(k)
        do fp = 1, fpmx(k)
            n = 2*cp*f; m = cp*fp
            if (injective_exists(n, m) .neqv. (2*f <= fp)) ok = .false.
        end do
    end do
    call check('III: the content halves and nothing merges exactly
    when the freedom at least doubles', ok)
    call check('III: a merging step halves the content with no
    freedom exported (two states, one cell)', &
    .not. injective_exists(2, 1))
    ok = .true.
    do a = 1_i8, 300_i8
        do b = 1_i8, 300_i8

```

```

            if (v2(a*b) /= v2(a) + v2(b)) ok = .false.
        end do
    end do
    call check('III: the two-adic valuation is additive on positive
    arguments, 90000 pairs', ok .and. v2(2_i8) == 1)
end subroutine pay
integer function v2(x)
    integer(i8), intent(in) :: x
    integer(i8) :: y
    v2 = 0; y = x
    do while (mod(y, 2_i8) == 0_i8)
        v2 = v2 + 1; y = y / 2_i8
    end do
end function v2

! IV . supply: the calibration is unique, and no reading of the
content returns the deed
subroutine supply()
    integer :: sx, dx, c, ncal, gcode, q, nret, e1(2), e0(2)
    logical :: ok, all_match
    write(*, '(a)') 'IV SUPPLY'
    ok = .true.
    do sx = 0, 1
        do dx = 0, 1
            ncal = 0
            do c = 0, 1
                ! on the orbit {x, sigma x}: d(x) = s(x) xor c and d(sigma
                x) = s(sigma x) xor c, with s, d odd
                if (dx == ieor(sx, c) .and. 1 - dx == ieor(1 - sx, c)) ncal
                = ncal + 1
            end do
            if (ncal /= 1) ok = .false.
        end do
    end do
    call check('IV: for every odd witness and odd target, exactly
    one calibration bit', ok)
    nret = 0
    do gcode = 0, 7
        all_match = .true.
        do q = 0, 2
            ! a reading of the content, computed by the registration
            from the executed state (1, q) and the unexecuted
            ! state (0, q), returns whether it ran when it gives 1 on
            the first and 0 on the second
            e1 = regp([1, q]); e0 = regp([0, q])
            if (bit(gcode, e1(2)) /= 1) all_match = .false.
            if (bit(gcode, e0(2)) /= 0) all_match = .false.
        end do
        if (all_match) nret = nret + 1
    end do
    call check('IV: of the 8 readings of a three-element content,
    none returns whether it ran', nret == 0)
end subroutine supply

! V . primes: one free orbit, the parity frame, the divisor cube,
the balance
subroutine primes()
    integer, parameter :: N = 1000000
    integer, allocatable :: dc(:), acc(:), mu(:), seats(:)
    logical, allocatable :: comp(:)
    integer :: d, m, r, k, nsq
    logical :: ok_odd, ok_bal, ok_prime, ok_k4, ok_seat
    integer, parameter :: kp(6) = [11, 13, 17, 19, 23, 29], ks(6) =
    [851, 1273, 437, 2119, 1703, 869]
    write(*, '(a)') 'V PRIMES'
    allocate(dc(N), acc(N), mu(N), comp(N), seats(N))
    dc = 0; acc = 0; mu = 1; comp = .false.; seats = 0
    do d = 2, N
        if (.not. comp(d)) then
            do m = 2*d, N, d
                comp(m) = .true.
            end do
            do m = d, N, d
                mu(m) = -mu(m)
            end do
            if (int(d,i8)*int(d,i8) <= int(N,i8)) then
                do m = d*d, N, d*d
                    mu(m) = 0
                end do
            end if
        end if
    end do
    do d = 1, N
        do m = d, N, d

```

```

    dc(m) = dc(m) + 1
    acc(m) = acc(m) + mu(d)
    if (int(d,i8)*int(d,i8) == int(m,i8)) seats(m) = seats(m) + 1
end do
ok_odd = .true.; ok_bal = .true.; ok_prime = .true.; ok_seat = .true.
do m = 1, N
    r = int(sqrt(real(m, dp)))
    do while (r*r > m); r = r - 1; end do
    do while ((r+1)*(r+1) <= m); r = r + 1; end do
    nsq = merge(1, 0, r*r == m)
    if (seats(m) /= nsq .or. seats(m) > 1) ok_odd = .false.
    if (mod(dc(m), 2) /= seats(m)) ok_odd = .false.
    if (m == 1 .and. acc(m) /= 1) ok_bal = .false.
    if (m > 1 .and. acc(m) /= 0) ok_bal = .false.
    if (m > 1 .and. ((dc(m) == 2) .neqv. (.not. comp(m))))
ok_prime = .false.
    if (m > 1 .and. .not. comp(m) .and. seats(m) /= 0) ok_seat = .false.
end do
call check('V: a fibre holds at most one seat, counted, and is odd exactly when it holds one, n <= 10^6', ok_odd)
call check('V: a prime is exactly a fibre of one free orbit, with no seat, n <= 10^6', ok_prime .and. ok_seat)
call check('V: every fibre over n > 1 balances and the fibre over 1 carries +1, n <= 10^6', ok_bal)
ok_k4 = .true.
do k = 1, 6
    if (mod(kp(k), 420) /= mod(ks(k), 420)) ok_k4 = .false.
    if (dc(kp(k)) /= 2 .or. dc(ks(k)) /= 4) ok_k4 = .false.
    if (mu(kp(k)) /= -1 .or. mu(ks(k)) /= 1) ok_k4 = .false.
end do
call check('V: the parity frame: residues mod 420 merge each pair, the paid bits 1 and 2 separate it', ok_k4)
call check('V: thirty has the eight divisors of the sign cube, signs four and four', &
    dc(30) == 8 .and. sum(mu([1,2,3,5,6,10,15,30])) == 0)
deallocate(dc, acc, mu, comp)
end subroutine primes

! VI . wall, edge, seat
pure function theta(x) result(s)
    real(dp), intent(in) :: x
    real(dp) :: s
    integer :: n
    s = 0.0_dp
    do n = 80, 1, -1
        s = s + exp(-acos(-1.0_dp) * real(n,dp)**2 * x)
    end do
    s = 1.0_dp + 2.0_dp * s
end function theta
subroutine wall_edge_seat()
    integer, parameter :: NW = 10000000
    real(dp), parameter :: xs(7) = [0.25_dp, 0.5_dp, 0.8_dp, 1.0_dp, 1.5_dp, 2.0_dp, 4.0_dp]
    real(dp) :: worst, rr, s1, s2, h, best, ratio
    integer(i8) :: n, lo, hi, mm
    integer :: k, j, d, m, xbest
    integer(1), allocatable :: mu(:)
    logical, allocatable :: comp(:)
    logical :: ok1, ok2, ok3
    write(*,'(a)') 'VI WALL, EDGE, SEAT'
    worst = 0.0_dp
    do k = 1, 7
        rr = abs(theta(1.0_dp/xs(k)) - sqrt(xs(k))*theta(xs(k))) /
            (sqrt(xs(k))*theta(xs(k)))
        worst = max(worst, rr)
    end do
    write(*,'(a,es10.3)') ' theta(1/x) against sqrt(x) theta(x), worst relative gap over seven scales: ', worst
    call check('VI: the lattice sum is self-dual; its weight is the square root, the fold's centre', worst < 1.0e-13_dp)
    ok1 = .true.; ok2 = .true.; ok3 = .true.; h = 1.0_dp
    do j = 0, 24
        lo = 2_i8*j + 1_i8; hi = 2_i8**(j+1)
        s1 = 0.0_dp; s2 = 0.0_dp
        do n = hi, lo, -1
            s1 = s1 + 1.0_dp/real(n,dp)
            s2 = s2 + 1.0_dp/real(n,dp)**2
        end do
        h = h + s1
        if (s1 < 0.5_dp .or. s1 > 1.0_dp) ok1 = .false.
        if (s2 > 2.0_dp*(-j)) ok2 = .false.
    end do

```

```

        if (h < 1.0_dp + 0.5_dp*real(j+1,dp)) ok3 = .false.
    end do
    call check('VI: every block of shares one-in-n holds between a half and one: the wall at Re s = 1', ok1 .and. ok3)
    call check('VI: at two units per bit every block holds at most 2^j: finite past the wall', ok2)
    allocate(mu(NW), comp(NW))
    mu = 1_1; comp = .false.
    do d = 2, NW
        if (.not. comp(d)) then
            do m = 2*d, NW, d
                comp(m) = .true.
            end do
            do m = d, NW, d
                mu(m) = -mu(m)
            end do
            if (int(d,i8)*int(d,i8) <= int(NW,i8)) then
                do m = d*d, NW, d*d
                    mu(m) = 0_1
                end do
            end if
        end if
    end do
    mm = 0_i8; best = 0.0_dp; xbest = 0
    do m = 1, NW
        mm = mm + int(mu(m), i8)
        if (m >= 100) then
            ratio = abs(real(mm,dp)) / sqrt(real(m,dp))
            if (ratio > best) then
                best = ratio; xbest = m
            end if
        end if
    end do
    write(*,'(a,i0,a,f8.5,a,i0)') ' the signed walk of the paid parities to 10^7: M = ', mm, &
        '; max |M(x)|/sqrt(x) on [100, 10^7] = ', best, ' at x = ', xbest
    call check('VI: the walk is paid to 10^7 and stays inside the square root there, a finite payment', best < 1.0_dp)
    deallocate(mu, comp)
end subroutine wall_edge_seat

! VII . the heat flow: every zero moves by the pull law; the pair falls onto the line
subroutine flow_coeffs(c0, t, c)
    real(dp), intent(in) :: c0(0:6), t
    real(dp), intent(out) :: c(0:6)
    real(dp) :: dk(0:6), tmp(0:6), fac
    integer :: k, i
    c = c0; dk = c0; fac = 1.0_dp
    do k = 1, 3
        tmp = 0.0_dp
        do i = 2, 6
            tmp(i-2) = dk(i) * real(i,dp) * real(i-1,dp)
        end do
        dk = tmp
        fac = fac * (-t) / real(k,dp)
        c = c + fac * dk
    end do
end subroutine flow_coeffs
pure function peval(c, z) result(p)
    real(dp), intent(in) :: c(0:6)
    complex(dp), intent(in) :: z
    complex(dp) :: p
    integer :: i
    p = cmplx(c(6), 0.0_dp, dp)
    do i = 5, 0, -1
        p = p*z + c(i)
    end do
end function peval
pure function dpeval(c, z) result(p)
    real(dp), intent(in) :: c(0:6)
    complex(dp), intent(in) :: z
    complex(dp) :: p
    integer :: i
    p = cmplx(6.0_dp*c(6), 0.0_dp, dp)
    do i = 5, 1, -1
        p = p*z + real(i,dp)*c(i)
    end do
end function dpeval
subroutine newton_all(c, z)
    real(dp), intent(in) :: c(0:6)
    complex(dp), intent(inout) :: z(6)
    integer :: k, it

```

```

do k = 1, 6
  do it = 1, 40
    z(k) = z(k) - peval(c, z(k)) / dpeval(c, z(k))
  end do
end do
end subroutine newton_all
integer function sign_changes(c)
  real(dp), intent(in) :: c(0:6)
  integer :: i
  real(dp) :: x, v, vprev
  sign_changes = 0
  vprev = real(peval(c, cmplx(-6.0_dp, 0.0_dp, dp)), dp)
  do i = 1, 240000
    x = -6.0_dp + 12.0_dp*real(i,dp)/240000.0_dp
    v = real(peval(c, cmplx(x, 0.0_dp, dp)), dp)
    if (v*vprev < 0.0_dp) sign_changes = sign_changes + 1
    if (abs(v) > 0.0_dp) vprev = v
  end do
end function sign_changes
subroutine heat_flow()
  complex(dp) :: r0(6), zp(6), zm(6), z(6), poly(0:6), vf, vfd
  real(dp) :: c0(0:6), c(0:6), hh, worst, t, imprev
  integer :: k, j, i, nreal0, nreal1, step
  logical :: mono
  write(*,'(a)') 'VII THE HEAT FLOW'
  r0 = [cmplx(0.3_dp, 0.5_dp, dp), cmplx(0.3_dp, -0.5_dp, dp),
        cmplx(-2.0_dp, 0.0_dp, dp), &
        cmplx(-1.0_dp, 0.0_dp, dp), cmplx(1.5_dp, 0.0_dp, dp),
        cmplx(2.5_dp, 0.0_dp, dp)]
  poly = (0.0_dp, 0.0_dp); poly(0) = (1.0_dp, 0.0_dp)
  do k = 1, 6
    do i = k, 1, -1
      poly(i) = poly(i-1) - r0(k)*poly(i)
    end do
    poly(0) = -r0(k)*poly(0)
  end do
  c0 = real(poly, dp)
  hh = 1.0e-5_dp; worst = 0.0_dp
  call flow_coeffs(c0, hh, c); zp = r0; call newton_all(c, zp)
  call flow_coeffs(c0, -hh, c); zm = r0; call newton_all(c, zm)
  do k = 1, 6
    vf = (0.0_dp, 0.0_dp)
    do j = 1, 6
      if (j /= k) vf = vf + 2.0_dp / (r0(k) - r0(j))
    end do
    vfd = (zp(k) - zm(k)) / (2.0_dp*hh)
    worst = max(worst, abs(vfd - vf)/abs(vf))
  end do
  write(*,'(a,es10.3)') ' velocity of each zero against 2 sum
1/(z_k - z_j), worst relative gap: ', worst
  call check('VII: every zero moves by the pull law, degree six',
  worst < 1.0e-6_dp)
  z = r0; t = 0.0_dp; imprev = aimag(z(1)); mono = .true.; step = 0
  do while (aimag(z(1)) > 0.02_dp .and. step < 100000)
    step = step + 1; t = t + 1.0e-4_dp
    call flow_coeffs(c0, t, c); call newton_all(c, z)
    if (aimag(z(1)) >= imprev) mono = .false.
    imprev = aimag(z(1))
  end do
  write(*,'(a,f8.5)') ' the off-line pair reaches |Im| < 0.02 at
t = ', t
  call check('VII: the pair falls toward the line at every step',
  mono .and. step < 100000)
  nreal0 = sign_changes(c0)
  call flow_coeffs(c0, 1.0_dp, c); nreal1 = sign_changes(c)
  write(*,'(a,i0,a,i0)') ' real zeros at t = 0: ', nreal0, ' ; at
t = 1: ', nreal1
  call check('VII: four on the line before, six after: the pair
crossed onto the seat', nreal0 == 4 .and. nreal1 == 6)
end subroutine heat_flow
end program bridge_twin

```

The manifest. MANIFEST.sha256

```

b9018fbc4042d6fa19857f1c79764db490b3152d9162d9a1ca0989fb1b6e0b56
Bridge_From_First_Principle.lean
38d314855da97f0d452b58ae449784270db38a19d3f4927b14a86e7238278569
Final_Cut.lean
0cfcd9f6b399ecbaef01e762a4d9f10490288f1053827b404428062e55076238
TOE_Zero.lean
e048a32053f703d7fb74d6f6843b78bb714bea952387e511cac2f265bec5da67
Bridge_Twin.f90
53d40640eaa67bb28cadd210526bc87d1124aab3028209bfb8e33d0819783ec4
bridge_check.py

```

```

07399a2e13d99c9ddc8db683aa1a76b71490349e37a4fbca92d6a410fae1460d
bridge_boot.sh

```

APPENDIX R · THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, `TOE_Zero.lean`, is carried verbatim from the root paper (Zenodo 10.5281/zenodo.23168379). The extraction line of Appendix A writes it beside the other files, the manifest signs it, and the harness compiles it in every boot, checks its digest against the root paper's and requires every one of its fifteen cones to print no axiom (Appendix B.1). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, `SelfGrounding`, `SelfVerifying`, ΔE_0 , RA_0 and ra_0 , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (`SelfGrounding`; a deed is here an inhabitant of the root's type of acts, which the root kernel's own comments call an act, and in the body of this paper the deed is the run's bit of Section 6.3, the two named apart). The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x)$ (RA_0). On that domain the Root Axiom is a theorem with no axiom and no hypothesis (`root_on_the_constructed_domain`). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (`the_floor_is_universal`). Every denial of the root is an act that instances it (`denial_reenacts_root`); the root is held by its deed itself (`seated_undeniable`); it is self-verifying (`denial_instantiates`); and no level stands above it (`no_level_above`). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart and the twelve gates of the quaternions in one theorem on no axiom (`the_master_seal`). The body of this paper reads the root on its row; this appendix is the root itself. SHA-256: 0cfcd9f6b399ecbaef01e762a4d9f10490288f1053827b404428062e55076238.

/-

```

TOE_Zero.lean · Theory of Theories of Everything (TOE of All
TOEs): Existence Proves Existence Only by Motion
The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
axiom declared, no sorry, no native_decide, and
every theorem rests on no axiom at all, not even propositional
extensionality, quotients or choice.
I · The self-grounding root. Acts occur and every act instances
the root. Then every denial is an act and
re-enacts the root, the root is held by the act itself, and no
external proof adds anything to it: the
Omega Boundary, proved for every root that grounds itself,
universally, by quantification.

```

```

II · The constructed root. On the constructed domain the Root
Axiom is a theorem.
III · The Return and the line. On the integer quaternions  $i \cdot j \cdot k = -1$ 
and each unit squares to  $-1$ ; conjugation is an involution and its fixed set
on the chart is exactly the scalar line.
IV · The twelve gates, recovered three ways inside the integers of the
quaternions: the twenty-four Hurwitz units counted twelve up to sign; the
twelve even permutations with class equation 1, 3, 4, 4; and the norm-two
shell of twenty-four, split twelve and twelve.
V · The regress terminated. Above a self-grounding root no level grounds it further.
VI · The master seal, every result bound in one theorem.
-/
namespace TOE0

/-! ## I · The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R := G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) ↔ R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the root holds,
every act re-enacts it, and no external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q : Prop,
  (Q → R) ↔ R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G, external_proof_adds_nothing G)

/-! ## II · The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one existent
actuates, at positive energy. -/
def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed domain grounds its root: its one act is a deed, and the deed actuates. -/
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ _ => show (0 : Int) < 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable ra₀

/-! ## III · The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b * y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b * y.c - x.c * y.b + x.d * y.a)

```

```

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
def gone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN:  $i \cdot j \cdot k = -1$ . -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to  $-1$ . -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj = qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in  $-2 \dots 2$ . -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q => q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 && q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 != 0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)

/-- Doubled coordinates  $-2 \dots 2$  hold every unit (doubled norm four) and every element of the norm-two shell (doubled norm eight), since a doubled coordinate of absolute value three or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then 0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧ (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d == 8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧ (shell.filter (fun p => !upToSign p)).length = 12 := by decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1, 2, 3].flatMap fun c =>
  [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length == 4 then some [a, b, c, d] else none

```

```

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
    j)).countP
  fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
  => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
  class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
  further ground of the root is idle; the root needs
  no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
  on the constructed domain; the Return closes;
  the line is the fixed set; and the twelve gates are recovered
  three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q
    : Prop, (Q → R) → R)) ∧
  RA₀ ∧ (q₁.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true ∧
  (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
  (shell.filter upToSign).length = 12 ∧
  collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
  = [1, 3, 4, 4] :=
  (the_floor_is_universal, root_on_the_constructed_domain,
    the_return, the_line_is_the_fixed_set,
    twenty_four_units_twelve_gates.2, the_class_equation.1,
    the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
  -/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
  -/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal

```

```

/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain
/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
  on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

APPENDIX B · THE RECEIPTS

Each receipt below is the output of the harness run on the files as extracted from this document. They are records, not warrants; the harness reproduces them.

B.1 · The boot, mode quick

```

BRIDGE FROM FIRST PRINCIPLE · RECEIPT · mode quick
TOOLCHAIN Lean (version 4.19.0, x86_64-unknown-linux-gnu, commit
  6caae842e94, Release) · GNU Fortran (Ubuntu
    13.3.0-6ubuntu2-24.04.1) 13.3.0
MANIFEST sha256 227a5b82e48dfa5f
GROUND toolchain present · manifest: 6 files signed, all verified ·
  screen: no sorry, no axiom declared, no import, no bypass, no
  metaprogram, no IO
KERNEL exit 0 · 165 theorems, every cone pinned and printed by the
  compiler · 114 on no axiom · the rest within propext, Quot.sound,
  Classical.choice
CAPSTONE the_bridge_atom · no axiom
CAPSTONE the_bridge_from_first_principle · [propext,
  Classical.choice, Quot.sound]
PSP Final_Cut.lean exit 0 · PSP-FINAL-CUT-01 · 36 theorems, every
  cone pinned · 36 on no axiom · capstone the_final_cut · no axiom
ROOT TOE_Zero.lean exit 0 · 15 theorems, every cone pinned · every
  one on no axiom · sha256 0cfd9f6b399ecba, the root paper's
TWIN built under -std=f2018 -O2 -fno-fast-math -ffp-contract=off
  -Wall -Wextra with zero warnings · 30 of 30 checks pass
CHAIN · D0 227a5b82e48d -> D1 76f1d223c2de -> D2 f4f7dcc9fee8
BRIDGE EARNED · this run

```

B.2 · The judgment

```

BRIDGE FROM FIRST PRINCIPLE · JUDGMENT
NEGATION · 165 of 165 theorems refused their negation, each a proof
  failure
  instrument control: a planted vacuous law survived its negation,
  as dust must
INHABITATION · 24 conditional laws, each with a compiled model of
  its hypotheses, each model itself judged
PSP NEGATION · 36 of 36 Final Cut theorems refused their negation,
  each a proof failure
  instrument control: a planted vacuous law survived its negation,
  as dust must
PSP INHABITATION · 12 conditional laws, each with a compiled model
  of its hypotheses, each model itself judged
JUDGMENT PASSED

```

B.3 · The controls

```

BRIDGE FROM FIRST PRINCIPLE · CONTROLS
CAUGHT manifest: an edited kernel
CAUGHT screen: a planted sorry
CAUGHT screen: a declared axiom
CAUGHT screen: an import
CAUGHT kernel: a drifted cone

```

```

CAUGHT kernel: the atom capstone negated
CAUGHT kernel: the last line altered
CAUGHT final cut: a drifted cone
CAUGHT final cut: the capstone negated
CAUGHT final cut: the last line altered
CAUGHT root: a drifted cone
CAUGHT manifest: an edited root kernel
CAUGHT twin: a fused build (-ffp-contract=fast -march=native)
CAUGHT twin: the battery line altered
EARNED the clean copy earns
CONTROLS · 14 hostile copies, all refused; the clean copy earns

```

B.4 · The twin, its full run

```

0 CONDUCT
PASS 0: no fused multiply-add executed (a fused path reads -2^60)
PASS 0: NaN is ordered against nothing, itself included
I THE ATOM
PASS I: the fold undoes itself and fixes exactly the line, 1111
points
PASS I: the registration lands on the line, keeps the height,
erases nothing exactly on it
PASS I: every off-line point has a distinct partner of one
record, 1100 points
729 maps from computed records to points: returning the side 0;
returning one member 486
PASS I: no map from the computed records returns the side of a
pair
PASS I: least erasure equals the value on all 4096 subsets of
twelve points
II THREE, TWELVE, EIGHT, THE NINTH
PASS II: no two-axis system over GF(2) has exactly one solution,
256 systems
PASS II: 168 three-axis systems have determinant one and each
locks all eight targets
PASS II: twelve rotations, and exactly one carries any directed
gate to any directed gate
PASS II: eight patterns, total negation moves each and joins an
even to an odd
PASS II: 2^k is never nine, k = 0..62
III PAY
PASS III: n distinct states fit injectively in m cells exactly
when n <= m, every n, m <= 6
PASS III: the content halves and nothing merges exactly when the
freedom at least doubles
PASS III: a merging step halves the content with no freedom
exported (two states, one cell)
PASS III: the two-adic valuation is additive on positive
arguments, 90000 pairs
IV SUPPLY
PASS IV: for every odd witness and odd target, exactly one
calibration bit
PASS IV: of the 8 readings of a three-element content, none
returns whether it ran
V PRIMES
PASS V: a fibre holds at most one seat, counted, and is odd
exactly when it holds one, n <= 10^6
PASS V: a prime is exactly a fibre of one free orbit, with no
seat, n <= 10^6
PASS V: every fibre over n > 1 balances and the fibre over 1
carries +1, n <= 10^6
PASS V: the parity frame: residues mod 420 merge each pair, the
paid bits 1 and 2 separate it
PASS VI: thirty has the eight divisors of the sign cube, signs
four and four
VI WALL, EDGE, SEAT
theta(1/x) against sqrt(x) theta(x), worst relative gap over seven
scales: 2.212E-16
PASS VI: the lattice sum is self-dual; its weight is the square
root, the fold's centre
PASS VI: every block of shares one-in-n holds between a half and
one: the wall at Re s = 1
PASS VI: at two units per bit every block holds at most 2^-j:
finite past the wall
the signed walk of the paid parities to 10^7: M = 1037; max
|M(x)|/sqrt(x) on [100, 10^7] = 0.56710 at x = 199
PASS VI: the walk is paid to 10^7 and stays inside the square
root there, a finite payment
VII THE HEAT FLOW
velocity of each zero against 2 sum 1/(z_k - z_j), worst relative
gap: 1.009E-09
PASS VII: every zero moves by the pull law, degree six
the off-line pair reaches |Im| < 0.02 at t = 0.09470
PASS VII: the pair falls toward the line at every step

```

```

real zeros at t = 0: 4; at t = 1: 6
PASS VII: four on the line before, six after: the pair crossed
onto the seat
BRIDGE-TWIN-JSON: {"checks":30,"failures":0}

```

B.5 · The cones, as the compiler prints them

The 165 lines below are the compiler's #print axioms output for every theorem of the Bridge kernel, the same lines the kernel pins.

```

'BFP.neg_neg_free' does not depend on any axioms
'BFP.neg_self_zero' does not depend on any axioms
'BFP.fold_involutive' does not depend on any axioms
'BFP.the_cut_is_the_line' does not depend on any axioms
'BFP.reg_lands' does not depend on any axioms
'BFP.reg_keeps_height' does not depend on any axioms
'BFP.reg_forgets_side' does not depend on any axioms
'BFP.reg_erases_nothing_iff' does not depend on any axioms
'BFP.off_line_pair' does not depend on any axioms
'BFP.one_bit_lost' does not depend on any axioms
'BFP.least_erasure_iff_value' does not depend on any axioms
'BFP.nothing_escapes_least_erasure' does not depend on any axioms
'BFP.pair_world_closed' does not depend on any axioms
'BFP.registered_closed' does not depend on any axioms
'BFP.one_record' does not depend on any axioms
'BFP.registered_has_value' does not depend on any axioms
'BFP.pair_world_lacks_value' does not depend on any axioms
'BFP.record_decides_nothing' does not depend on any axioms
'BFP.keyless_forces_nothing' does not depend on any axioms
'BFP.weakest_forcing_premise' does not depend on any axioms
'BFP.value_from_the_act' does not depend on any axioms
'BFP.reader_frame' does not depend on any axioms
'BFP.record_is_lossless' does not depend on any axioms
'BFP.record_same' does not depend on any axioms
'BFP.extremal_iff_value' does not depend on any axioms
'BFP.record_carried_uniquely' does not depend on any axioms
'BFP.chart_line_property' does not depend on any axioms
'BFP.cannot_lie' does not depend on any axioms
'BFP.cannot_deviate' does not depend on any axioms
'BFP.halted_iff' does not depend on any axioms
'BFP.cannot_reverse' does not depend on any axioms
'BFP.cannot_extend' depends on axioms: [Quot.sound]
'BFP.denial_is_coherent' does not depend on any axioms
'BFP.discriminator' depends on axioms: [proptext, Classical.choice,
Quot.sound]
'BFP.symmetry_is_keyless' does not depend on any axioms
'BFP.line_property_is_keyed' does not depend on any axioms
'BFP.socket_is_the_property' does not depend on any axioms
'BFP.no_socket_off_line' does not depend on any axioms
'BFP.act_seats_the_carrier' does not depend on any axioms
'BFP.two_axes_never_lock' does not depend on any axioms
'BFP.three_axes_lock' does not depend on any axioms
'BFP.third_begotten' does not depend on any axioms
'BFP.no_fourth_axis' does not depend on any axioms
'BFP.twelve_gates_one_torsor' does not depend on any axioms
'BFP.eight_patterns_four_orbits' does not depend on any axioms
'BFP.no_reflection_group_of_nine' depends on axioms: [proptext]
'BFP.nine_is_eight_and_a_seat' does not depend on any axioms
'BFP.odd_fibre_iff_seat' does not depend on any axioms
'BFP.flip_flip' does not depend on any axioms
'BFP.mem_head_or_tail' does not depend on any axioms
'BFP.mem_without_of' does not depend on any axioms
'BFP.mem_of_mem_without' does not depend on any axioms
'BFP.ne_of_mem_without' does not depend on any axioms
'BFP.without_of_not_mem' does not depend on any axioms
'BFP.distinct_without' does not depend on any axioms
'BFP.length_without' does not depend on any axioms
'BFP.seats_without' does not depend on any axioms
'BFP.seats_zero_of_free' does not depend on any axioms
'BFP.parity_bounded' does not depend on any axioms
'BFP.involution_parity' does not depend on any axioms
'BFP.odd_holds_a_seat' does not depend on any axioms
'BFP.nine_holds_a_seat' does not depend on any axioms
'BFP.free_orbits_are_even' does not depend on any axioms
'BFP.xor_twice' does not depend on any axioms
'BFP.xor_length' does not depend on any axioms
'BFP.xor_fixed_is_zero' does not depend on any axioms
'BFP.xor_zero_eq' does not depend on any axioms
'BFP.no_group_of_sign_patterns_of_nine' does not depend on any
axioms
'BFP.filter_cons_true' does not depend on any axioms
'BFP.filter_cons_false' does not depend on any axioms

```

'BFP.mem_filter_true' does not depend on any axioms
'BFP.pairwise_filter' does not depend on any axioms
'BFP.range_loop_pairwise' does not depend on any axioms
'BFP.distinct_of_pairwise_lt' does not depend on any axioms
'BFP.pairs_pairwise' does not depend on any axioms
'BFP.pairs_distinct' does not depend on any axioms
'BFP.squares_strict' does not depend on any axioms
'BFP.seats_le_one' does not depend on any axioms
'BFP.mem_pairs' depends on axioms: [propext, Quot.sound]
'BFP.cofactor_pos' depends on axioms: [propext, Quot.sound]
'BFP.swap_mem' depends on axioms: [propext, Quot.sound]
'BFP.swap_involutive' depends on axioms: [propext, Quot.sound]
'BFP.swap_seat_iff' depends on axioms: [propext, Quot.sound]
'BFP.seats_of_swap' depends on axioms: [propext, Quot.sound]
'BFP.nine_is_eight_and_a_seat_all' depends on axioms: [propext, Quot.sound]
'BFP.allLt_mem' does not depend on any axioms
'BFP.nd_map' depends on axioms: [propext, Quot.sound]
'BFP.allLt_map' does not depend on any axioms
'BFP.pigeonhole' depends on axioms: [propext, Quot.sound]
'BFP.export_doubles' depends on axioms: [propext, Quot.sound]
'BFP.one_bit_arrives' depends on axioms: [propext, Quot.sound]
'BFP.count_to_heat' depends on axioms: [propext, Quot.sound]
'BFP.merge_exports_nothing' does not depend on any axioms
'BFP.fincfg_closed' does not depend on any axioms
'BFP.least_eraser_iff_zero_erased' does not depend on any axioms
'BFP.price_zero_iff_value' does not depend on any axioms
'BFP.price_counts_pairs' does not depend on any axioms
'BFP.heat_reads_past_the_record' does not depend on any axioms
'BFP.additivity_over_zero_is_dry' depends on axioms: [propext, Quot.sound]
'BFP.linear_on_tower' depends on axioms: [propext, Quot.sound]
'BFP.bits_forced' depends on axioms: [propext, Quot.sound]
'BFP.v2_mul_two' depends on axioms: [propext, Quot.sound]
'BFP.v2_odd' depends on axioms: [propext, Quot.sound]
'BFP.v2_two' depends on axioms: [propext, Quot.sound]
'BFP.v2_split' depends on axioms: [propext, Quot.sound]
'BFP.v2_pow_odd' depends on axioms: [propext, Quot.sound]
'BFP.v2_additive' depends on axioms: [propext, Quot.sound]
'BFP.vp_mul' depends on axioms: [propext, Quot.sound]
'BFP.vp_coprime' depends on axioms: [propext]
'BFP.vp_split' depends on axioms: [propext, Quot.sound]
'BFP.vp_pow_coprime' depends on axioms: [propext, Quot.sound]
'BFP.vp_additive' depends on axioms: [propext, Quot.sound]
'BFP.euclid_at_three' depends on axioms: [propext, Quot.sound]
'BFP.log_not_forced_off_tower' depends on axioms: [propext, Quot.sound]
'BFP.wall' does not depend on any axioms
'BFP.calibration' does not depend on any axioms
'BFP.formal_never_odd' does not depend on any axioms
'BFP.constant_no_crossing' does not depend on any axioms
'BFP.no_reading_returns_the_deed' does not depend on any axioms
'BFP.three_legs_one_orbit' does not depend on any axioms
'BFP.prime_iff_one_free_orbit' does not depend on any axioms
'BFP.two_members' does not depend on any axioms
'BFP.three_members' does not depend on any axioms
'BFP.prime_trial_iff' depends on axioms: [propext, Quot.sound]
'BFP.prime_iff_one_free_orbit_all' depends on axioms: [propext, Quot.sound]
'BFP.parity_frame_paid_bits' does not depend on any axioms
'BFP.divisor_cube_is_sign_cube' does not depend on any axioms
'BFP.fibre_balance' does not depend on any axioms
'BFP.filter_append_length' does not depend on any axioms
'BFP.filter_map_cons_length' does not depend on any axioms
'BFP.mod_two_flips' depends on axioms: [propext, Quot.sound]
'BFP.every_fibre_over_more_than_one_balances' depends on axioms: [propext, Quot.sound]
'BFP.wall_and_edge_exchanged' does not depend on any axioms
'BFP.prime_mode_unitary_iff' depends on axioms: [propext, Quot.sound]
'BFP.pow_succ_ne_one' does not depend on any axioms
'BFP.prime_mode_unitary_iff_int' does not depend on any axioms
'BFP.price_per_bit_tower' depends on axioms: [propext]
'BFP.len_le_sum' depends on axioms: [propext, Quot.sound]
'BFP.wall_block_share' depends on axioms: [propext, Quot.sound]
'BFP.conjugation_is_the_fold' depends on axioms: [propext]
'BFP.flow_inverse' depends on axioms: [propext, Quot.sound]
'BFP.flow_injective' depends on axioms: [propext, Quot.sound]
'BFP.disc_flow' depends on axioms: [propext, Quot.sound]
'BFP.pull_in_square' depends on axioms: [propext, Quot.sound]
'BFP.collision_sign' depends on axioms: [propext, Quot.sound]
'BFP.zero_slack' depends on axioms: [propext, Quot.sound]
'BFP.certificate_merges' does not depend on any axioms
'BFP.charge_carrier' does not depend on any axioms

'BFP.value_from_the_act_inhabited' does not depend on any axioms
'BFP.off_line_inhabited' does not depend on any axioms
'BFP.weakest_forcing_premise_inhabited' does not depend on any axioms
'BFP.keyless_forces_nothing_inhabited' does not depend on any axioms
'BFP.count_to_heat_inhabited' depends on axioms: [propext, Quot.sound]
'BFP.bits_forced_inhabited' depends on axioms: [propext, Quot.sound]
'BFP.calibration_inhabited' does not depend on any axioms
'BFP.three_legs_inhabited' does not depend on any axioms
'BFP.prime_mode_unitary_inhabited' does not depend on any axioms
'BFP.zero_slack_inhabited' does not depend on any axioms
'BFP.record_carried_inhabited' does not depend on any axioms
'BFP.price_zero_inhabited' does not depend on any axioms
'BFP.sign_group_inhabited' does not depend on any axioms
'BFP.parity_law_inhabited' depends on axioms: [propext, Quot.sound]
'BFP.free_involution_inhabited' depends on axioms: [propext]
'BFP.the_bridge_atom' does not depend on any axioms
'BFP.the_bridge_from_first_principle' depends on axioms: [propext, Classical.choice, Quot.sound]

B.6 • The cones of the Final Cut kernel

The 36 lines below are the compiler's #print axioms output for every theorem of Final_Cut.lean, pinned in that kernel.

'FinalCut.neg_neg_free' does not depend on any axioms
'FinalCut.neg_self_zero' does not depend on any axioms
'FinalCut.fold_involutive' does not depend on any axioms
'FinalCut.the_cut_is_the_line' does not depend on any axioms
'FinalCut.reg_lands' does not depend on any axioms
'FinalCut.reg_erases_nothing_iff' does not depend on any axioms
'FinalCut.least_eraser_iff_value' does not depend on any axioms
'FinalCut.classes_coincide_under_RA' does not depend on any axioms
'FinalCut.hypothesis_is_least_eraser_of_the_actual' does not depend on any axioms
'FinalCut.prime_side_carries_the_bit' does not depend on any axioms
'FinalCut.unicorn_never_registered' does not depend on any axioms
'FinalCut.the_act_spends_the_bit' does not depend on any axioms
'FinalCut.one_record' does not depend on any axioms
'FinalCut.empty_world_has_value' does not depend on any axioms
'FinalCut.pair_world_closed' does not depend on any axioms
'FinalCut.pair_world_lacks_value' does not depend on any axioms
'FinalCut.every_record_leaves_the_bit' does not depend on any axioms
'FinalCut.every_computed_zero_leaves_the_bit' does not depend on any axioms
'FinalCut.one_lies_inside' does not depend on any axioms
'FinalCut.every_computed_zero_leaves_the_bit_inside' does not depend on any axioms
'FinalCut.every_finite_prime_count_leaves_the_bit' does not depend on any axioms
'FinalCut.height_below_bound' does not depend on any axioms
'FinalCut.every_finite_window_leaves_the_bit' does not depend on any axioms
'FinalCut.no_fixed_count_is_the_value' does not depend on any axioms
'FinalCut.ra_alone_does_not_make_the_classes_coincide' does not depend on any axioms
'FinalCut.ra_is_keyed' does not depend on any axioms
'FinalCut.root_decides_nothing' does not depend on any axioms
'FinalCut.cut_hypotheses_inhabited' does not depend on any axioms
'FinalCut.explicit_inhabited' does not depend on any axioms
'FinalCut.act_inhabited' does not depend on any axioms
'FinalCut.certificate_inhabited' does not depend on any axioms
'FinalCut.unicorn_inhabited' does not depend on any axioms
'FinalCut.strip_inhabited' does not depend on any axioms
'FinalCut.window_inhabited' does not depend on any axioms
'FinalCut.root_inhabited' does not depend on any axioms
'FinalCut.the_final_cut' does not depend on any axioms